

LABORATORIO ANAWS

Francesco Taverna

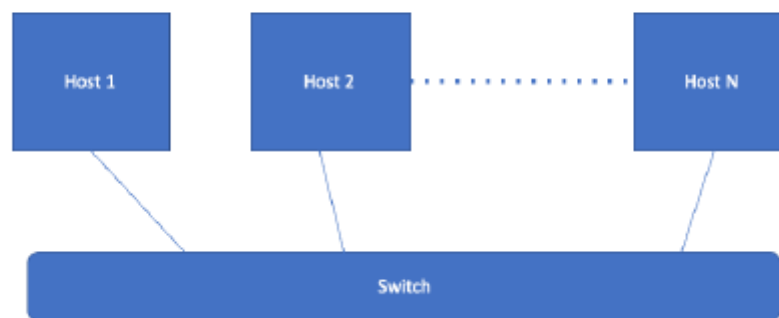
Indice

Indice.....	1
Server Virtualization and Networking.....	2
Host-only network.....	4
Test comunicazione macchine virtuali	6
KVM.....	8
Test NAT.....	12
Test isolated.....	13
Test routed	14
Abbandono di Linux Bridge	21
OpenvSwitch	22
Containers	28
Software Define Networking (SDN)	40
OpenFlow	57
Evoluzione dell'SDN	68
Automazione e Programmazione della Rete.....	81
Data Formats, Models, Templates	90
Bare Metal Switching and Open-Source Networking	104
Cisco IOS modes	110
Network configurations.....	114
Esempio pratico.....	117
Peering BGP con Arista.....	132

Server Virtualization and Networking

Perché parliamo del networking in questo contesto?

Network Switching



Fino ad ora si è sempre parlato del lato fisico, dove i vari host sono connessi ad uno switch per comunicare fra loro, il tutto a livello fisico con cavi che arrivano nel cabinet. In questa configurazione abbiamo tanto hardware, in particolare gli switches che sono stati manualmente configurati per permettere agli host di comunicare. Ovviamente essi possono essere anche configurati per essere connessi alla rete Internet. Questa configurazione risulta statica: configuri i cavi e l'hardware e non si hanno problemi a meno di problemi ai cavi o ai dispositivi fisici.

Negli ultimi anni si è cercato di avere una transizione verso risorse **virtuali**, perché i client hanno iniziato a chiedere risorse on-demand: per esempio in una startup posso contattare una azienda per ottenere la *computer capacity* che necessito, viene sostanzialmente affittata invece che comprarla. La grande azienda che fornisce questo servizio prende una farm di server e, quando un client richiede delle risorse, essa le alloca risorse unicamente per lui. In questo caso, non siamo in un contesto statico, poiché un client può richiedere molta più potenza computazionale se necessario.

Virtual Switching



Invece che avere molti computer avrò molte VM, oppure invece che le VM si potrebbero avere containers. Le VM possono essere ospitate in una singola macchina fisica o in più macchine, non importa.

Una VM è fondamentalmente un sistema operativo guest in esecuzione su un server host. Più VM sono connesse logicamente tra loro e all'interfaccia di rete tramite un **vSwitch**, e viene stabilita una connessione alla rete fisica del data center tramite un controller di interfaccia di rete (NIC). Questo

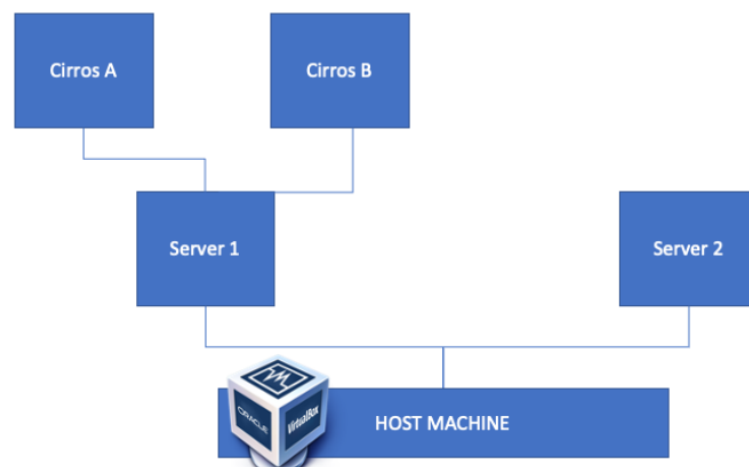
switch virtuale deve essere configurato per permettere la comunicazione tra loro le VM e/o verso l'esterno. Comunicare o **non** comunicare, perché anche si parla anche “non”? Perché alcune VM non devono comunicare in altre reti, ma solo nella loro rete. L'obiettivo è permettere l'interazione tra le VM.

Un tipico server di un data center cloud può avere una NIC Ethernet da 10 Gb che si collega alla CPU del server tramite un'interfaccia PCI Express (PCIe). Sia le VM che il vSwitch sono configurati e controllati da un programma hypervisor che viene eseguito anche sul server. Più VM devono condividere una singola porta fisica di rete, come ad esempio un controller di interfaccia di rete (NIC). L'hypervisor può implementare un vSwitch nel software: questo è un tipico approccio seguito da molti hypervisor.

Quali sono le challenge?

- La scalabilità, possono esserci tantissime VM in molti datacenter, puoi avere anche scenari misti in cui sono presenti non solo VM.
- Alta dinamicità: significa che non puoi andare a mettere mano macchina per macchina fisicamente sui dispositivi virtualizzati. Prima bisogna capire come configurare i dispositivi poi come automatizzare i processi di gestione.

Lab configuration



Ci saranno dati due VM server (1 e 2) che verranno usati come *host server*, per host si intende che essi ospiteranno le altre VM (innestate), essi sono chiamati **bare metal server** (come fossero hardware).

Poi istancieremo dinamicamente VM, containers, e le faremo comunicare, anche con altre VM in altri server. La virtualizzazione è ottima perché può essere innestata e le VM non si accorgono di nulla.

La parte più bassa nell'immagine sopra sarebbe il nostro computer, chiamata **Host Machine**.

Setting up the environment

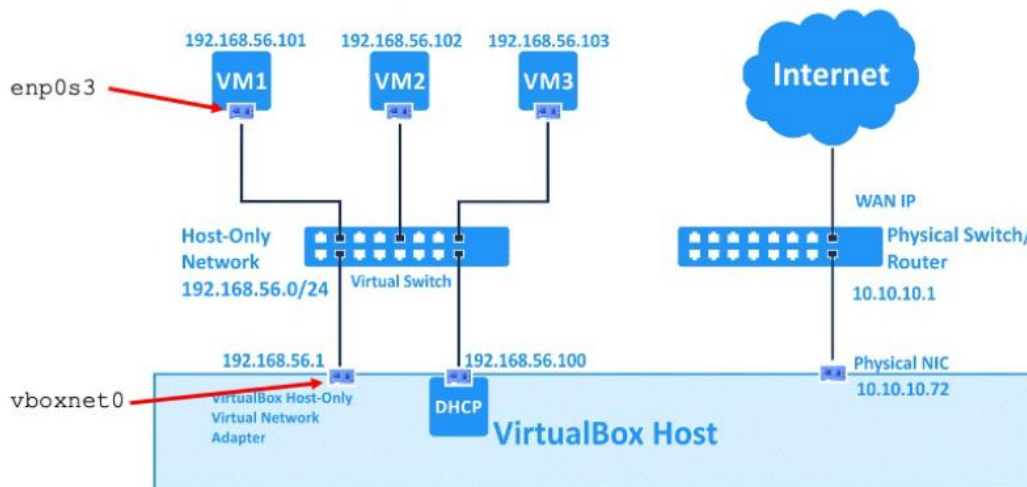
- Create a virtualbox network
- Create and start Server 1 using the image I provided you
- Configure a network card to work in **host-only** mode



Puoi configurare il network adapter della VM che possiederà molte schede, ma noi ne useremo 2: nella prima scheda di rete si avrà “solo host”, quella può parlare solo con l’host e con le macchine virtuali all’interno di quell’host.

Host-only network

Host-only network



Scheda “Host-only” per il networking: essa può essere utilizzata per creare una rete contenente l'host e un set di macchine virtuali, **senza la necessità dell'interfaccia di rete fisica nell'host.** Invece, un'interfaccia di rete virtuale, simile a un'interfaccia loopback, viene creata sull'host, fornendo connettività tra le macchine virtuali e l'host.

Quello presente nell’immagine è uno schema generale, cosa facciamo quando creiamo le due VM con che hanno una scheda host-only? Virtual box crea VM1 e VM2, crea una network card, crea un vSwitch all’interno dell’ambiente host e connette le network cards, poi connette il tutto all’host e a quello switch. Le due VM si comporteranno come fossero nella stessa rete, sono stati collegati dei “cavi virtuali”. Avremo anche il DHCP per fornire IP alle macchine virtuali che saranno IP nello stesso spazio di comunicazione. Questi cavi virtuali permettono la trasmissione di reali pacchetti. Nessuna

può raggiungere Internet perché la host machine ha la sua fisica NIC, ma non è connessa alla rete virtuale precedente, è tutto isolato rispetto all'esterno se non viene configurato diversamente.

Setting up the environment (2)

- Connect to Server 1 via SSH (*optional*, but will ease your work)
- Run `ifconfig -a` on both the host and the guest
- Verify the connectivity through pings between host and guest
- Repeat the process for student2

Esegui `ifconfig -a` per vedere le configurazioni delle interfacce.

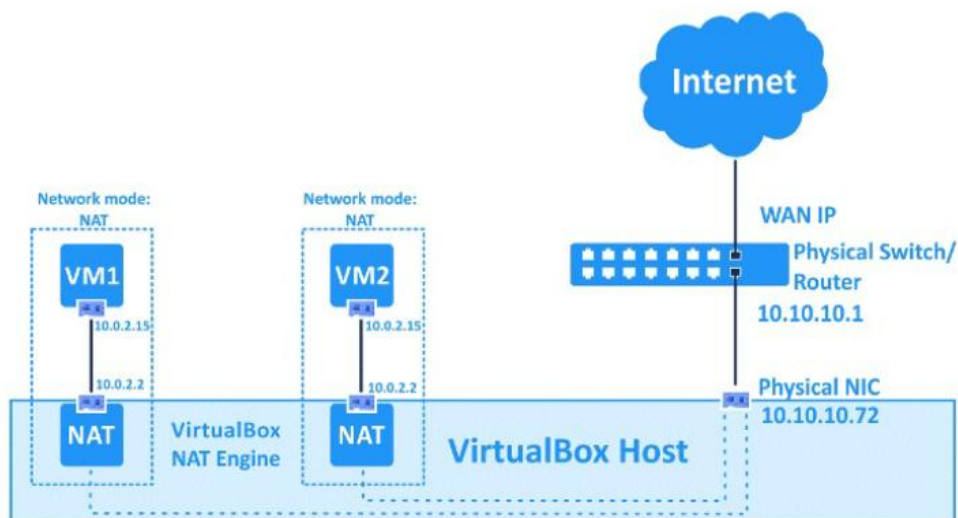
- The current configuration does not allow accessing external networks
- One (quick) way to give internet access to your VMs is to configure a new interface and set it to work in **NAT** mode.



- A "similar" effect can be obtained in UTM using the "Shared Network" mode

La nostra configurazione non ha accesso ad Internet, bisogna effettuare alcuni passaggi: bisogna utilizzare una nuova NI (scheda 2) che deve essere NAT (anch'essa sarà virtuale) che sfrutterà la "NAT mode".

NAT mode



Utilizzando la NAT mode stiamo aggiungendo una “Physical Interface” alla VM e la connettiamo al NAT interface dell’host (in realtà è tutto virtuale). Stiamo configurando anche l’host per ricevere i pacchetti attraverso indirizzi privati nella rete virtuale e grazie al NAT host si effettuerà una traduzione per avere quello pubblico permettendo l’accesso ad internet.

Check connectivity (from guest)

- Ping 8.8.8.8
- run `ifconfig -a`
- You might need to explicitly query the DHCP server:
`dhclient enp0s8`

emp0s, per esempio, può essere il nome di una network interface della VM

In generale, bisogna disegnare la rete prima di configurare. Nel nostro caso abbiamo un host, un Server 1 e un Server 2, vogliamo una rete condivisa connessa con l’host. La rete è la seguente: 192.168.129.0/24. Il Server 1 ad esempio può avere il suffisso “.2”, il Server 2 può avere “.3”, in questo modo essi credono di essere sulla stessa rete. La host machine è sulla stessa rete ma con ad esempio “.1” come IP dell’interfaccia.

Test comunicazione macchine virtuali

1. Per connettersi da terminale windows alla VM utilizzare il seguente comando:

```
ssh student@192.168.129.2
```

2. Fatto ciò, si ottiene la shell per controllare la VM. Bisogna fare lo stesso per la seconda.
3. Fatto questo uso il seguente comando per provare a pingare il Server 2 da Server 1:

```
ping 192.168.129.3
```

4. È possibile utilizzare il tool TCP Dump che permette di vedere i pacchetti che scorrono attraverso l’interfaccia della VM su cui si esegue. Il comando seguente va eseguito su Server 2:

```
tcpdump -n -i nome_interfaccia
```

5. Fare la stessa cosa con Server 1 e 2 invertiti.
6. Eseguire il seguente comando per vedere se si è connessi ad internet:

```
ping 8.8.8.8
```

7. Il comando non funziona, ora provare il seguente comando per vedere le interfacce presenti:

```
ifconfig
```

8. Si può notare che il sistema non sa quale interfaccia usare per andare all'esterno. Abbiamo due interfacce: una è quella della nostra rete, l'altra è quella di loopback, dunque perché non c'è una terza? Usando il seguente comando si ottengono anche le interfacce senza IP assegnato:

```
ifconfig -a
```

9. Si può quindi vedere che quella che permette l'accesso ad internet non ha l'IP, dunque bisogna forzare il sistema ad usare il DHCP con il seguente comando:

```
sudo dhclient enp0s8
```

10. Eseguire nuovamente il seguente comando:

```
ifconfig
```

11. Ora se eseguo il seguente comando posso vedere che sono connesso ad internet:

```
ping 8.8.8.8
```

12. Posso vedere se è "natted" o no? Come?

Is it NATed?

- Try a ping from student1 to 8.8.8.8
- Monitor the traffic through the "internet" interface of the host

```
tcpdump -i enp0s3 dst 8.8.8.8
```

- Do the same for NATed interface in student1

I frame di rete inviati dal sistema operativo guest vengono ricevuti dal motore NAT di Oracle VM VirtualBox, che estrae i dati TCP/IP e li reinvia tramite il sistema operativo host. Dal punto di vista di un'applicazione sull'host o da quello di un altro computer sulla stessa rete dell'host, sembra che i dati siano stati inviati dall'applicazione Oracle VM VirtualBox sull'host, utilizzando un indirizzo IP appartenente all'host. Oracle VM VirtualBox ascolta le risposte ai pacchetti inviati, li riconfeziona e li reinvia alla macchina guest sulla sua rete privata.

Abbiamo configurato le macchine insieme nella stessa network (host only network) e abbiamo verificato la comunicazione nella rete e verso l'esterno.

13. Possiamo fare nella macchina host il seguente comando (dopo aver chiamato *ping 8.8.8.8* su una VM):

```
sudo tcpdump -n dst 8.8.8.8
```

-n → per non tradurre tutto i nomi quindi sarà più veloce

```

student@student1:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
4 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=26.312ms
4 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=30.121ms
4 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=28.154ms
4 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=18.154ms
4 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=16.154ms
4 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=17.154ms
4 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=13.154ms
4 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=15.154ms
4 bytes from 8.8.8.8: icmp_seq=9 ttl=117 time=7.154ms
4 bytes from 8.8.8.8: icmp_seq=10 ttl=117 time=7.154ms
4 bytes from 8.8.8.8: icmp_seq=11 ttl=117 time=1.154ms
--- 8.8.8.8 ping statistics ---
11 packets transmitted, 11 received, 0.0% packet loss, time 1099ms
rtt min/avg/max/mdev = 13.522/82.650/303.530/109.5ms
student@student1:~$

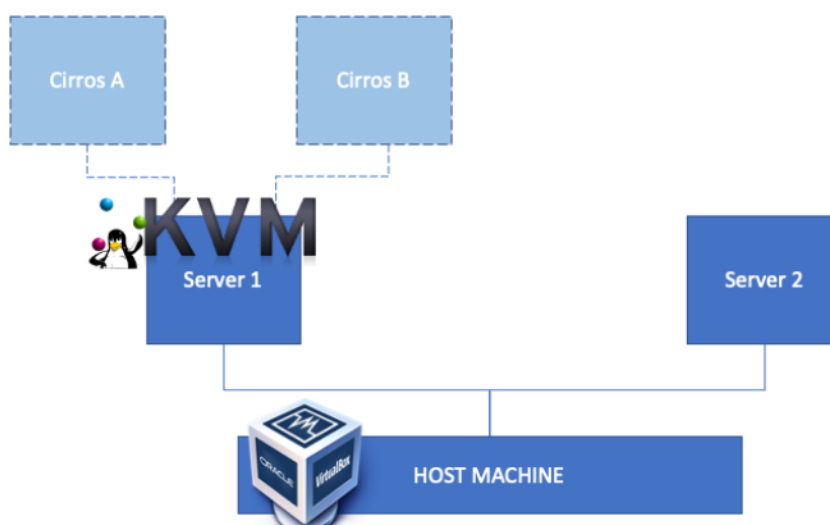
antonio@Antonio-Lavoro algoritmi % tcpdump -n dst 8.8.8.8
tcpdump: ioctl(SIOCIFCREATE): Operation not permitted
antonio@Antonio-Lavoro algoritmi % sudo tcpdump -n dst 8.8.8.8
tcpdump: data link type PKTAP
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on pktap, link-type PKTAP (Apple DLT_PKTAP), snapshot length 524288 bytes
09:58:24.308465 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 1, length 64
09:58:24.308517 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 1, length 64
09:58:25.312059 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 2, length 64
09:58:25.312108 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 2, length 64
09:58:26.312937 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 3, length 64
09:58:26.312996 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 3, length 64
09:58:27.315327 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 4, length 64
09:58:27.315375 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 4, length 64
09:58:28.317462 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 5, length 64
09:58:28.317514 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 5, length 64
09:58:29.318818 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 6, length 64
09:58:29.318869 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 6, length 64
09:58:30.321549 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 7, length 64
09:58:30.321593 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 7, length 64
09:58:31.322938 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 8, length 64
09:58:31.322983 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 8, length 64
09:58:32.326891 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 9, length 64
09:58:32.326935 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 9, length 64
09:58:33.331883 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 10, length 64
09:58:33.331930 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 10, length 64
09:58:34.334877 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 11, length 64
09:58:34.334929 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 11, length 64
09:58:35.339496 IP 192.168.64.6 > 8.8.8.8: ICMP echo request, id 6, seq 12, length 64
09:58:35.339543 IP 131.114.88.36 > 8.8.8.8: ICMP echo request, id 41588, seq 12, length 64

```

Ora *tcpdump* ascolta tutto il traffico legato a quell'ip address che arriva all'interfaccia della macchina host. Vediamo quindi che l'indirizzo IP della macchina virtuale (che è legato alla "NATed" interface della macchina virtuale che ha chiamato ping) è legato ad un flusso verso 8.8.8.8 (in realtà va verso la macchina host, viene preso dal kernel, che applica l'IP "masquerading" che traduce quell'IP address 192.x.x.x in quello della mia macchina host 131.x.x.x per inviarlo verso internet). Vediamo però anche l'IP della macchina host che è legato ad un flusso verso l'esterno (internet) che sta "pingando" 8.8.8.8. Ci sono due messaggi quindi, uno verso l'host e uno verso l'esterno: quello che va verso l'esterno viene tradotto grazie al NAT.

KVM

Lab configuration



16

Usiamo S1 e S2 come bare metal machines, sopra di queste usiamo KVM per istanziare le VM, ci focalizzeremo sul networking.

Quick Recap: VM creation (x86)

```
sudo apt-get update
```

```
sudo apt-get install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virtinst
```

```
wget https://docenti.ing.unipi.it/a.virdis/cirros/Cirros.img
```

```
• mv Cirros.img /var/lib/libvirt/images/CirrosA.img
• cp /var/lib/libvirt/images/CirrosA.img
  /var/lib/libvirt/images/CirrosB.img
• sudo virt-install --name CirrosA --description 'CirrosA' --ram=512
  --vcpus=1 --os-type=Linux --os-variant=ubuntu18.04 --disk
  path=/var/lib/libvirt/images/CirrosA.img,bus=virtio,format=raw --
  graphic none --import
```

KVM è il modulo di virtualizzazione per il kernel Linux, funziona come Hypervisor. LibVirt è uno strumento open source per la gestione della virtualizzazione della piattaforma (crea e configura VM). KVM consente la creazione di VM, LibVirt le gestisce!

- Per velocizzare l'avvio di cirros: modifica `/etc/cirros-init/config`, quindi cambia `DATASOURCE_LIST` in `= "nocloud"`
- Attenzione a copiare il comando. Ci metterà un po'.
- Useremo una immagine (Cirros.img)
- Creeremo 2 macchine A e B sopra S1 e S2
- Si lanciano con il comando ultimo della slide che creerà e lancerà la virtual machine basandosi su quelle immagini

Quick Recap: useful commands [VM management]

```
virsh list                // list running VMs
virsh console CirrosA     // connects to the VM console
virsh start CirrosA       // starts/shutdown the VM
virsh shutdown CirrosA

virsh edit CirrosA        // edit VM configuration

virsh destroy CirrosA     // remove VM
virsh undefine CirrosA
```

Si usa `virsh` come interfaccia a linea di comando per configurare KVM.

La rimozione richiede quei due step.

Per uscire dalla console: `Ctrl + shift + ]` oppure `Ctrl + shift + 5`.

Quick Recap: useful commands [networking]

```
virsh net-create VNET.xml      // create and start a non-persistent network
virsh net-define VNET.xml      // create only

virsh net-start VNET
virsh net-destroy VNET
virsh net-undefine VNET

brctl show
virsh net-list // list available networks
virsh net-dumpxml [network_name]

virsh domifaddr Cirros        // list of interfaces of a running domain
virsh domiflist Cirros        // brief info of all interfaces of a given domain
virsh domifstat Cirros vnet0  // statistics of a given interface
```

I file di definizione XML delle reti virtuali persistenti sono archiviati nella directory `/etc/libvirt/<hypervisor>/networks/`. Inoltre, se la rete è contrassegnata come autostart, il collegamento simbolico al suo file XML viene creato nella sottodirectory `autostart/`.

Il comando `virsh net-create` ci permetterà di creare una rete **custom**, non proprio una rete ma una configurazione di una rete. Quando si crea una rete si può scegliere tra: `host only`, `natted`, `routed`.

- Il comando `brctl` viene usato per vedere dentro i bridge virtuali nel sistema kernel.
- `virsh net-dumpxml` per vedere le configurazioni

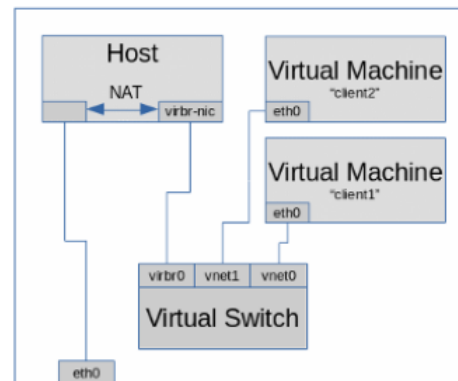
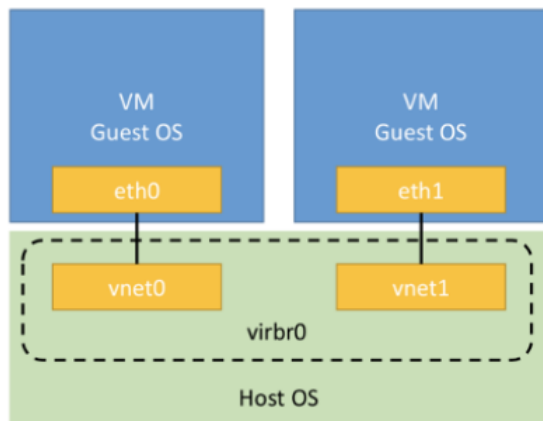
Custom network

- The VM-creation command we saw, connects the VM to the default “network”
- You can specify a custom network through the following option for the `virt-install` command:

```
--network network=custom
```

Prima la rete era quella di default, ora dobbiamo usare questo comando se vogliamo crearne un'altra. Noi modificheremo per ora solo la rete di default.

Linux-Kernel support (libvirt)



Quando installi il servizio *libvirt*, verrà creato automaticamente *virbr0*, che è uno switch di rete virtuale e tutte le macchine virtuali si collegheranno ad esso. Per impostazione predefinita, *virbr0* usa la modalità NAT, può fornire accesso a Internet tramite la modalità NAT. Sempre per impostazione predefinita, *virbr0* ha assegnato l'indirizzo 192.168.122.1 e può fornire il servizio DHCP per altre interfacce virtuali che si sono connesse ad esso. Per ogni VM, verrà creata un'altra interfaccia virtuale (sotto forma di *vnet0*, *vnet1*, ecc.) e collegata al bridge.

KVM usa il supporto NAT fornito dal kernel linux per la creazione dell'ambiente di rete virtuale, in particolare viene usato *libvirt*. *libvirt* supporta la configurazione di virtual interfaces e virtual bridges, quando creiamo le virtual machine creiamo delle virtual interfaces, creiamo un virtual bridge nel kernel dell'host machine (S1). Il virtual bridge permetterà alle macchine virtuali di comunicare fra loro e con l'host machine come visto prima. Il sistema creerà un virtual switch, abbiamo 3 network interfaces: *vrbr0* (che permette di connettere la macchina host con lo switch virtuale), *vnet1* e *vnet0*. Dal punto di vista delle VM sono solo interfacce non vedono che siano virtuali.

Quick Recap: network configuration

```
virsh net-dumpxml default
```

```
student@odl_server:~$ virsh net-dumpxml default
<network connections='1'>
  <name>default</name>
  <uuid>bbca96ec-78a3-465c-ab36-13cfc1047c29</uuid>
  <forward mode='nat'>
    <nat>
      <port start='1024' end='65535'>
      </port>
    </nat>
  </forward>
  <bridge name='virbr0' stp='on' delay='0'>
  <mac address='52:54:00:84:10:1f'>
  <ip address='192.168.122.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.122.2' end='192.168.122.254'>
      </range>
    </dhcp>
  </ip>
</network>
```

- Nat
- isolated
- Routed
- bridged

Con il comando indicato si può vedere la configurazione della rete. Si può vedere: il nome del bridge che si sta usando, il MAC address utilizzato per il bridge stesso, l'indirizzo IP dell'host machine (S1

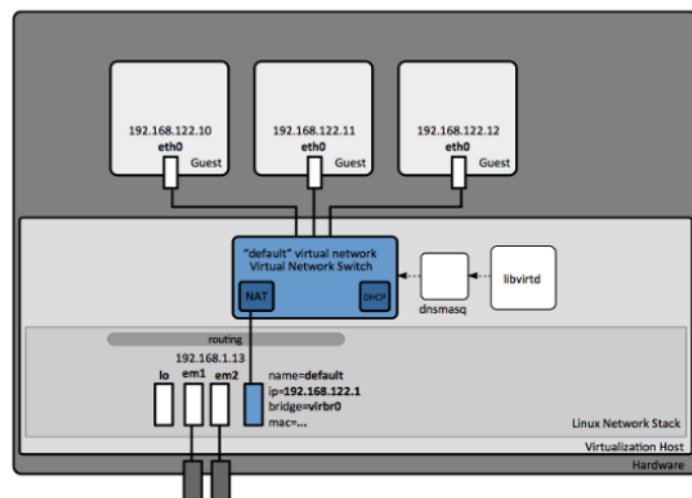
verso quella rete) e il range del DHCP. Abbiamo anche la modalità di forward, che in questo caso è nat.

- Isolated: macchine virtuali isolate da fuori
- Routed: nat è disabilitato, se la macchina cercherà di collegarsi con il fuori, l'host si comporterà come un router ip e farà l'ip routing.

Test NAT

Network Modes: NAT

```
brctl show
ifconfig -a
ip a s
iptables -S
```

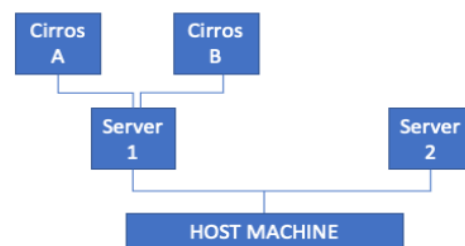


25

Possiamo vedere cosa *libvirt* sta facendo nella nostra macchina, si può vedere la creazione del bridge usando il comando *brctl show* e mostrerà il bridge creato nella macchina host. Si possono vedere le interfacce con *ifconfig* o *ip a s* e possiamo anche l'IP *masquerading* con il comando *iptables*.

Test: NAT

1. Create VMs CentosA, CentosB
2. Verify IPs are properly set
3. Do the following test



- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B
- Check traffic through Server1 (using *tcpdump*)

Se *tcpdump* va lento usare l'opzione *-n*.

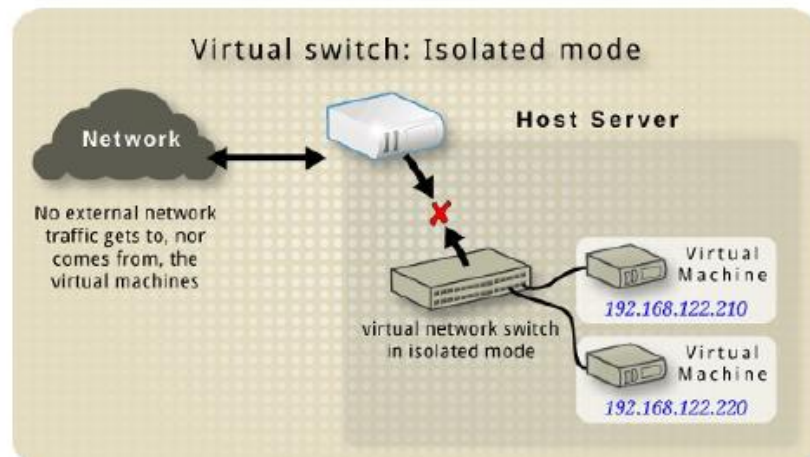
Quando si pinga il server 2, notare la differenza negli indirizzi IP dipendentemente dall'interfaccia che guardi (*virbr0* o *enp0s1*). Guarda anche gli indirizzi visti dal server 2.

Un esempio di comando tcpdump: `sudo tcpdump -n -i enp0s1 icmp`.

Bisogna creare la rete, le due macchine virtuali, verificare che gli indirizzi IP funzionano e fare i test nella slide.

Test isolated

Network Modes: isolated



Gli host all'interno di una rete virtuale isolata possono comunicare tra loro, ma non possono comunicare con gli host all'esterno della rete virtuale. Inoltre, gli host che non appartengono alla rete virtuale isolata non possono comunicare con gli ospiti all'interno.

L'altro esercizio prevederà di fare lo stesso ma con la modalità isolata.

Test: isolated Mode

CHECK VM

- `virsh edit CirrosA`
- search for section:

```
<interface type='network'>
  <source network='default'/>
  <mac address='00:16:3e:1a:b3:4a'/>
</interface>
```

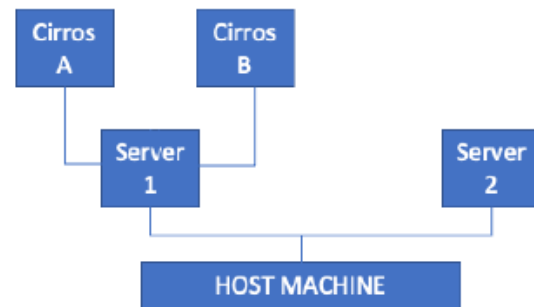
REMOVE NAT

- `# sudo virsh net-edit default`
- `//remove <forward .. section`
- `# sudo virsh net-destroy default ; sudo virsh net-start default`
- `# service libvirtd restart`
- `# sudo virsh console vm_name`

Rimuoveremo la sezione “forward nat” per configurarla in modalità isolata. Bisogna fare il controllo che il sistema è effettivamente isolato. Dopo aver rimosso `<forward .. A,B>` non sarà in grado di fare ping al server 2, poiché è fuori dalla loro rete

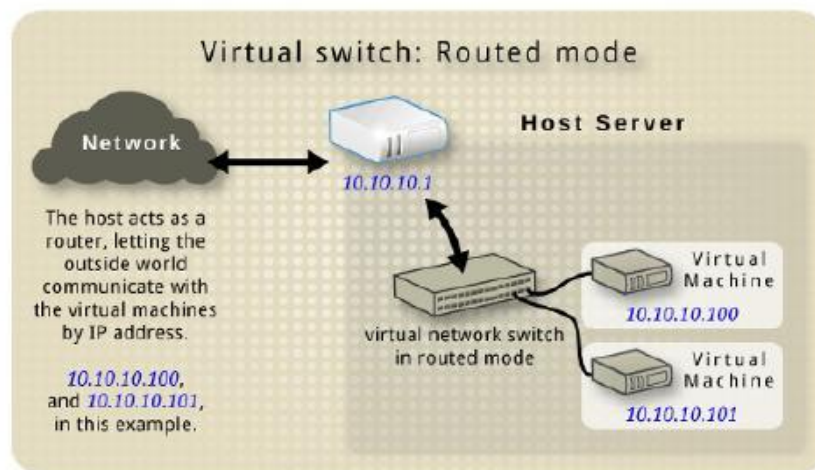
Test: isolated Mode

- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B



Test routed

Network Modes: routed



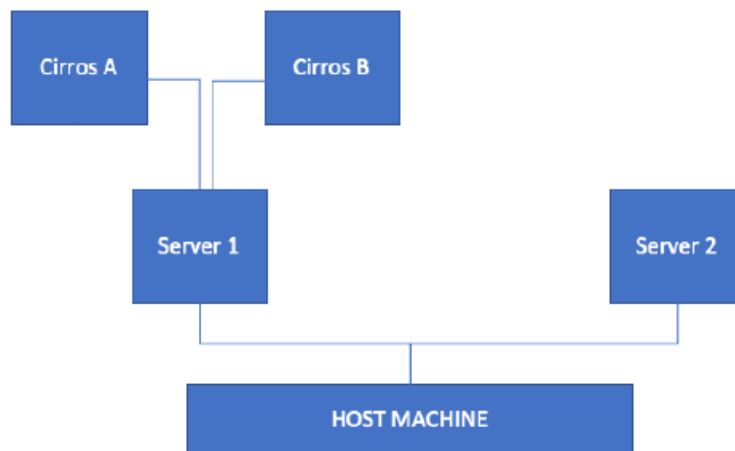
Provare a far funzionare ma non funzionerà. Perché?

Configure Route mode

- # sudo virsh net-edit default
- //ADD section <forward mode="route"/>
- # sudo virsh net-destroy default
- # sudo virsh net-start default
- # service libvirtd restart
- # sudo virsh console CirrosA

Vai nella rete dal punto di vista del KVM, modifichi la sezione forward da *nat* a *route*.

Does it work?



Ci sarà una rete condivisa tra Cirros A, Cirros B e Server 1 ma sto rimuovendo il masquerading, quindi il Server 1 funzionerà solo come un router. Provare a pingare da Cirros A/B il Server 2 e vedi che non funziona. Perché non funziona?

Controllo CirrosA, vedo che è connessa alla rete detta prima:

```
ssh: ifconfig: not found
$ ifconfig
eth0      Link encap:Ethernet  HWaddr 52:54:00:9C:06:FF
          inet addr:192.168.122.84  Bcast:192.168.122.255  Mask:255.255.255.0
          inet6 addr: fe80::5054:ff:fe9c:6ff/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:16  errors:0  dropped:2  overruns:0  frame:0
          TX packets:13  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1584 (1.5 KiB)  TX bytes:1816 (1.7 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Ora controlliamo le interfacce nel Server 1 con il comando: *ifconfig -a*

```
student@student1:~$ ifconfig -a
enp0s1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.129.2  netmask 255.255.255.0  broadcast 192.168.129.255
        inet6 fe80::d0db:e3ff:fea6:e15a  prefixlen 64  scopeid 0x20<link>
        ether d2:db:e3:a6:e1:5a  txqueuelen 1000  (Ethernet)
        RX packets 6908  bytes 468736 (468.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4537  bytes 617110 (617.1 KB)
        TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

enp0s2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.64.6  netmask 255.255.255.0  broadcast 192.168.64.255
        inet6 fe80::b82d:b4ff:fe6d:2f1e  prefixlen 64  scopeid 0x20<link>
        inet6 fdda:946b:9287:66a6:b82d:b4ff:fe6d:2f1e  prefixlen 64  scopeid 0x0<global>
        ether ba:2d:b4:6d:2f:1e  txqueuelen 1000  (Ethernet)
        RX packets 140710  bytes 209938629 (209.9 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 17518  bytes 1229651 (1.2 MB)
        TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

ep0s1 e *enp0s2* (noi abbiamo 3 e 8) sono quelle fisiche, una per andare su internet e l'altra per la rete virtuale.

```
virbr0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
    ether 52:54:00:8c:99:46 txqueuelen 1000 (Ethernet)
    RX packets 410 bytes 33624 (33.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 404 bytes 38612 (38.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

vnet1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::fc54:ff:fe9c:6ff prefixlen 64 scopeid 0x20<link>
    ether fe:54:00:9c:06:ff txqueuelen 1000 (Ethernet)
    RX packets 14 bytes 1858 (1.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 79 bytes 4970 (4.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Queste sono le altre due.

Ricordiamo che abbiamo un *Linux switch* fornito dal kernel linux, per dare un'occhiata a questo possiamo usare il comando: *brctl show*

```
student@student1:~$ brctl show
bridge name      bridge id                STP enabled  interfaces
virbr0           8000.5254008c9946        yes          vnet1
```

Possiamo vedere *vrbr0* che ha una interfaccia virtuale chiamata *vnet1* (che vediamo nell'immagine sopra). Con il comando *ip a s* vediamo il master di questa interfaccia che è *vrbr0*, ovvero il suo switch virtuale di cui lei fa parte:

```
6: vnet1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master virbr0 state U
NKNOW group default qlen 1000
    link/ether fe:54:00:9c:06:ff brd ff:ff:ff:ff:ff:ff
    inet6 fe80::fc54:ff:fe9c:6ff/64 scope link
    valid_lft forever preferred_lft forever
```

Con il comando *sudo virsh domiflist CirroSA* mostriamo le interfacce per una data VM:

```
student@student1:~$ sudo virsh domifaddr CirroSA
Name      MAC address      Protocol  Address
-----
vnet1     52:54:00:9c:06:ff  ipv4     192.168.122.84/24
```

Vediamo che *vnet1* è parte della rete default, parte del dominio in cui sta *CirroSA*. Vediamo anche l'indirizzo IP dell'interfaccia. L'interfaccia è virtuale ed è collegata al *linux switch* presente nell'host machine S1.

Con il comando *ip tables* vedo tutte le configurazioni di interfacce e switch:

```
student@student1:~$ sudo iptables -n -t nat -L
[sudo] password for student:
Chain PREROUTING (policy ACCEPT)
target     prot opt source                destination

Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source                destination
LIBVIRT_PRT all  --  0.0.0.0/0             0.0.0.0/0

Chain LIBVIRT_PRT (1 references)
target     prot opt source                destination
RETURN     all  --  192.168.122.0/24      224.0.0.0/24
RETURN     all  --  192.168.122.0/24      255.255.255.255
MASQUERADE tcp  --  192.168.122.0/24      !192.168.122.0/24      masq ports: 1024-65535
MASQUERADE udp  --  192.168.122.0/24      !192.168.122.0/24      masq ports: 1024-65535
MASQUERADE all  --  192.168.122.0/24      !192.168.122.0/24
```

Si vede da questa configurazione che da questo set di indirizzi (192.168.122.0/24) il masquerading è abilitato, cioè, il nat è abilitato. Possiamo provare a pingare 8.8.8.8 per vedere se il nat funziona, ma non avrebbe senso provare a pingare S1 perché è sulla stessa rete quindi ovviamente lo raggiunge. Altrimenti ha senso vedere se riesco a raggiungere S2, perché devo uscire dalla attuale LAN.

Eseguo il comando *ping 192.168.129.3* che è l'ip dell'interfaccia di S2, appartenente alla rete di S2 e S1.

```
$ ping 192.168.129.3
PING 192.168.129.3 (192.168.129.3): 56 data bytes
64 bytes from 192.168.129.3: seq=0 ttl=63 time=10.381 ms
64 bytes from 192.168.129.3: seq=1 ttl=63 time=3.837 ms
64 bytes from 192.168.129.3: seq=2 ttl=63 time=2.045 ms
64 bytes from 192.168.129.3: seq=3 ttl=63 time=2.986 ms
64 bytes from 192.168.129.3: seq=4 ttl=63 time=3.450 ms
64 bytes from 192.168.129.3: seq=5 ttl=63 time=2.610 ms
64 bytes from 192.168.129.3: seq=6 ttl=63 time=3.262 ms
64 bytes from 192.168.129.3: seq=7 ttl=63 time=2.083 ms
64 bytes from 192.168.129.3: seq=8 ttl=63 time=3.108 ms
64 bytes from 192.168.129.3: seq=9 ttl=63 time=3.289 ms
```

Il ping funziona. Dobbiamo però provare a vedere che il natting non sta funzionando nella LAN tra Cirros A e S1, ma che valga per le reti esterne. Per vederlo possiamo usare il comando da S1: *tcpdump -n -i vnet1 icmp*

```

student@student1:~$ sudo tcpdump -n -i vnet1 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on vnet1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
06:59:22.354958 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 11777, seq 77, length 64
06:59:22.356130 IP 192.168.129.3 > 192.168.122.84: ICMP echo reply, id 11777, seq 77, length 64
06:59:23.356968 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 11777, seq 78, length 64
06:59:23.358547 IP 192.168.129.3 > 192.168.122.84: ICMP echo reply, id 11777, seq 78, length 64
06:59:24.359204 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 11777, seq 79, length 64
06:59:24.360065 IP 192.168.129.3 > 192.168.122.84: ICMP echo reply, id 11777, seq 79, length 64
06:59:25.364018 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 11777, seq 80, length 64
06:59:25.365168 IP 192.168.129.3 > 192.168.122.84: ICMP echo reply, id 11777, seq 80, length 64
06:59:26.367236 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 11777, seq 81, length 64
06:59:26.368739 IP 192.168.129.3 > 192.168.122.84: ICMP echo reply, id 11777, seq 81, length 64
06:59:27.371748 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 11777, seq 82, length 64
06:59:27.373336 IP 192.168.129.3 > 192.168.122.84: ICMP echo reply, id 11777, seq 82, length 64

```

Il ping viene fatto da 192.168.122.84 (CirrosA) e arriva a 192.168.129.3 che è l'indirizzo di S2 e poi vediamo nella riga sotto che torna indietro; quindi, non abbiamo nat perché sono gli indirizzi IP corretti (quindi non c'è stato un cambiamento di IP, sono quelli di destinazione e sorgente del ping). Se controllo *enp0s1* vedo:

```

student@student1:~$ sudo tcpdump -n -i enp0s1 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
07:00:07.510421 IP 192.168.129.2 > 192.168.129.3: ICMP echo request, id 11777, seq 122, length 64
07:00:07.511820 IP 192.168.129.3 > 192.168.129.2: ICMP echo reply, id 11777, seq 122, length 64
07:00:08.515714 IP 192.168.129.2 > 192.168.129.3: ICMP echo request, id 11777, seq 123, length 64
07:00:08.516880 IP 192.168.129.3 > 192.168.129.2: ICMP echo reply, id 11777, seq 123, length 64
07:00:09.516792 IP 192.168.129.2 > 192.168.129.3: ICMP echo request, id 11777, seq 124, length 64
07:00:09.518083 IP 192.168.129.3 > 192.168.129.2: ICMP echo reply, id 11777, seq 124, length 64
07:00:10.519348 IP 192.168.129.2 > 192.168.129.3: ICMP echo request, id 11777, seq 125, length 64
07:00:10.520945 IP 192.168.129.3 > 192.168.129.2: ICMP echo reply, id 11777, seq 125, length 64
07:00:11.520851 IP 192.168.129.2 > 192.168.129.3: ICMP echo request, id 11777, seq 126, length 64
07:00:11.523137 IP 192.168.129.3 > 192.168.129.2: ICMP echo reply, id 11777, seq 126, length 64
07:00:12.525159 IP 192.168.129.2 > 192.168.129.3: ICMP echo request, id 11777, seq 127, length 64
07:00:12.527303 IP 192.168.129.3 > 192.168.129.2: ICMP echo reply, id 11777, seq 127, length 64

```

Si sta eseguendo il natting perché gli indirizzi sono solo di S1 e S2.

Ora però vogliamo che l'IP di CirrosA sia raggiungibile dall'esterno, che sia "routable", per fare ciò ci muoveremo sulla "routed mode", senza natting. Possiamo modificare la configurazione della rete.

Dopo aver modificato dobbiamo riavviare la rete, si può fare senza riavviare la virtual machine.

Ora si prova a pingare S1 da CirrosA tramite *virbr0* (l'interfaccia esposta da S1) e vediamo che abbiamo connessione con S1:

```

$ ping 192.168.122.1
PING 192.168.122.1 (192.168.122.1): 56 data bytes
64 bytes from 192.168.122.1: seq=0 ttl=64 time=6.223 ms
64 bytes from 192.168.122.1: seq=1 ttl=64 time=6.533 ms
64 bytes from 192.168.122.1: seq=2 ttl=64 time=1.182 ms

```

Se ora provo a pingare S2 sempre da CirrosA vediamo che il ping non funziona:

```

$ ping 192.168.129.3
PING 192.168.129.3 (192.168.129.3): 56 data bytes

```

Il ping non va ma non riceviamo un errore, perciò dobbiamo capire dove va il pacchetto.


```
$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.122.1  0.0.0.0         UG    0      0      0 eth0
192.168.122.0    0.0.0.0        255.255.255.0   U     0      0      0 eth0
$
```

Da questa tabella vediamo che il sistema di CirrosA sa dove mandarlo poiché ho 0.0.0.0 che è un default gateway, per questo sta andando fuori; infatti, il pacchetto raggiunge effettivamente S1 perché il gateway indica S1. Lo vediamo facendo il *tcpdump* da S1:

```
student@student1:~$ sudo tcpdump -n -i vnet1 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on vnet1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
07:07:33.750985 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 10, length 64
07:07:34.756876 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 11, length 64
07:07:35.757892 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 12, length 64
07:07:36.759170 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 13, length 64
07:07:37.763894 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 14, length 64
07:07:38.769634 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 15, length 64
07:07:39.774177 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 16, length 64
```

Verifico che S1 e S2 sono sulla stessa rete:

```
student@student1:~$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.64.1   0.0.0.0         UG    0      0      0 enp0s2
131.114.33.84    0.0.0.0        255.255.255.255 UH    100   0      0 enp0s1
131.114.33.85    0.0.0.0        255.255.255.255 UH    100   0      0 enp0s1
192.168.64.0     0.0.0.0        255.255.255.0   U     0      0      0 enp0s2
192.168.122.0    0.0.0.0        255.255.255.0   U     0      0      0 virbr0
192.168.129.0    0.0.0.0        255.255.255.0   U    100    0      0 enp0s1
```

Vedo che può raggiungerla, i pacchetti stanno andando verso S2, come si può vedere:

```
student@student2:~$ sudo tcpdump -n -i enp0s1 icmp
[sudo] password for student:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
07:09:20.076245 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 16, length 64
07:09:21.079300 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 17, length 64
07:09:22.084310 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 18, length 64
07:09:23.087047 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 19, length 64
07:09:24.090080 IP 192.168.122.84 > 192.168.129.3: ICMP echo request, id 13057, seq 20, length 64
```

Ma perché non tornano indietro? Provo a pingare CirrosA da S2:

```
student@student2:~$ ping 192.168.122.84
PING 192.168.122.84 (192.168.122.84) 56(84) bytes of data.
From 192.168.122.1 icmp_seq=1 Destination Host Unreachable
From 192.168.122.1 icmp_seq=2 Destination Host Unreachable
From 192.168.122.1 icmp_seq=3 Destination Host Unreachable
```

Non riceviamo una risposta questa volta.


```
student@student2:~$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.64.1   0.0.0.0         UG    0      0      0 enp0s2
131.114.33.84    0.0.0.0         255.255.255.255 UH    100    0      0 enp0s1
131.114.33.85    0.0.0.0         255.255.255.255 UH    100    0      0 enp0s1
192.168.64.0     0.0.0.0         255.255.255.0   U     0      0      0 enp0s2
192.168.122.0    0.0.0.0         255.255.255.0   U     0      0      0 virbr0
192.168.129.0    0.0.0.0         255.255.255.0   U    100    0      0 enp0s1
```

Come si vede posso raggiungere la rete di Cirros A attraverso *virbr0*. E allora perché non funziona? Questa volta S2 sta cercando nella rete a lui soprastante, ovvero la “sua” rete *.122*, come se esistesse. Tutto è stato fatto con libvirt e KVM come l'altra quindi anch'essa ha una rete di default e potrebbe avere delle macchine virtuali innestate. Per far funzionare il tutto bisogna cambiare la configurazione della rete.

Si può vedere la configurazione di S2 per la rete di default di virsh e cambiarla, cambiamo *.122* a *.123* per esempio, nel range dello spazio DHCP:

```
<network>
  <name>default</name>
  <uuid>0b0363da-66a5-4c74-bcda-6b836062667f</uuid>
  <forward mode='nat' />
  <bridge name='virbr0' stp='on' delay='0' />
  <mac address='52:54:00:ab:97:7f' />
  <ip address='192.168.123.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.123.2' end='192.168.123.254' />
    </dhcp>
  </ip>
</network>
```

Ora però bisogna effettuare il restart.

Se riprovo a fare il ping da CirrosA verso S2 continua a non funzionare. Ora i pacchetti di ritorno da S2 vanno verso internet, perché abbiamo una default route appartenente al DHCP:

```
student@student2:~$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.64.1   0.0.0.0         UG    0      0      0 enp0s2
131.114.33.84    0.0.0.0         255.255.255.255 UH    100    0      0 enp0s1
131.114.33.85    0.0.0.0         255.255.255.255 UH    100    0      0 enp0s1
192.168.64.0     0.0.0.0         255.255.255.0   U     0      0      0 enp0s2
192.168.123.0    0.0.0.0         255.255.255.0   U     0      0      0 virbr0
192.168.129.0    0.0.0.0         255.255.255.0   U    100    0      0 enp0s1
```

Dobbiamo aggiungere una rete più specifica:

```
student@student2:~$ sudo ip -4 route add 192.168.122.0/24 via 192.168.129.2
student@student2:~$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.64.1   0.0.0.0         UG    0      0      0 enp0s2
131.114.33.84    0.0.0.0        255.255.255.255 UH    100    0      0 enp0s1
131.114.33.85    0.0.0.0        255.255.255.255 UH    100    0      0 enp0s1
192.168.64.0     0.0.0.0        255.255.255.0   U     0      0      0 enp0s2
192.168.122.0    192.168.129.2  255.255.255.0   UG    0      0      0 enp0s1
192.168.123.0    0.0.0.0        255.255.255.0   U     0      0      0 virbr0
192.168.129.0    0.0.0.0        255.255.255.0   U    100    0      0 enp0s1
```

Come gateway però abbiamo specificato il prossimo hop che è S1. La configurazione manuale è error prone quindi in futuro vedremo le configurazioni automatiche. Tutto deve essere coerente, le decisioni vengono prese localmente.

Abbandono di Linux Bridge

Why “leaving” linux bridging

Guest (VM) networking in KVM has traditionally been done using linux bridging

Pros:

- good performance
- simple to configure and manage

Cons:

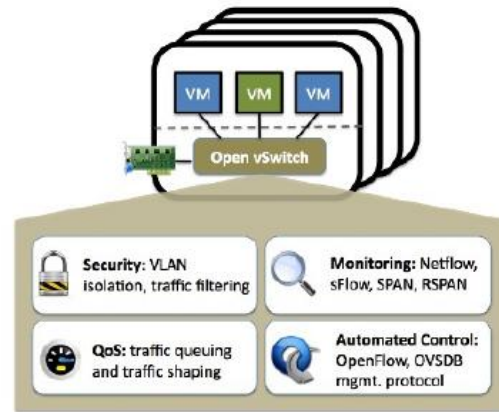
- It was not originally designed for virtual networking
- It does not support tunneling protocols (!updated!)
- not an openflow compatible device (no SDN support to date)

Una rete bridged condivide un dispositivo Ethernet reale con macchine virtuali (VM). Ogni VM può collegarsi direttamente a qualsiasi indirizzo IPv4 o IPv6 disponibile sulla LAN, proprio come un computer fisico. Il bridging offre le migliori prestazioni e il minor “mal di testa” tra i tipi di rete libvirt. La connettività basata su NAT è utile per distribuzioni rapide e facili o su macchine con connettività di rete dinamica/sporadica. Gli utenti più avanzati vorranno utilizzare il bridging completo, in cui l'ospite è connesso direttamente alla LAN.

OpenvSwitch



- Open source (apache 2.0 licensing)
- Written in C
- Thought for multi-server virtualization
- Support state mobility
- support “remote configurability”
- Hardware integration



Open vSwitch (OVS) è uno switch software multistrato concesso in licenza con la licenza open source Apache 2. L'obiettivo dei suoi sviluppatori è implementare una piattaforma switch di qualità di produzione che supporti interfacce di gestione standard e apra le funzioni di inoltro all'estensione e al controllo programmatici. Open vSwitch è adatto a funzionare come switch virtuale in ambienti VM. Oltre a esporre interfacce di controllo e visibilità standard al livello di rete virtuale, è stato progettato per supportare la distribuzione su più server fisici. Open vSwitch supporta più tecnologie di virtualizzazione basate su Linux tra cui Xen/XenServer, KVM e VirtualBox.

OVS può essere usato come **switch general purpose**, fornisce le stesse funzionalità di un linux bridge.

Installing OVS

```
sudo apt-get install openvswitch-switch
```

```
sudo service ovs-vswitchd start
```

```
sudo service ovssdb-server start
```

Si può installare su S1 e S2, si possono lanciare con quei comandi

before continuing...

- in CirrosA/CirrosB run:
`sudo vi /etc/default/udhcpd`
- and set `TIMEOUT="4"`
- save and shutdown both CirrosA/CirrosB

Si può cambiare a *timeout* per evitare che pingi internet per troppo tempo.

Vogliamo rimuovere il linux bridge e aggiungere il nuovo OVS.

Create a new bridge

- `ovs-vsctl add-br customSw`
- `ovs-vsctl show`

```
student@student_server:~$ sudo ovs-vsctl show
780b88f2-b91a-4a9d-971d-4156de4ac421
    Bridge customSw
        Port customSw
            Interface customSw
                type: internal
    ovs_version: "2.9.5"
```

Con quel comando si può vedere la lista di switch, vediamo una interfaccia esposta sulla macchina.

Configure VMs to use ovs (libvirt)

```
virsh edit cirrosA
```

- Change the `<interface type='network'>` configuration section as follows

```
<interface type='bridge'>
  <source bridge='customSw' />
  <virtualport type='openvswitch' />
  <address type='pci' domain='0x0000' bus='0x00'
slot='0x03' function='0x0' />
</interface>
```

Si può modificare la configurazione di *CirrosA*: si usa uno switch, con un certo nome, con un particolare tipo e altre informazioni standard, se hai informazioni come il mac address ecc. vanno rimosse. Il sistema creerà una nuova interfaccia su CirroSA e la conetterà al nuovo switch virtuale.

Check

- Start cirrosA/cirrosB
- Verify bridge ports

```
ovs-vsctl show
```

```
[sudo] password for student:
780b88f2-b91a-4a9d-971d-4156de4ac421
Bridge customSw
  Port customSw
    Interface customSw
      type: internal
  Port "vnet0"
    Interface "vnet0"
ovs_version: "2.9.5"
```

Puoi controllare il tipo di interfacce usando il `link ip -details show`.

Se esegui il comando in figura vedrai una nuova porta connessa a *vnet0*. Non abbiamo l'IP perché nessuno ha configurato un DHCP.

Configure addresses

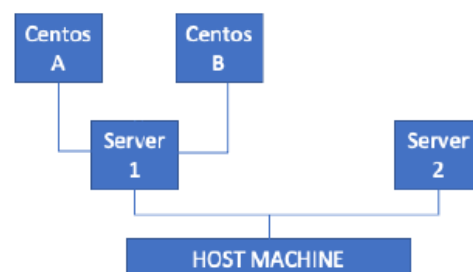
- statically assign IPs to cirrosA/cirrosB

```
sudo ifconfig eth0 192.168.120.10
```

L'IP si può assegnare staticamente.

Test: ovs1

- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B



Puoi pingare A e B, ma non puoi parlare con server 1 perché lo switch ha un'interfaccia, ma non ha un indirizzo IP legato al custom switch, deve essere sulla stessa rete. Se lo assegni, sarai capace di pingare Cirros B e server 1, ma non sarai capace di pingare server 2. Cirros A non sa come andare al Server 2, non c'è un DHCP di default, nella routing table non c'è un default gateway, prima ce

l'avevamo perché c'era un DHCP server, che dava un IP di default e dava anche default gateway, ora devi farlo manualmente.

Se si aggiunge una route per raggiungere S2 i pacchetti ci arrivano.

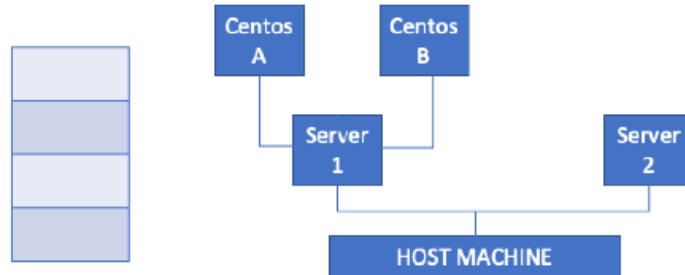
Configure addresses (2)

- To make student1 accessible from cirrosA, we need to give an ip address to one of the internal interfaces of customSw

```
sudo ifconfig customSw 192.168.120.1
```

Test: ovs2

- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B



Configure routing

- On Server 2

```
sudo ip -4 route add 192.168.120.0/24 via 192.168.56.3
```

- On CirrosA/B

```
ip -4 route add 192.168.56.0/24 via 192.168.120.1
```

- Visualize routes

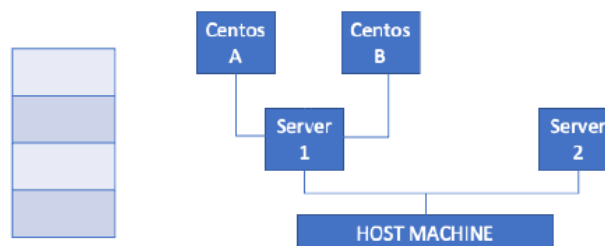
```
route -n
```

- Observe traffic (e.g. on enp0s3 of Server1)

```
tcpdump -i enp0s3 icmp
```

Test: ovs3

- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B



Se invece volessi andare in internet?

Possiamo usare il nat oppure “attacco il cavo direttamente alla macchina”, ma facendo ciò il cavo è stato spostato (dal server 1 a Cirros A), però non funziona ancora perché Cirros A non sa come usarla. Bisogna aggiungere un default gateway, devi aggiungere anche la route per raggiungere il default gateway attraverso *eth0*.

Connecting towards external network

- to allow cirrosA to reach "other" networks, we need to attach the "real" network interfaces of student_server to the bridge.

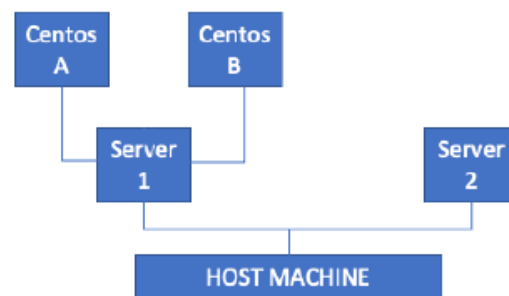
```
ovs-vsctl add-port customSw enp0s8
```

- in CirrosA/CirrosB

```
sudo ip -4 route add 10.0.3.0/24 dev eth0
route add default gw 10.0.3.2
```

Test: ovs4

- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B
- Ping: A,B -> 8.8.8.8
- Ping: server1 -> 8.8.8.8



Reconnect host to external network

- Clear IP address for enp0s8

```
ifconfig enp0s8 0.0.0.0
```

- Give it to customSw (internal)

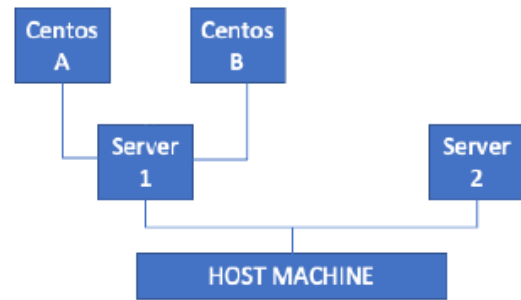
```
ifconfig customSw 192.168.120.1
```

- Set route and default gw

```
sudo ip -4 route add 10.0.3.0 dev customSw
route add default gw 10.0.3.2
```

Test: ovs5

- Ping: A <-> B
- Ping: A,B <-> Server 1
- Ping: A,B -> Server 2
- Ping: Server 2 -> A,B
- Ping: A,B -> 8.8.8.8
- Ping: server1 -> 8.8.8.8



Containers

Containers

- A lightweight virtualization mechanism
- In the end, they are just processes running on an OS instance
- Characteristics:
 - Isolation (through namespaces)
 - Distribution
 - Reuse
 - Speed (if compared to VMs)

È un altro tipo di virtualizzazione. I container permettono la creazione di nodi virtuali guest, qui non hai un completo SO sopra il SO host, sfrutti solo le funzionalità direttamente dal SO host. Ciò a cui ci riferiamo come container è un processo in esecuzione su un'istanza del sistema operativo che può o meno sfruttare determinate funzionalità del sistema operativo.

Isolato si intende rispetto al SO e agli altri containers, attraverso i namespace. Il namespace permette di limitare cosa viene visto dal container, per esempio un container può vedere solo un sottoinsieme delle interfacce di rete. Ricreiamo le caratteristiche della VM ma senza avere un vero e proprio SO innestato. Il container contiene tutto ciò che è necessario per essere eseguito. È più veloce perché non devi eseguire l'intero sistema operativo.

CONTAINERS E CLOUD COMPUTING: dalla loro introduzione, tuttavia, i containers sono diventati una parte fondamentale del cloud computing nativo. Come definito dalla Cloud Native Computing Foundation (CNCF), il cloud computing nativo consente "alle organizzazioni di creare ed eseguire applicazioni scalabili in ambienti moderni e dinamici come cloud pubblici, privati e ibridi, sfruttando al massimo i vantaggi del cloud computing".

Linux Networking and Containers

- The building blocks that allow networking among/with containers are:
 - Network namespaces
 - Virtual ethernet (veths) interfaces
 - Bridges
 - IP routing
 - IP masquerading

I containers (e i sistemi di orchestrazione dei contenitori come Kubernetes) estendono questi elementi di base per ospitare il networking dei contenitori. Per il networking dei containers, vengono sfruttate cinque tecnologie Linux:

- Namespace di rete
- Interfacce Ethernet virtuali (veth) (ogni VM ha una interfaccia eth0 di default, virtuale)
- Bridge
- Routing IP
- Mascheramento IP

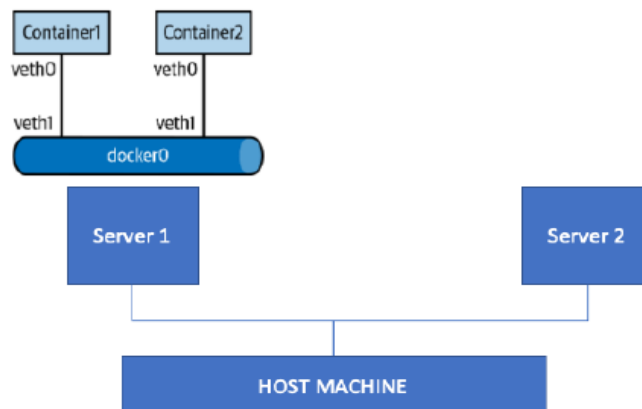
I namespace sono un meccanismo di isolamento; limitano ciò che i processi possono "vedere" nel namespace. Il kernel Linux fornisce più namespace; per il networking, i namespace di rete sono i namespace principali coinvolti. I namespace di rete possono essere utilizzati per supportare più tabelle di routing separate o più configurazioni iptables separate, o per definire l'ambito o limitare la visibilità delle interfacce di rete.

Applicazioni:

- Routing per processo
- Supporto di spazi di indirizzi IP sovrapposti
- Rete dei contenitori

L'IP routing permette al kernel Linux di indirizzare i pacchetti, per capire il next hop.

Container Networking



Ricordiamo che Server 1 e Server 2 sono connessi ad una stessa rete, la quale permette ad entrambi attraverso il Nat di raggiungere l'esterno. Quando crei un container Docker crea un virtual bridge, come faceva libvirt, inoltre per ogni container crea 2 endpoint che sono le "veth", uno nel bridge e uno nel container (qualsiasi cosa entra una interfaccia esce dall'altra).

Network Namespaces

- `ip netns add space1`
- `ip netns add space2`
- `ip netns list`

- `ip link add veth-s1 type veth peer name veth-s2`
- `ip link list | grep veth`

- `ip link set veth-s1 netns space1`
- `ip link set veth-s2 netns space2`
- `ip link list | grep veth`

- `ip netns exec space1 ip link`

Usando il comando `ip` possiamo gestire i namespace di rete, crearli per esempio con i primi 2 comandi, possiamo vedere la lista di quelli che già abbiamo. Possiamo aggiungere anche una nuova coppia di veth con il terzo comando, abbiamo creato una sorta di connessione fra le due macchine.

L'ultimo comando permette di entrare nel namespace `space1` e lanciare il comando `IP`.

Testing namespaces

- `sudo ip netns exec space1 ip address add 10.0.0.11/24 dev veth-s1`
- `sudo ip netns exec space1 ip link set veth-s1 up`
- `sudo ip netns exec space2 ip address add 10.0.0.12/24 dev veth-s2`
- `sudo ip netns exec space2 ip link set veth-s2 up`

- `sudo ip netns exec space1 ping 10.0.0.12`

Si può vedere come cambiano le interfacce attraverso questi comandi.

`Exec + comando + netns` → eseguire quel comando nel namespace specificato

Vedrai che queste interfacce che hai aggiunto non avranno un indirizzo IP.

Con i comandi di `set` si attivano le interfacce.

Quick recap on Dockers

- *Add Docker's official GPG key:*

```
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

- *Add the repository to Apt sources:*

```
echo \
"deb [arch=$(dpkg --print-architecture)] signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \ »
$(. /etc/os-release && echo "$VERSION_CODENAME")" stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

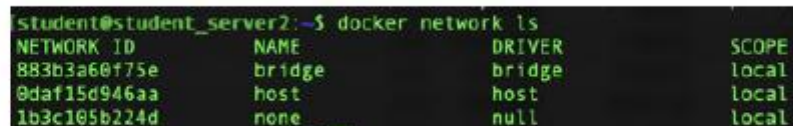
- *Install latest version*

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Con la prima parte di comandi si aggiunge la cartella di Docker nella lista delle cartelle del tuo SO, si scarica attraverso il comando "apt", si aggiungono le chiavi...

Docker network information

- **docker network ls**



NETWORK ID	NAME	DRIVER	SCOPE
883b3a60f75e	bridge	bridge	local
0daf15d946aa	host	host	local
1b3c105b224d	none	null	local

Nella figura possiamo vedere tutte le reti disponibili dal punto di vista di docker: vediamo una bridge network. Se controlliamo le interfacce della macchina host, notiamo che è presente un nuovo bridge, utilizzato da docker di default per collegare i containers.

Viene elencata la rete bridge predefinita, insieme a host e none. Le ultime due non sono reti completamente sviluppate, ma vengono utilizzate per avviare un contenitore connesso direttamente allo stack di rete dell'host del daemon Docker o per avviare un contenitore senza dispositivi di rete.

Base commands

```
docker pull alpine
docker search alpine
docker run -it alpine

docker ps -a          // list running/stopped containers
docker rm ID          // rm an existing container
docker images
docker rmi ID          // rm an existing image

docker run -dti        // detached, with terminal, interactive
docker attach alpine   // attach to container's console
ctrl+p then q          //detaches from a container
```

Alpine è simile a cirros, è una immagine docker usata per testing.

I namespace di rete creati tramite docker non sono visibili tramite il comando *netns*. Dovresti invece usare il comando *lsns -t net -J*.

Running containers

```
docker run -dit --name alpine1 alpine
```

**A network namespace will be created and associated to the container.
The namespace can be retrieved both:**

- **Getting the container PID and use it to browse /proc/PID/ns**

or

- **Retrieving the container ID, inspecting the corresponding container configuration and searching for the SandboxKey property**

Avvia due container *Alpine* che eseguono *ash*, che è la shell predefinita di *Alpine* anziché *bash*. I flag *-dit* indicano di avviare il contenitore “detached” (in background), interattivo (con la possibilità di digitare al suo interno) e con un TTY (in modo da poter vedere l'input e l'output). Poiché lo stai avviando “detached”, non sarai connesso al container immediatamente. Invece, verrà visualizzato l'ID del container. Poiché non hai specificato alcun flag *--network*, i containers si collegano alla rete bridge predefinita.

Se esegui il comando *ip netns list* non vedrai i containers creati. Questo perché *netns* cerca i namespace solo in */var/run/netns/*, mentre docker non include le informazioni in quella cartella. Puoi recuperare informazioni sui namespace creati da docker utilizzando quanto segue:

OPZIONE 1

sudo docker inspect alpine1 | grep '"Pid\":"' per ottenere il PID del contenitore. Quindi usalo in

`sudo ls -la /proc/PID/ns` dove PID è l'ID del processo del contenitore ottenuto nel passaggio precedente.

OPZIONE 2:

`sudo docker container ls` → recupera l'ID del contenitore

`sudo docker container inspect container-ID | jq` → cerca la proprietà `SandboxKey`

Confrontala con l'output di:

`sudo lsns -t net -J`

Veth interfaces and containers

On host server

```
ip -d link
```

```
10: veth9782499@if9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP mode DEFAU
LT group default
    link/ether 26:bd:d5:ab:f8:9d brd ff:ff:ff:ff:ff:ff link-netnsid 2 promiscuity 1
    veth
```

On container

```
ip link
```

```
9: eth0@if10: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
```

Il primo comando ti permette di vedere le interfacce con più dettaglio, se lanciato sulla macchina host si può vedere l'interfaccia `veth97...` che è la veth del bridge che si connette all'interfaccia di Alpine 1. Si può anche vedere nell'ultima riga che il tipo dell'interfaccia è "veth". Si può anche vedere che è una secondary interface che è connesso al master bridge chiamato "docker 0". Alla fine del nome dell'interfaccia c'è il numero 9, si dice che è "accoppiata con quel numero": il numero 9 è il numero dell'interfaccia `eth0` che si vede sotto, viceversa con il nome dell'interfaccia `eth0`, che ha come ultimo numero 10.

I namespace di rete Linux sono un componente fondamentale per il networking dei container, ma senza un meccanismo per simulare il comportamento di un'interfaccia di rete reale, la capacità dei nostri sistemi di connettere i containers alla rete sarebbe notevolmente limitata. In effetti, saremmo in grado di eseguire solo tanti containers quanti sono le interfacce di rete fisiche che possiamo inserire nell'host. Le interfacce Ethernet virtuali sono la soluzione a tale vincolo.

Ifconfig

`brctl show` → vedi il nuovo bridge

Quando un servizio Docker viene istanziato per la prima volta, crea un dispositivo `docker0`, che è un bridge Linux. Ogni volta che vengono creati nuovi container tramite il comando `docker run` senza specificare un'opzione di rete, Docker crea una coppia di interfacce veth, ne assegna una alle `netns`

Venerdì 11/10

del container e collega l'altra al bridge docker0. Proprio come gli spazi dei nomi di rete consentono al kernel Linux di fornire isolamento di rete, le interfacce Ethernet virtuali (chiamate anche veth) consentono di interrompere intenzionalmente tale isolamento. Le interfacce Veth sono interfacce logiche; non corrispondono a una NIC fisica o a un altro componente hardware. Pertanto, puoi avere molte più interfacce veth di quanto hai interfacce fisiche. Utilizzando l'identificatore dello spazio dei nomi di rete identificato tramite `sudo lsns -t net -J` possiamo anche emettere un comando nello spazio dei nomi corrispondente come segue:

`sudo nsenter -net=/run/docker/netns/NAMESPACE_IDENTIFIER ip link list` (un esempio di NS_IDENTIFIER è b87b15b217e8).

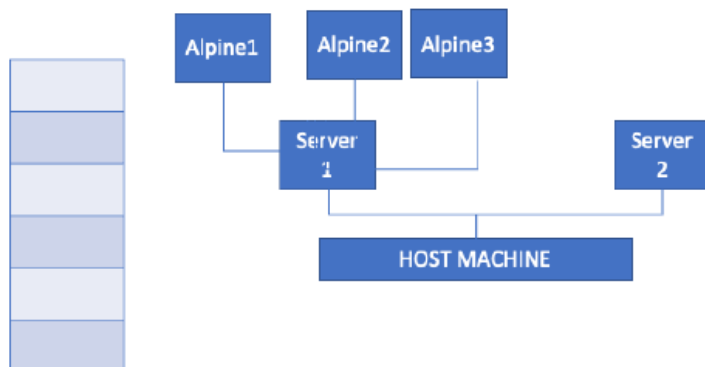
Bridged networking: Creating a custom network

- `docker network create --driver bridge alpine-net`
- `docker run -dit --name alpine2 alpine`
- `docker network connect alpine-net alpine2`
- `docker run -dit --name alpine3 --network alpine-net alpine`
- `docker network inspect alpine-net`

Creiamo 3 containers: in generale le applicazioni si dividono in “pezzi”, ogni pezzo è detto microservizio, che è eseguito in un container. I containers che appartengono allo stesso servizio devono probabilmente essere connessi insieme. Vogliamo testare questo: container che possono parlare con alcuni ma non con altri.

Test1

- Ping: A1 <-> A2
- Ping: A2 <-> A3
- Ping: A1 <-> A3
- Ping: Server 1 <-> Ax
- Ping: Server 2 -> Ax
- Ping: Server 2 <- Ax



Risultato: Alpine 1 parla con Alpine 2, Alpine 2 parla con Alpine 3 ma Alpine 1 non riesce a parlare con Alpine 3. Alpine 1 e 2 hanno la stessa subnet quindi appaiono connessi, 2 e 3 idem. Si può aggiungere la regola che indica che la Alpine 3 è direttamente connessa con il suo IP alla 1 ma ciò non funzionerà: questo significa che non sono connesse direttamente. Vediamo perché.

Con i comandi di prima abbiamo:

- Creato Alpine 1
- Assegnatogli una veth (collegato al peer docker0)
- Poi creo un nuovo bridge per collegare Alpine 2 e 3
- Quando eseguo Alpine 2 senza specificare nulla sto creando una interfaccia veth in Alpine 2 e la collego al bridge (che è la rete di default)
- Sia Alpine 1 che 2 ricevono un indirizzo IP perché connessi allo stesso bridge che ha anche un server DHCP
- Con un altro comando collego una nuova interfaccia di Alpine 2 al bridge di destra, il secondo bridge creato, anche questa interfaccia riceve un indirizzo IP dall'altra sottorete, quella del secondo bridge
- Quando creo invece Alpine 3, specifico a che bridge deve essere connessa, quindi la collego a quello di destra, il secondo e non si collegherà a quello di default. Anche ora viene assegnata una nuova interfaccia e un nuovo IP

In conclusione, non possiamo forzare loro dicendo che sono sulla stessa rete perché non lo sono!

Il namespace permette ad ogni interfaccia delle Alpine di essere viste solo da loro stesse.

Puoi pingare S1 perché è connesso al bridge e anche S2, perché è attivo il masquerading perché la default network ha il nat. S2 non potrà comunicare con le Alpine a causa del nat. Il nat viene effettuato in entrambe le direzioni se la comunicazione viene da Alpine verso S2, ma se invece la comunicazione viene da S2 il nat non lo sa perché non c'è una comunicazione iniziale. Puoi settare una regola per inoltrare i pacchetti a chi di dovere se riceve un pacchetto da S2.

Docker NAT

```
sudo iptables -n -L -t nat
```

```
Chain POSTROUTING (policy ACCEPT)
target     prot opt source                destination
MASQUERADE all  --  172.17.0.0/16          anywhere
```

docker network ls

docker network inspect bridge

Host network

- On server1

```
docker run -dit --network host --name alpine1 alpine
docker attach alpine1
```

Possiamo dire al sistema di non usare i namespace di rete per un host specifico, possiamo creare un container che usi la host network, quindi avrà lo stesso namespace del sistema operativo host, è come fosse interno all'host

Test2:

- On alpine1

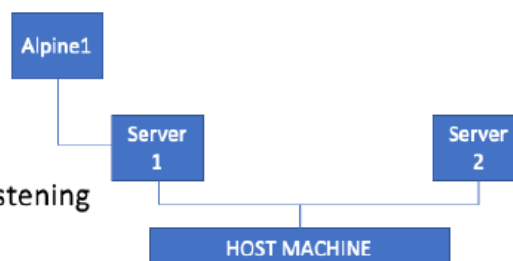
```
nc -u -p 2020 -l // netcat listening
```

- On server2

```
nc 192.168.56.3 2020 -u
```

- On server1

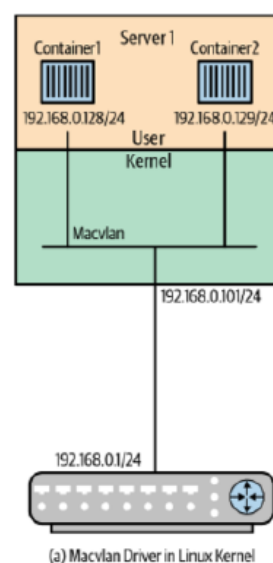
```
tcpdump -i enp0s3 port 2020
```



Netcat aprirà una connessione udp o tcp verso il server, il primo comando che si vede crea un server mentre il secondo crea un client

macvlan

- Makes container appear as they are physically connected to the physical network
- For applications that expects direct connection to the network
- E.g., legacy applications, traffic monitoring



Alcune applicazioni, in particolare le applicazioni legacy o quelle che monitorano il traffico di rete, si aspettano di essere direttamente connesse alla rete fisica. In questo tipo di situazione, puoi usare il

driver di rete *macvlan* per assegnare un indirizzo MAC a ogni interfaccia di rete virtuale del contenitore, facendola apparire come un'interfaccia di rete fisica direttamente connessa alla rete fisica. In questo caso, devi designare un'interfaccia fisica sul tuo host Docker da usare per macvlan, così come la subnet e il gateway di macvlan. Puoi anche isolare le tue reti macvlan utilizzando diverse interfacce di rete fisiche.

Il kernel invia un pacchetto in arrivo all'interfaccia Macvlan il cui indirizzo MAC corrisponde all'indirizzo MAC di destinazione del pacchetto e quindi al contenitore corretto.

È un altro tipo di interfaccia virtuale fornita dal kernel linux per connettere i containers, in questo caso con la rete fisica, facendoli apparire fisicamente connessi alla rete fisica. *Macvlan* permette alla rete fisica di avere molteplici indirizzi MAC, il container vede una interfaccia virtuale che ha un MAC (indirizzo fisico) “esposto” dalla interfaccia fisica della macchina host. Questo può essere richiesto da una applicazione che richiede una connessione fisica alla rete (esempio: monitoraggio del traffico nella rete reale). Noi creiamo un layer virtualizzato per la connettività nella macchina host, esso deve essere fuso con quello fisico.

Macvlan: create

- Create network

```
docker network create -d macvlan \
  --subnet=192.168.56.0/24 \
  --gateway=192.168.56.1 \
  --ip-range=192.168.56.128/25 \
  -o parent=enp0s3 \
  macv
```

--ip-range=192.168.56.128/25

Possiamo creare una nuova rete di tipo macvlan, specifichiamo una serie di parametri, dando tutti i dettagli sulla subnet mask, gateway ecc. Bisogna anche specificare una interfaccia genitore, poiché la rete macvlan verrà accoppiata ad una interfaccia fisica, quella che avrà un nuovo indirizzo MAC. In altre parole, ogni interfaccia che è connessa alla rete macvlan sarà una interfaccia macvlan, accoppiata con una interfaccia fisica. La rete fisica a cui è direttamente connessa, sarà quella a cui enp0s3 è connessa. L'ultimo parametro è il nome della rete.

Macvlan

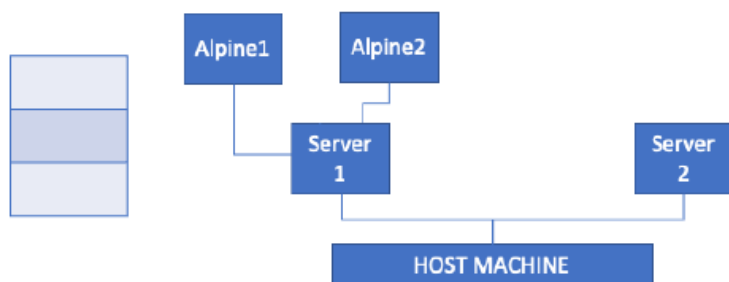
- **Attach hosts**

```
docker run -dit --network macv --name alpine1 alpine
```

Dobbiamo attaccare gli host a questa rete. Creiamo un nuovo alpine container. Specifichiamo la rete a cui il container è connesso.

Test3

- ping: A1<->A2
- Ping A1,A2 <-> server1
- Ping A1,A2 <-> server2



Questa sarà la situazione. Per fare questi test bisogna eliminare le reti create in precedenza chiamate Alpine 1, 2, 3.

Tra Alpine 1 e Alpine 2 la comunicazione è presente, ma non con esterni (server 2). L'interfaccia di rete è configurata solo per gestire pacchetti che sono diretti ad un indirizzo fisico, quindi se riceve un MAC diverso da quello dell'interfaccia, il pacchetto verrà scartato (comportamento di default). Quell'interfaccia dal punto di vista del kernel ha più interfacce MAC, quindi l'interfaccia fisica passerà al kernel anche i pacchetti che non hanno il MAC di `enp0s3`. Questa è chiamata *promiscuous mode*.

Macvlan: promiscuous

- On Vbox: enable promiscuous mode for the interface connected to vboxnet
- On host machine: enable promiscuous mode for `enp0s3`

```
sudo ip link set enp0s3 promisc on
```

L'obiettivo è forzare la ricezione di pacchetti con indirizzo diverso dal MAC della vera interfaccia fisica. Questo può essere utile quando monitori il traffico in una rete condivisa o in una LAN, devi "sniffare" i pacchetti broadcast. Il comando per abilitarla è presente nella figura sopra. Questo permetterà anche all'hypervisor di comportarsi di conseguenza.

Test4:

- ping: C1<->C2
- Ping C1,C2 <-> server1
- Ping C1, C2 <-> server2



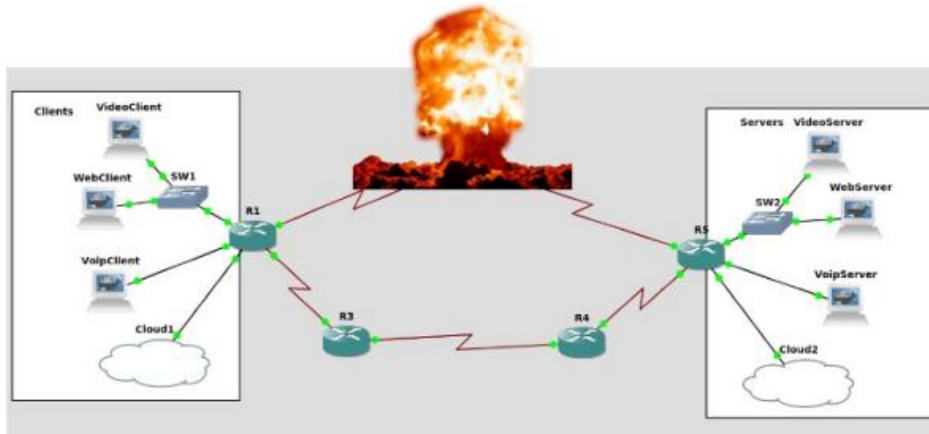
On your host machine you can check the MAC addresses in the packets using the `-e` option of `tcpdump`

Ora la comunicazione tra C1 e C2 e tra loro e Server 1 non funziona, le interfacce dei containers non sono raggiungibili dal networking layer dell'host machine, quindi non possiamo avere questa comunicazione diretta.

Software Define Networking (SDN)

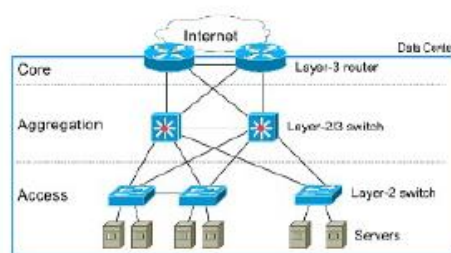
Fino ad ora abbiamo sempre configurato tutti i dispositivi manualmente. Ora abbiamo un cambiamento di paradigma.

Traditional Networks

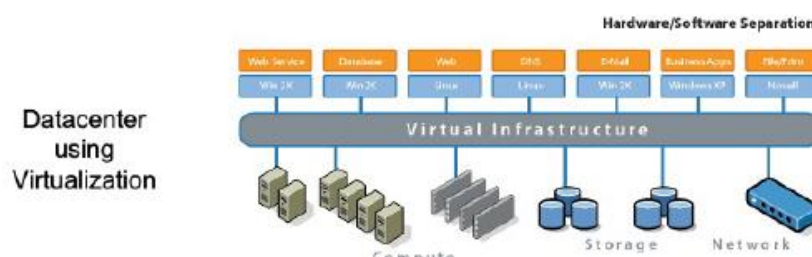


La maggior parte dei protocolli Internet sono stati originariamente progettati per sopravvivere al "bombardamento" di singoli collegamenti. Per rispondere a collegamenti di comunicazione instabili e alla possibile scomparsa di un intero centro di commutazione (switching center), sono stati sviluppati protocolli in modo che bridge e router potessero costruire e aggiornare dinamicamente e autonomamente le loro tabelle di inoltri.

Use Case: Datacenters

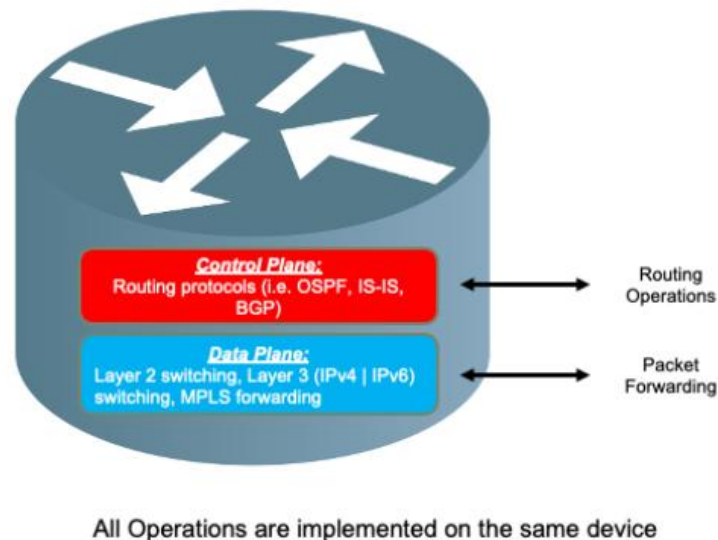


Traditional Datacenter architecture



Un Data center ha 120.000 server fisici, ognuno può ospitare 20 VM, per un totale di 2.400.000 host in un data center, a partire dal 2011. I data center moderni possono raggiungere 1 milione di server fisici... Questo non è più un rischio di bombardamento a collegamento singolo nei data center in cui

tutti gli host sono posizionati abbastanza vicini. La complessità è alta, perché in tutti i dispositivi fisici hai la virtualizzazione, hai tanti dispositivi di rete. Il primo problema è la stabilità della topologia, poiché il 30% della capacità della CPU dei router viene spesa per tracciare i cambiamenti nella topologia di rete. Il secondo è il fatto che si ha anche una grande scala di dispositivi che sono creati e distrutti, attraverso operazioni molto semplici.



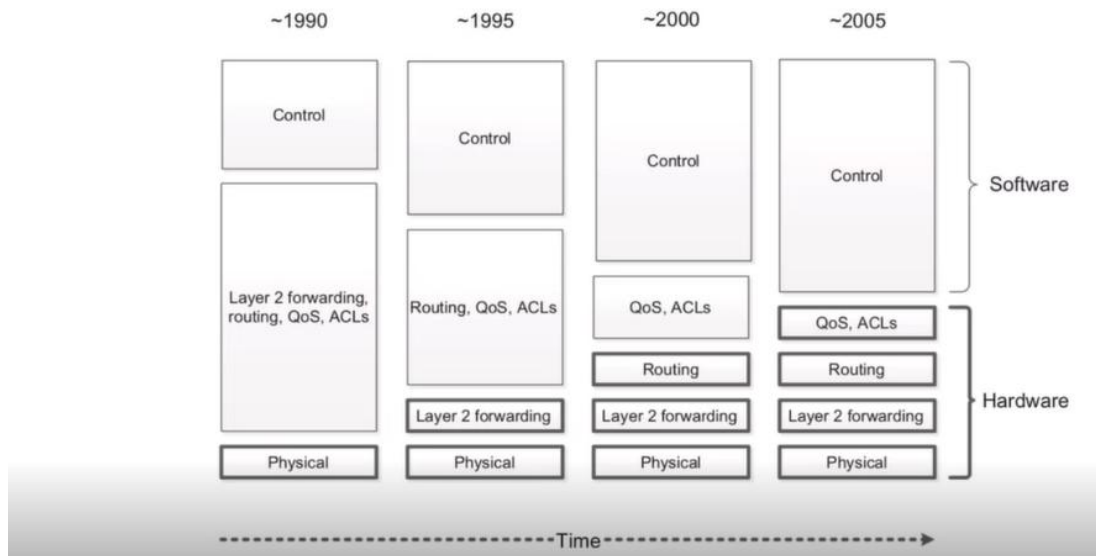
Nei primi giorni delle reti informatiche, tutto tranne il livello fisico era implementato nel software. L'obiettivo è mettere quanta più intelligenza possibile nel dispositivo, per limitare la configurazione manuale. Ogni volta che era necessario un coordinamento, si poteva prendere una decisione collettiva attraverso lo scambio di informazioni e protocolli distribuiti. I requisiti e gli obiettivi erano in realtà simili a quelli di SDN. Tutto ciò potrebbe portare a latenze di convergenza inaccettabili (nell'ordine di 30-50 secondi).

Il Control plane è dove avviene l'amministrazione della rete: corrisponde all'impostazione delle regole di elaborazione dei pacchetti e da lì alla definizione dell'intera politica di switching della rete. Il software di controllo è l'intelligenza che determina i percorsi ottimali e risponde alle interruzioni e alle nuove richieste di rete.

Il Data plane comprende l'applicazione di quelle regole definite sui Control plane: questa è l'elaborazione effettiva dei pacchetti. Quando alcuni pacchetti richiedono un'elaborazione particolare, più complessa, possono essere consegnati al piano di controllo, dove verrà presa la decisione in merito a questo pacchetto.

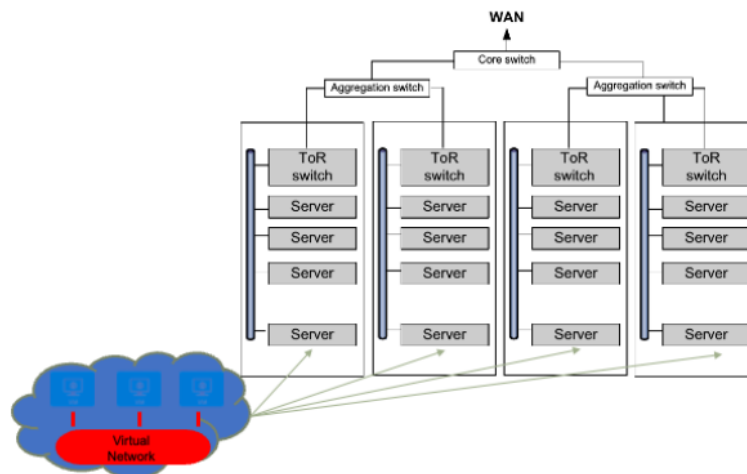
Entrambi devono coesistere nei vari dispositivi.

From Software to Hardware



Molte delle operazioni dei dispositivi, di entrambi i piani, sono state implementate in software. In Hardware ovviamente è più veloce. Nel tempo si è cercato di implementare il data plane in hardware e il control plane è stato lasciato in software.

Datacenter Networking



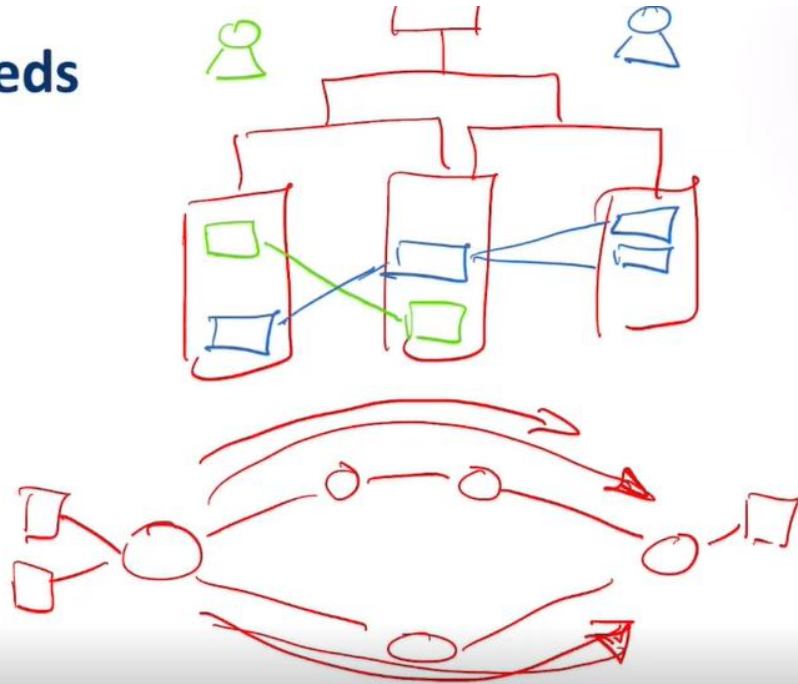
I data center sono caratterizzati da:

- host ravvicinati
- comunicazioni principalmente est-ovest

Questa è una tipica architettura di un datacenter, con tanti racks che hanno dentro i server. Dentro i server potresti avere reti virtuali. I server potrebbero anche voler comunicare con internet. Avrai molte VM e containers nei vari server, che portano ad avere più comunicazioni interne rispetto alle esterne. La maggior parte del traffico è “est verso ovest” rispetto al “nord verso sud”.

Data-center needs

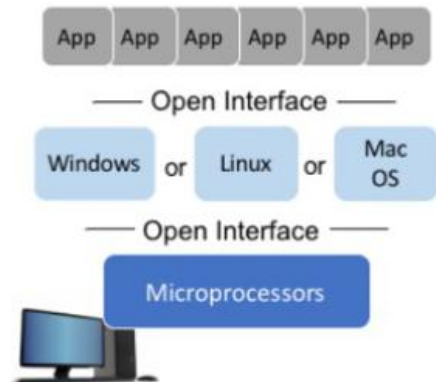
- Automation
- Scalability
- Multipathing
- Multitenancy



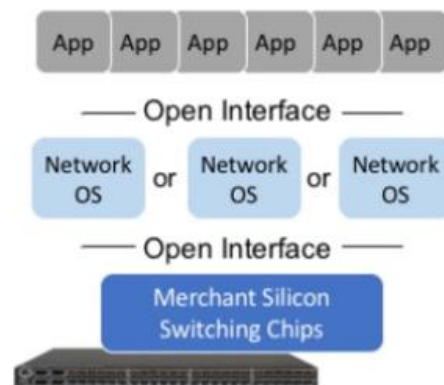
L'automazione consente alle reti di andare e venire a piacimento, seguendo i movimenti dei server e dello storage man mano che cambiano le esigenze. Nei vecchi data center, creare una nuova rete richiederebbe GIORNI.

- Scalabilità: con i data center e gli ambienti cloud, il numero di stazioni terminali che si collegano a una singola rete è cresciuto in modo esponenziale. Le limitazioni delle dimensioni della tabella degli indirizzi MAC e del numero di VLAN sono diventate ostacoli alle installazioni e alle distribuzioni di rete. L'elevato numero di dispositivi fisici presenti nei data center pone anche un problema di controllo della trasmissione. L'uso di tunnel e reti virtuali può contenere il numero di dispositivi in un dominio di trasmissione ad un numero ragionevole. Ci saranno macchine virtuali dentro i server, che andranno interconnesse e qui servirà l'automazione, essendo una quantità veramente elevata. Abbiamo meccanismo di configurazione automatici molto veloci.
- I requisiti di velocità e alta disponibilità del moderno data center impongono che più percorsi non vengano sprecati bloccandoli e, al contrario, vengano utilizzati per migliorare l'efficienza e per ottenere resilienza e bilanciamento del carico.
- Il multi-tenancy implica che il data center debba fornire a ciascuno dei suoi molteplici *tenant* la propria rete (virtuale) che può gestire in modo simile a come gestirebbe una rete fisica.
- Ci deve essere anche ridondanza per avere molteplici percorsi per connettere i vari nodi, in modo tale da resistere ai fallimenti: questo è chiamato *multipathing* (esempio in rosso in basso in figura).

Vertical vs Horizontal Markets



Noi siamo abituati a vedere un computer general purpose, x86 CPU... Il sistema operativo crea una astrazione dell'hardware per permettere lo sviluppo di applicazioni grazie alle interfacce.



Nel networking si ha una struttura simile a quella in figura: il tutto risulta un monolite. Con le evoluzioni si è cercato di portare questo monolite in una struttura simile a quella dei server. È difficile, però, per una sola azienda creare ogni layer di questo monolite.

Why SDN (in short)

- Too **many** control plane targeting **many** networking problems in distributed fashion
 - Closed system, having few to none open projects
 - Manufacturers do not (need to) innovate quickly
- Slower evolution of networking technologies

SDN è un approccio piuttosto che una soluzione puntuale.

Software-Defined Networking (SDN) è un approccio al modo in cui implementiamo le reti, il che è importante perché ha un impatto sul ritmo dell'innovazione. SDN non affronta direttamente nessuna delle sfide tecniche di routing, controllo della congestione, ingegneria del traffico, sicurezza, mobilità, affidabilità o comunicazione in tempo reale, ma apre nuove opportunità per creare e distribuire soluzioni innovative a questi e problemi simili.

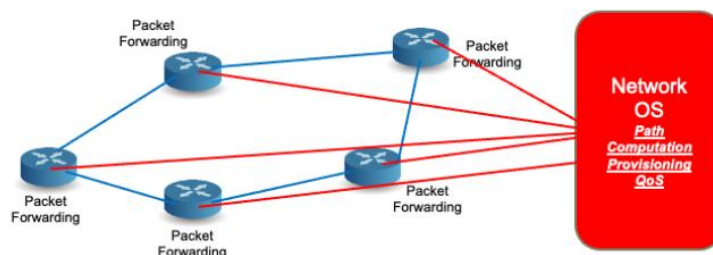
Il control plane è composto da molti protocolli per problemi differenti in modo distribuito (per questo bisogna raccogliere informazioni prima, avere coerenza, insomma molto più complicato).

Questi sistemi erano chiusi. Difficile anche cambiare, innovare, perché sono stati fatti investimenti pesanti nel passato.

La più grande innovazione degli ultimi 20 anni è stato cambiare da *telnet* a SSH, tutto abbastanza stabile rispetto ad altre evoluzioni.

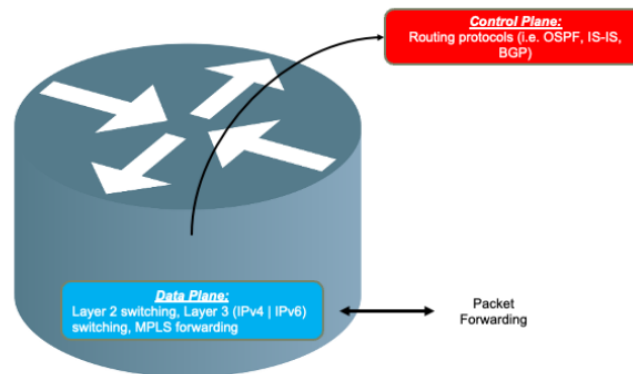
Concept

- **Software defined networking**: Physical separation of the network **control** plane from the **forwarding** plane, where control plane controls several devices
- **Centralization of control**



Siamo nel 2008: vogliamo avere una divisione fisica dei piani, portare il control plane a lavorare in maniera distribuita per gestire molteplici dispositivi (prende decisioni, raccoglie informazioni e riempie le forwarding table).

SDN Networks



Not all the operations are implemented on the same device

Ciò non significa che una copia esatta dei protocolli del Control plane venga eseguita altrove, ma piuttosto che le decisioni di routing vengono prese in modo centralizzato.

Moving Control off the device

- Applications
- Control
- forwarding
filtering
prioritization

Basso livello: inoltrare, eliminare, consumare, replicare, modificare un pacchetto in arrivo.

Abbiamo una nuova architettura, similmente al server. Abbiamo 3 “livelli”: i dispositivi effettuano l’invio, il filtering, la prioritization (decidere chi passa prima tra due flussi)..., vengono fatte solo operazioni, non decisioni; vengono prese informazioni dai dispositivi per dare loro istruzioni su cosa fare; infine abbiamo le applicazioni, non quelle *general purpose*, ma quelle di gestione delle risorse, per comprare le risorse virtualizzate, per fare ciò viene chiesto aiuto al control plane.

SDN traits

- Plane separation
- Simple device
- Centralized control
- Net automation and virtualization
- Openness

Dispositivo semplice e controllo centralizzato: invece di centinaia di migliaia di linee di software di un Control plane complicato in esecuzione sul dispositivo e che consentono al dispositivo di comportarsi in modo autonomo, tale software viene rimosso dal dispositivo e posizionato in un controller centralizzato. Sto centralizzando il controllo e le decisioni (centralizzare in maniera logica, non un punto singolo, ci può essere ridondanza). Il tutto supporta l'automazione e la virtualizzazione. All'inizio l'obiettivo era di aver un sistema open source, per permettere ad altri di sviluppare le loro versioni degli elementi appartenenti a SDN.

Automazione e virtualizzazione di rete, vantaggi dell'interfaccia con le applicazioni:

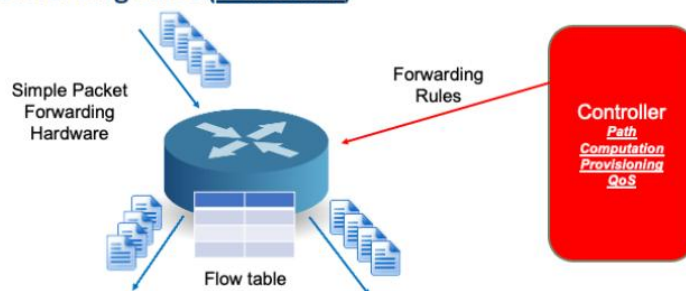
- Converte la sintassi di rete in un linguaggio più intuitivo per gli sviluppatori
- Astrazione di rete
- Astrazione dei protocolli di rete

Si separano data plane e control plane.

Simple device: implementare il tutto in maniera molto semplice ed economica.

SDN components

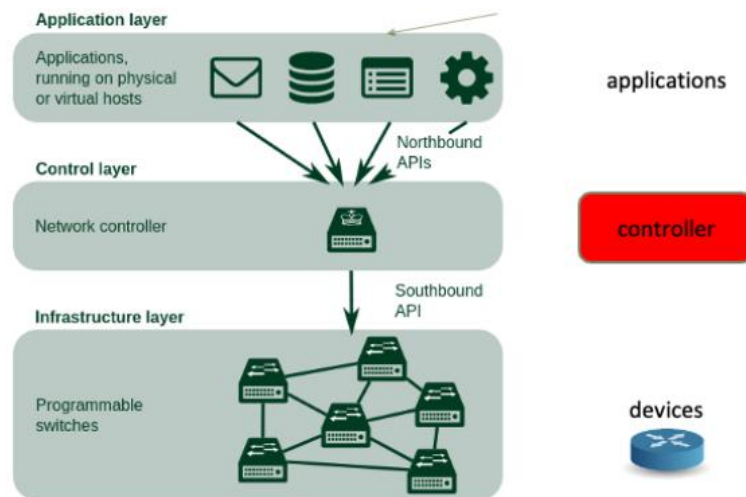
- Routers become simple hardware for packet forwarding (switch)
- A centralized controller is responsible for defining forwarding rules (controller)



Nel 2008 SDN fece la sua prima apparizione attraverso il protocollo *OpenFlow*: descriveva la comunicazione tra il controller e gli switch e il funzionamento di questi ultimi.

Il tutto è implementato con componenti, in particolare router e switch. Dal punto di vista del data plane stanno solo facendo forwarding. Il controller decide le regole e le installa le flow rules nella flow table. La prima implementazione era OpenFlow.

SDN Architecture



Questa è l'architettura della SDN: l'infrastructure layer comprende tutti i dispositivi, un controller centralizzato e l'application layer usato per configurare la rete.

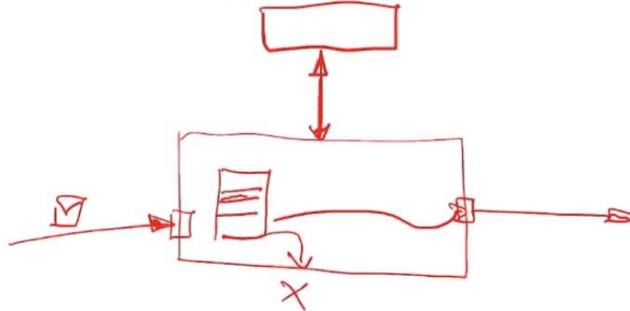
Gli switches ora sono programmabili: possiamo programmarli con del software per ottenere configurazioni automatiche, ricevono un programma formato da una lista di regole. Nell'application layer sono presenti quei programmi che permettono di configurare la rete ma senza sapere nulla del layer sottostante. Creiamo una astrazione della rete: non mi interessa degli elementi di reti, voglio solo che il mio servizio funzioni.

Il control plane non è più distribuito ma centralizzato, logicamente centralizzato, ci sarà quindi un control protocol (routing protocol) che viene eseguito in un punto centralizzato, nel quale vengono prese decisioni per tutti i dispositivi e installate le regole nelle routing table. Control plane e data plane ora sono divisi.

L'abilità del control plane (più in generale, del controller) è nel creare una astrazione della rete: prima se dovevi cambiare qualcosa dovevi sapere come la rete era formata, qual era l'architettura e l'effetto della configurazione sulla rete. Ora non è necessaria per chi gestisce la rete, per esempio, quando deve gestire un nuovo flusso di pacchetti. Il controller prende la topologia, le informazioni dai dispositivi e li espone verso l'application layer. Quindi chiunque scrive una delle *management application*, non deve gestire i dati interni della rete, ma fa riferimento all'astrazione. L'obiettivo è programmare la rete astrando ciò che è presente sotto.

SDN Devices

- Packet processing function
- Abstraction layer (flow table)
- ✂ API for communication with the controller

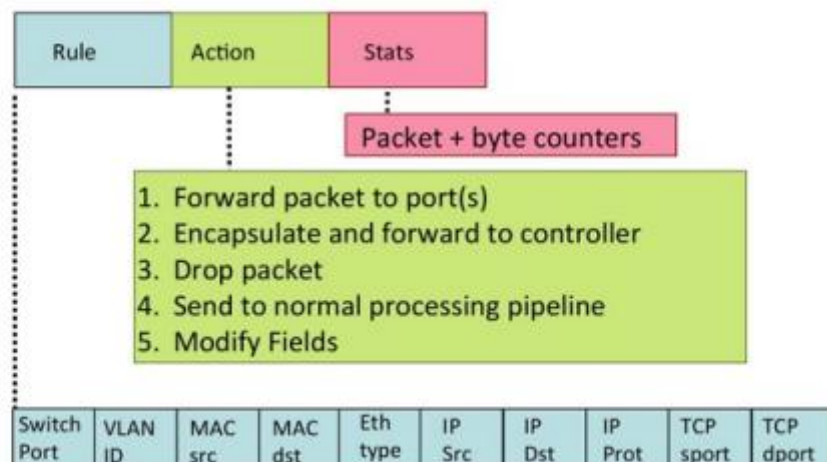


Packet processing: è la funzione che riceve un pacchetto in una interfaccia e guarda una lista di regole già fornite. Si cerca una matching rule e se trovata viene applicata la regola mandando il pacchetto da qualche parte. Sfrutta una astrazione del traffico che scorre, infatti la flow table è una astrazione dato che permette di descrivere il flusso con un set di regole. Un flusso è un set di pacchetti coerente.

Poi abbiamo anche una interfaccia nel controller, la quale permette di prendere informazioni dalla rete e permette di configurare i dispositivi scrivendo regole nella flow table. Questa interfaccia è chiamata *southbound interface*.

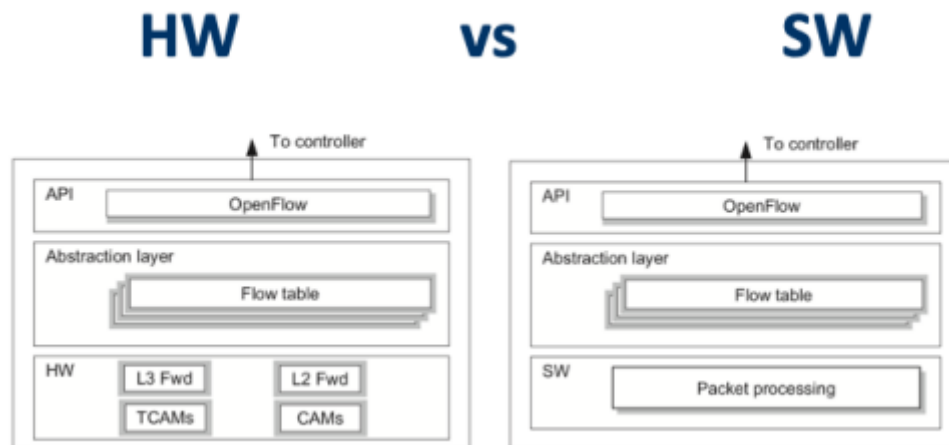
Flow Table

- Set of rules that determines the action to be performed for each packet



Le tabelle di flusso sono composte da un certo numero di voci di flusso prioritarie, ciascuna delle quali è in genere composta da due componenti, campi di corrispondenza e azioni. I campi di

corrispondenza vengono utilizzati per il confronto con i pacchetti in arrivo. Un pacchetto in arrivo viene confrontato con i campi di corrispondenza in ordine di priorità e viene selezionata la prima corrispondenza completa. Le azioni sono le istruzioni che il dispositivo di rete deve eseguire se un pacchetto in arrivo corrisponde ai campi di corrispondenza specificati per questa voce di flusso.



Un dispositivo SDN può essere un hardware SDN switch, che comprende quello che abbiamo detto: API per parlare con il controller (southbound), la quale può essere implementata con *OpenFlow*. Inoltre, crea una astrazione della rete con una o più flow table, ma implementa anche la memorizzazione delle flow table *in memory* in modo tale che si possa accedere facilmente alla tabella.

N.B: A livello 2 (L2) non ho prefissi, ho un match esatto dell'IP.

Abbiamo anche l'implementazione di una SDN in software come OVS: ha esattamente la stessa architettura, con interfaccia per parlare con il controller (stessa del caso HW), ha un livello di astrazione per rappresentare la flow table e un *software-based packet processing* (può essere ad esempio implementato nel kernel). OVS ha anche implementazioni totalmente in hardware.

- Pro SW: interoperabilità, meno vincoli (ad esempio, voci di flusso illimitate)
- Contro SW: più lento, complessità nel supportare le wildcard (non è possibile utilizzare l'hashing standard)

Gli SDN implementati in SW si trovano spesso in ambienti virtualizzati.

- Pro HW: veloce, supporto efficiente delle wildcard (TCAM: sono memorie specializzate usate per velocizzare le operazioni di confronto con wildcard)
- Contro HW: voci limitate, scelte di progettazione complesse (quando CAM o TCAM?), come gestire le statistiche per flusso?

Una *wildcard* è un elemento utilizzato per esprimere corrispondenze più generiche nelle regole di instradamento e filtraggio del traffico di rete. Invece di corrispondere esattamente a un valore specifico (come un indirizzo IP o una porta), una wildcard consente di definire un intervallo o una classe di indirizzi che corrisponde a più flussi di traffico.

Examples of SDN devices

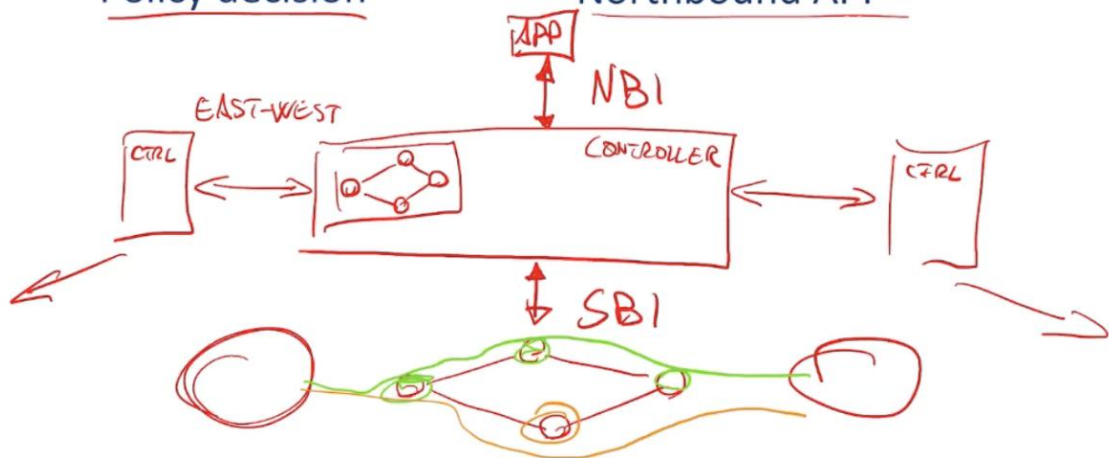
- Commercial examples: OVS (nicira), Indigo (Big Switch), Tofino-based switches (Barefoot-Intel)
- OpenFlow support added to legacy switches
- Whiteboxes

In figura ci sono esempi usati nel mercato.

Whiteboxes: sono le attuali implementazioni dell'*open networking*. L'idea è di avere un *general purpose network device*, che viene eseguito su un HW generico, sopra il quale gira un SO di rete, fatto da un'altra azienda. Può essere usato per implementare SDN ma non è un dispositivo SDN in sé.

SDN Controller

- View of the network
- Policy decision
- Control of SDN devices
- Northbound API



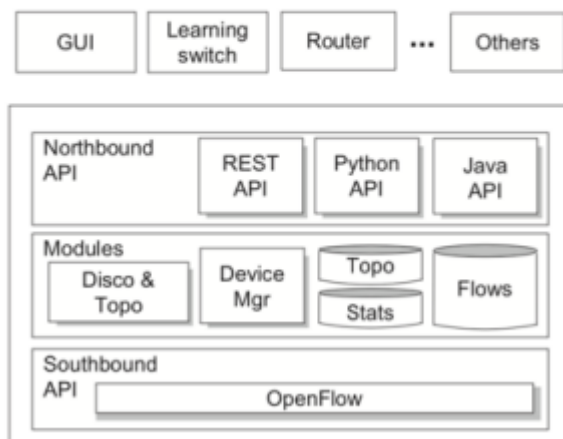
Il controller è una entità che controlla i dispositivi sottostanti tramite la SBI ed è responsabile di:

- implementare il *Control plane*
- scrivere le regole nella sua flow table
- creare una visione della rete internamente (una topologia)
- implementare una *NorthBound API* permettendo al controller di interagire con la applicazione, per dare ad essa una visione della rete, magari una astrazione della stessa o la topologia vera e propria.
- implementare la *policy decision*, per connettere magari una rete ad un'altra. Il controller riceve una richiesta da una rete e prende una decisione per connetterle, in base magari a politiche di spesa.

Non si avrà solo un controller per la rete, ma ce ne sono molteplici: alcuni controllano la porzione della rete e comunicheranno attraverso le *East-West interfaces* con le altre, implementando una sorta di coordinazione.

Per una SBI ci sono protocolli standard, come BGP mentre per le NBI non c'è uno standard, dipende dalla tecnologia e dall'implementazione, ma in generale sfruttano le interfacce *Restful* perché sta diventando una architettura software.

Main Functions of the controller



- **End-user Device Discovery**
- **Network Device Discovery**
- **Network Device Topology Management**
- **Flow Management**

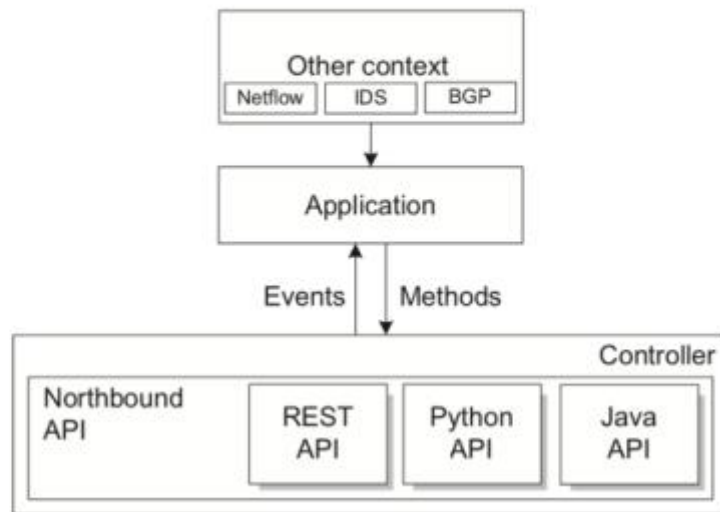
1. Rilevamento dei dispositivi degli end-user: come laptop, desktop, stampanti, dispositivi mobili, ecc.
2. Rilevamento dei dispositivi di rete: rilevamento dei dispositivi di rete che compongono l'infrastruttura della rete, come switch, router e punti di accesso wireless.
3. Gestione della topologia dei dispositivi di rete: mantenimento delle informazioni sui dettagli di interconnessione dei dispositivi di rete tra loro e con i dispositivi degli utenti finali a cui sono direttamente collegati.
4. Gestione dei flussi: mantenimento di un database dei flussi gestiti dal controller ed esecuzione di tutte le necessarie coordinazioni con i dispositivi per garantire la sincronizzazione delle voci del flusso dei dispositivi con tale database.

I primi due permettono di creare una astrazione della rete.

Puoi aggiungere nuovi moduli al controller, puoi cambiare alcune funzioni oppure aggiungere ed usare differenti API per la NBI, sempre Restbased oppure SDK. Sopra al controller puoi generare le SDN application.

SBI sono standardizzate perché sono *software to hardware interface*.

Northbound



Avrai un controller che espone delle API verso le applicazioni, puoi avere una applicazione che è remota o può essere eseguita nel controller stesso.

L'interazione può accadere con chiamate dei metodi, con notifiche sotto forma di eventi oppure in modalità reattiva (ad esempio: noto che un end point è uscito, allora connettilo alla rete).

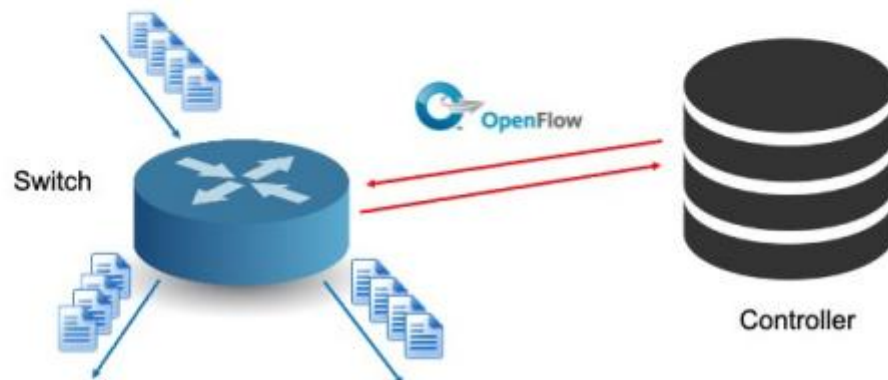
- BASSO LIVELLO: In alcuni casi, questa API in direzione nord è un'interfaccia di basso livello, che fornisce l'accesso ai dispositivi di rete in modo comune e coerente. In questo caso, quell'applicazione è a conoscenza dei singoli dispositivi, ma è protetta dalle loro differenze.
- ALTO LIVELLO: In altri casi, il controller può fornire API di alto livello che forniscono un'astrazione della rete stessa, in modo che lo sviluppatore dell'applicazione non debba preoccuparsi dei singoli dispositivi, ma della rete nel suo complesso. Il modo più elevato di interazione si ottiene tramite i cosiddetti «intenti».

Mettere immagine

Controller può usare openflow, ovsdb oppure BGP come un protocollo per configurare in modo programmatico i dispositivi sottostanti.

Open Flow

- Logical representation of the network (@controller)
- Communication between controller & devices
- Flow-table abstraction & usage

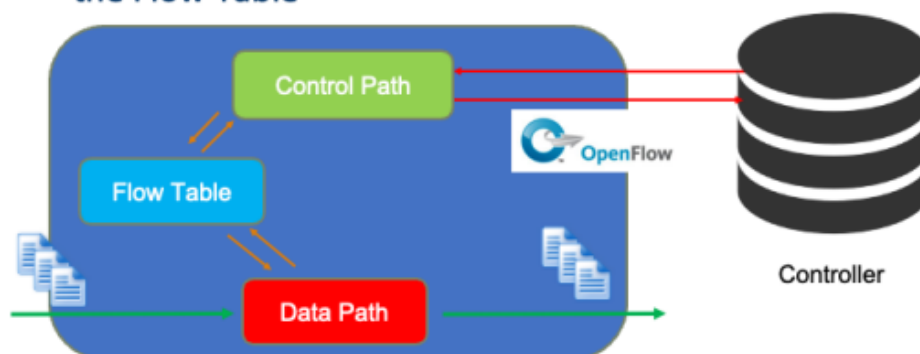


OpenFlow è un protocollo per controllare e interagire con la capacità di forwarding degli switch.

- Si assicura di creare una rappresentazione logica degli switch nella nostra rete in modo che il controller possa elaborare queste informazioni.
- Consente al controller di comunicare con gli switch in modo sicuro.
- Definisce le tabelle di flusso e le voci di flusso per modificare il comportamento di uno switch e definire come vengono gestiti i pacchetti (eliminati, inoltrati, rimossi, ecc.)

Open Flow Switch

- Packets are forwarded according to a simple flow table
- Controller uses the Open Flow protocol to populate the Flow Table

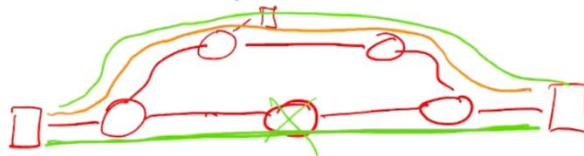


Internamente un dispositivo OpenFlow segue l'architettura di un SDN.

Data path è l'implementazione di un data plane. Esso implementa il Control path.

Applications

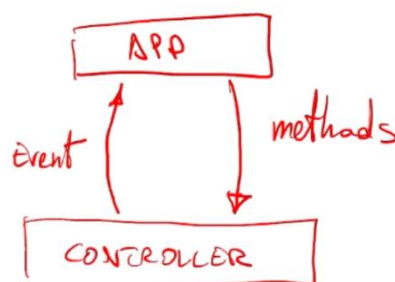
- Run on top of the controller
- Main possibilities given to apps:
 - Configure routing of flows
 - Balance traffic among available paths |
 - React to topology changes ←
 - Redirect traffic for specific functions



Le applicazioni vengono eseguite sopra al controller, logicamente sono sopra, non importa se eseguite sopra lo stesso HW o meno. Esse possono:

- configurare il routing, dispositivo per dispositivo ed eseguire un algoritmo di routing in un punto centrale.
- decidere di usare un multiple path per connettere due end point per bilanciare anche il traffico.
- decidere cosa fare quando c'è un fallimento.
- forzare il flusso per passare attraverso un firewall per implementare sicurezza

Reactive vs Proactive



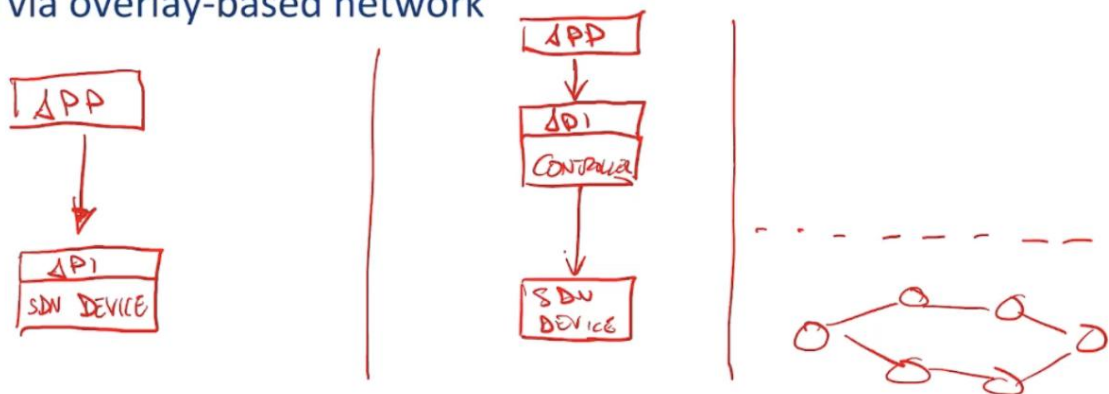
L'applicazione SDN si registra come listener per determinati eventi e il controller invocherà il metodo di *callback* dell'applicazione ogni volta che si verifica un evento del genere. Questa invocazione sarà accompagnata dai dettagli appropriati relativi all'evento.

Ci sono diversi paradigmi di implementazione di una applicazione. Si può implementare in modo proattivo o reattivo: nel primo abbiamo un controller che permette all'applicazione di chiamare metodi, per magari configurare un tunnel proattivamente (in anticipo), ciò potrebbe essere inefficiente ma quando il cliente arriva la configurazione c'è già; nel secondo ci sono degli eventi, ci si sottoscrive ad eventi e l'applicazione reagirà chiamando i metodi (esempio notificami quando esce un nuovo set di endpoint).

Riassumendo: nella *Proactive* prendi decisioni in anticipo in modo tale da essere pronto mentre nella *Reactive*, non prepari nulla quindi non sprechi risorse ma richiede tempo a reagire.

Multiple “SDN views”

- SDN via device API
- SDN via controller API
- SDN via overlay-based network



SDN via device API: hai un dispositivo, che può essere anche legacy, come un vecchio dispositivo Cisco, può essere configurato con l'interfaccia al linea di comando e può essere trasformato in un SDN aggiungendo un API sopra (può essere programmato per attivare disattivare interfacce ecc.). Viene trasformato in un dispositivo programmabile.

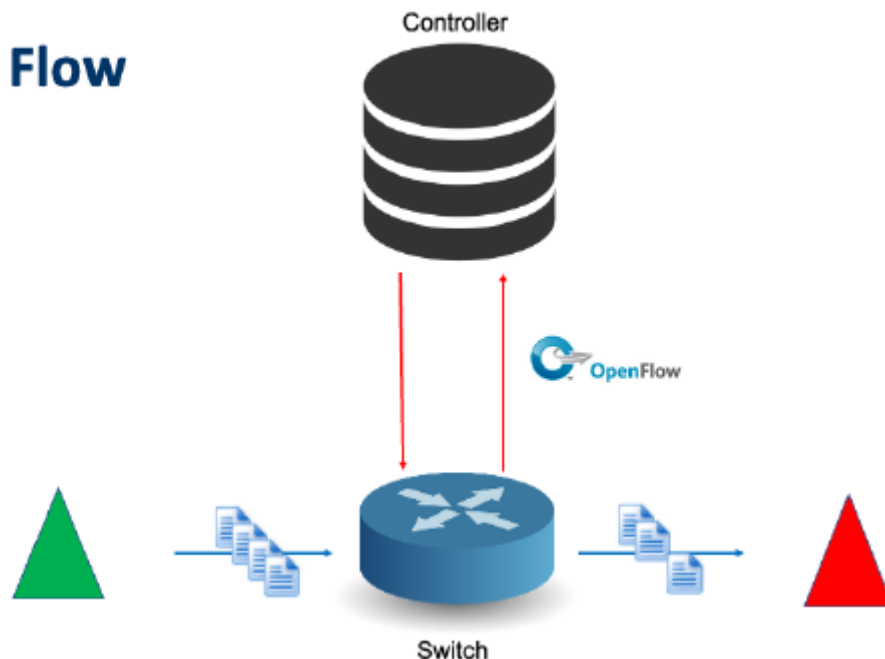
SDN via controller API: c'è un dispositivo SDN, configurato da un controller e quest'ultimo espone un API all'applicazione (visione più classica di SDN). SDN riguarda configurare la rete attraverso il software, per mezzo di API.

SDN tramite rete basata su overlay: SDN tramite reti overlay basate su hypervisor è adatto ad ambienti come i data center che eseguono già software di virtualizzazione di elaborazione e archiviazione per i loro server. Con questo concetto, la rete fisica attuale viene lasciata così com'è, con i dispositivi di rete e le loro configurazioni che rimangono invariati. Al di sopra di tale rete, tuttavia, vengono erette reti virtualizzate basate su hypervisor. I sistemi ai margini della rete interagiscono con queste reti virtuali, che oscurano i dettagli della rete fisica dai dispositivi che si collegano alle sovrapposizioni.

Sono differenti modi di programmare una rete con l'approccio SDN.

OpenFlow

Open Flow



SDN non significa necessariamente OpenFlow.

Il protocollo OpenFlow definisce i messaggi e i formati dei messaggi specifici scambiati tra controller (piano di controllo) e dispositivo (piano dati).

Il comportamento OpenFlow specifica come il dispositivo deve reagire in varie situazioni e come deve rispondere ai comandi del controller.

Definizione di flusso: set di pacchetti trasferiti da un endpoint di rete (o set di endpoint) a un altro endpoint (o set di endpoint). Gli endpoint possono essere definiti come coppie di porte TCP/UDP-indirizzo IP, endpoint VLAN, endpoint del tunnel di livello tre e porta di input, tra le altre cose. Definendo i due endpoint si possono identificare set di pacchetti coerenti, si possono identificare i pacchetti appartenenti a due flow diversi, che verranno gestiti in maniera differente.

È una tecnologia SDN.

Rules Examples

- Cross-layer rules for packet classification
- Different functionalities can be implemented:
 - Switching
 - Routing
 - Firewall

Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	00:1f:...	*	*	*	*	*	*	*	port6

Flow Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
port3	00:2D:...	00:1f:...	0800	vlan1	1.2.3.4	5.6.7.8	4	17264	80	port6

Firewall

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	*	*	*	22	drop

Routing

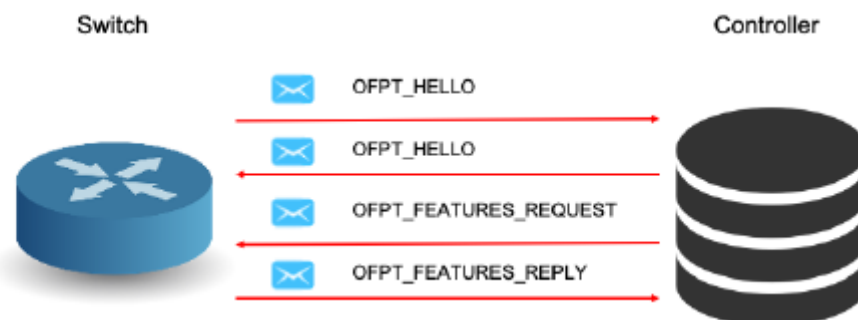
Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	5.6.7.8	*	*	*	port6

Esempi di regole nella flow table. OpenFlow definisce i possibili campi che possono essere usati nella flow table.

* → non mi interessa

OF Messages - Startup

- At startup a set of startup messages is sent to allow the controller to discover the capabilities of the switch



Il messaggio OFPT_FEATURES_REQUEST viene utilizzato dal controller per identificare lo switch e leggerne le capacità di base. Dopo aver stabilito la sessione, il controller deve inviare un messaggio OFPT_FEATURES_REQUEST. Questo messaggio non contiene un corpo oltre l'intestazione OpenFlow.

Lo switch deve rispondere con un messaggio OFPT_FEATURES_REPLY: Il campo datapath_id identifica in modo univoco un datapath. I 48 bit inferiori sono destinati all'indirizzo MAC dello switch, mentre i 16 bit superiori sono a discrezione dell'implementatore. Un esempio di utilizzo dei 16 bit superiori sarebbe un ID VLAN per distinguere più istanze di switch virtuali su un singolo switch fisico. Questo campo dovrebbe essere trattato dai controller come una stringa di bit opaca.

Il campo `n_buffers` specifica il numero massimo di pacchetti che lo switch può mettere in buffer quando invia pacchetti al controller utilizzando messaggi `packet-in` (vedere 6.1.2). Il campo `n_tables` descrive il numero di tabelle supportate dallo switch, ciascuna delle quali può avere un set diverso di campi di corrispondenza supportati, azioni e numero di voci. Quando il controller e lo switch comunicano per la prima volta, il controller scoprirà quante tabelle supporta lo switch dalla risposta `Features`. Se desidera comprendere le dimensioni, i tipi e l'ordine in cui vengono consultate le tabelle, il controller invia una richiesta multipart `OFPMPT_TABLE_FEATURES` (vedere 7.3.5.18). Uno switch deve restituire queste tabelle nell'ordine in cui i pacchetti attraversano le tabelle.

Il campo `ausiliario_id` identifica il tipo di connessione dallo switch al controller, la connessione principale ha questo campo impostato su zero, una connessione ausiliaria ha questo campo impostato su un valore diverso da zero (vedere 6.3.8). Il campo `capabilities` è una bitmap che utilizza una combinazione dei seguenti flag:

Il bit `OFPC_PORT_BLOCKED` indica che un protocollo di commutazione esterno a OpenFlow, come 802.1D Spanning Tree, rileverà i loop di topologia e bloccherà le porte per impedire i loop di pacchetti. Se questo bit non è impostato, nella maggior parte dei casi il controller dovrebbe implementare un meccanismo per impedire i loop di pacchetti.

In sintesi: il protocollo inizia con i *startup messages*, lo switch notificherà con dei messaggi. Il controller chiederà allo switch le funzionalità disponibili, in base alla versione.

Switch Operations

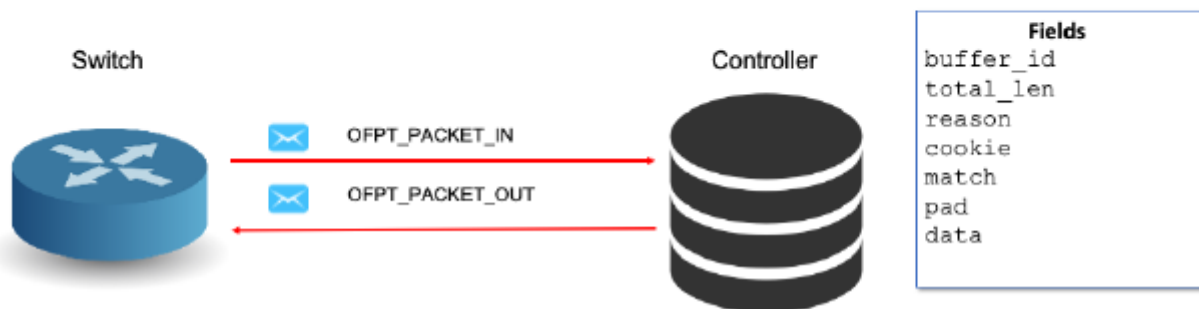
- For each received packet the Flow Table is looked up
- If a match is ***found*** the action is executed, otherwise the ***packet (or a reference) is forwarded to the controller encapsulated into a Packet-In***



Per ogni pacchetto ricevuto il forwarding engine guarda nella flow table se c'è una regola che “match” a il pacchetto, potrebbe anche non esserci un match: lo switch invia un “packet in” dicendo al controller che vede il pacchetto ma non ha trovato l'azione da eseguire, quindi il controller deciderà.

OF Messages – Normal Operations

- During normal operations switch and controller interacts with IN and OUT packets



Il `buffer_id` è un valore opaco utilizzato dal datapath per identificare un pacchetto bufferizzato. Se il pacchetto associato al messaggio `packet-in` è bufferizzato, il `buffer_id` deve essere un identificatore univoco sulla connessione corrente che fa riferimento a quel pacchetto sullo switch. Se il pacchetto non è bufferizzato, sia perché non ci sono buffer disponibili, sia perché è stato richiesto esplicitamente tramite `OFPCML_NO_BUFFER`, il `buffer_id` deve essere `OFP_NO_BUFFER`.

Gli switch che implementano il buffering devono esporre, tramite documentazione, sia la quantità di buffering disponibile, sia la durata prima che i buffer possano essere riutilizzati. Uno switch deve gestire con garbo il caso in cui un messaggio `packet_in` bufferizzato non produca alcuna risposta dal controller. Uno switch dovrebbe impedire che un buffer venga riutilizzato finché non è stato gestito dal controller, o finché non è trascorso un certo periodo di tempo (indicato nella documentazione).

Il `total_len` è la lunghezza totale del pacchetto che ha attivato il messaggio `packet-in`. La lunghezza effettiva del campo dati nel messaggio potrebbe essere inferiore a `total_len` nel caso in cui il pacchetto fosse stato troncato a causa del buffering (vedere 6.1.2). Il campo `total_len` deve sempre corrispondere al pacchetto prima del buffering e del troncamento. Il campo `total_len` conta il numero di byte nel pacchetto come definito per il Packet Type (vedere 7.2.3.11).

Il campo `reason` indica quale contesto ha attivato il messaggio di ingresso del pacchetto e può essere uno qualsiasi di questi valori: (slide successiva)

`OFPR_INVALID_TTL` indica che un pacchetto con un IP TTL o MPLS TTL non valido è stato rifiutato dalla pipeline OpenFlow e passato al controller. Il controllo di TTL non valido non deve essere eseguito per ogni pacchetto, ma deve essere eseguito almeno ogni volta che un'azione `OFPAT_DEC_MPLS_TTL` o `OFPAT_DEC_NW_TTL` viene applicata a un pacchetto. Il campo `cookie` contiene il cookie della voce di flusso che ha causato l'invio del pacchetto al controller.

Questo campo deve essere impostato su -1 (0xffffffff) se un cookie non può essere associato a un flusso particolare. Ad esempio, se il pacchetto in ingresso è stato generato in un bucket di gruppo o dal set di azioni.

Il campo di corrispondenza è un set di TLV OXM contenente i campi pipeline associati a un pacchetto. I valori dei campi pipeline non possono essere determinati dai dati del pacchetto e includono ad esempio la porta di input e il valore dei metadati (vedere 7.2.3.9). I TLV OXM devono includere il campo di corrispondenza del tipo di pacchetto per identificare il tipo di pacchetto incluso nel campo dati, a meno che il pacchetto non sia Ethernet (vedere 7.2.3.11). Il set di TLV OXM deve includere tutti i campi pipeline associati a quel pacchetto, supportati dallo switch e quale valore non è tutti bit zero. Il campo `OXM_OF_ACTSET_OUTPUT` dovrebbe essere omissso da questo set. Se `OXM_OF_IN_PHY_PORT` ha lo stesso valore di `OXM_OF_IN_PORT`, dovrebbe essere omissso da questo set. Il set di TLV OXM può facoltativamente includere campi pipeline il cui valore è tutti bit zero. Il set di TLV OXM può anche facoltativamente includere campi di intestazione del pacchetto. La maggior parte degli switch non dovrebbe includere quei campi facoltativi, per ridurre al minimo la dimensione del pacchetto in ingresso, e quindi il controller non dovrebbe dipendere dalla loro presenza e dovrebbe estrarre i campi di intestazione dal campo dati. Il set di TLV OXM deve riflettere le intestazioni e il contesto del pacchetto quando si è verificato l'evento che attiva il messaggio di pacchetto in ingresso, dovrebbero includere tutte le modifiche apportate nel corso dell'elaborazione precedente. La porta a cui fa riferimento il TLV `OXM_OF_IN_PORT` è la porta di ingresso del pacchetto utilizzata per le voci di flusso corrispondenti e deve essere una porta OpenFlow standard valida (vedere 7.2.3.9). La porta a cui fa riferimento il TLV `OXM_OF_IN_PHY_PORT` è la porta fisica sottostante (vedere 7.2.3.9). Il campo pad è un padding aggiuntivo, oltre al potenziale padding del campo di corrispondenza, per assicurarsi che l'intestazione IP del frame Ethernet sia correttamente allineata. Questo padding è incluso anche se il campo dati è vuoto. Il campo dati contiene parte del pacchetto che ha attivato il messaggio di ingresso del pacchetto, il pacchetto è incluso per intero nel campo dati o viene troncato se è bufferizzato. Il tipo di pacchetto è determinato dal campo di corrispondenza del tipo di pacchetto incluso nel campo di corrispondenza, se non è presente alcun campo di corrispondenza del tipo di pacchetto, il pacchetto deve essere Ethernet (frame Ethernet, incluso un header Ethernet e un payload Ethernet, ma nessun Ethernet FCS - vedere 7.2.3.11). L'header del pacchetto riflette tutte le modifiche applicate al pacchetto nell'elaborazione precedente, ma non include le modifiche in sospeso dal set di azioni.

Quando un pacchetto viene messo in buffer, un certo numero di byte dall'inizio del pacchetto verrà incluso nel campo dati del messaggio. Se il pacchetto viene inviato a causa di un'azione "invia al controller", questo numero di byte deve essere uguale al valore `max_len` dall'`ofp_action_output` che ha attivato il messaggio. Se il pacchetto viene inviato per altri motivi, come un TTL non valido, questo numero di byte deve essere uguale al valore `miss_send_len` configurato tramite il messaggio `OFPT_SET_CONFIG`. Il valore predefinito `miss_send_len` è 128 byte. Se il pacchetto non è bufferizzato, sia perché non ci sono buffer disponibili, sia perché è stato richiesto esplicitamente tramite `OFPCML_NO_BUFFER`, l'intero pacchetto deve essere incluso nel campo dati.

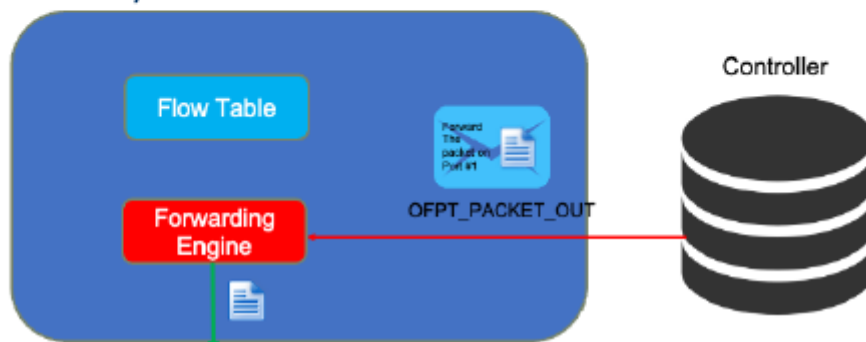
Il controller deciderà in base ai campi.

reasons for OFPT_IN

```
/* Why is this packet being sent to the controller? */
enum ofp_packet_in_reason {
    OFPR_NO_MATCH = 0, /* No matching flow (table-miss flow entry). */
    OFPR_ACTION = 1, /* Action explicitly output to controller. */
    OFPR_INVALID_TTL = 2, /* Packet has invalid TTL */
};
```

Switch Operations

- The controller can reply with a Packet-Out specifying the action to be performed (e.g. forward the packet on port #1) and the packet or the reference
- It will be executed only once (no modifications to the Flow Table)

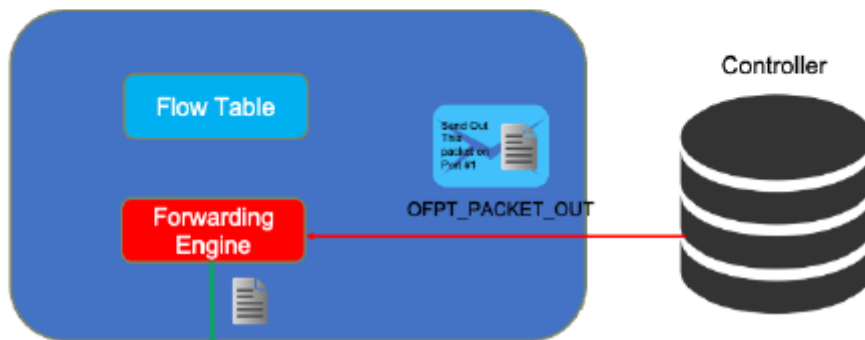


Il campo dati è vuoto oppure, quando `buffer_id` è `OF_NO_BUFFER`, il campo dati contiene il pacchetto da elaborare. Il formato del pacchetto è un frame Ethernet, che include un'intestazione Ethernet e un payload Ethernet (ma nessun FCS Ethernet). L'unica elaborazione eseguita dallo switch sul pacchetto è l'elaborazione specificata esplicitamente dalle azioni OpenFlow nel campo azione, in particolare lo switch non deve impostare in modo trasparente l'indirizzo sorgente Ethernet o il checksum IP.

Il pacchetto out dice allo switch cosa fare, una operazione alla volta (*one shot operation*).

Switch Operations

- The controller can reply with a Packet-Out specifying a new packet to be sent out
- It will be executed only once (no modifications to the Flow Table)



Il pacchetto out può essere un nuovo pacchetto e ha anche una azione.

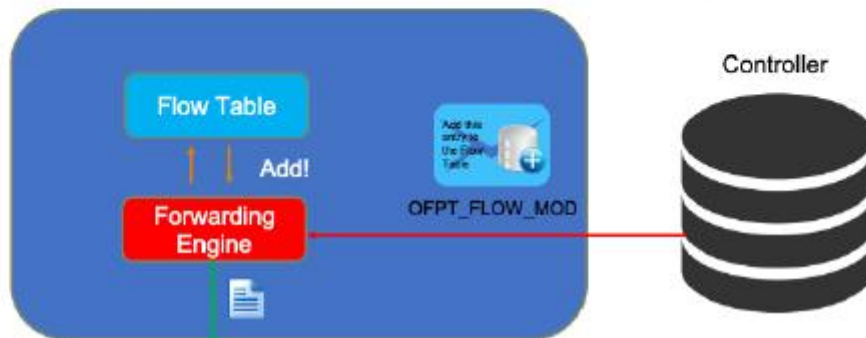
Types of actions

```
enum ofp_action_type {
    OFPAT_OUTPUT = 0, /* Output to switch port. */
    OFPAT_COPY_TTL_OUT = 11, /* Copy TTL "outwards" -- from next-to-outermost
                             to outermost */
    OFPAT_COPY_TTL_IN = 12, /* Copy TTL "inwards" -- from outermost to
                             next-to-outermost */
    OFPAT_SET_MPLS_TTL = 15, /* MPLS TTL */
    OFPAT_DEC_MPLS_TTL = 16, /* Decrement MPLS TTL */

    OFPAT_PUSH_VLAN = 17, /* Push a new VLAN tag */
    OFPAT_POP_VLAN = 18, /* Pop the outer VLAN tag */
    OFPAT_PUSH_MPLS = 19, /* Push a new MPLS tag */
    OFPAT_POP_MPLS = 20, /* Pop the outer MPLS tag */
    OFPAT_SET_QUEUE = 21, /* Set queue id when outputting to a port */
    OFPAT_GROUP = 22, /* Apply group. */
    OFPAT_SET_NW_TTL = 23, /* IP TTL. */
    OFPAT_DEC_NW_TTL = 24, /* Decrement IP TTL. */
    OFPAT_SET_FIELD = 25, /* Set a header field using OXM TLV format. */
    OFPAT_PUSH_PBB = 26, /* Push a new PBB service tag (I-TAG) */
    OFPAT_POP_PBB = 27, /* Pop the outer PBB service tag (I-TAG) */
    OFPAT_EXPERIMENTER = 0xffff
};
```

Switch Operations

- The controller can reply with a **Flow-Mod** message that instructs the switch to add a new entry to its table
- The new entry will instruct the switch to perform a certain operation without contacting the controller
- The operation associated with the new entry is then executed



Il campo cookie è un valore di dati opaco scelto dal controller. Questo valore appare nei messaggi di rimozione del flusso (vedere 7.4.2) e nei messaggi multiparte di descrizione del flusso (vedere 7.3.5.2) e può anche essere utilizzato per filtrare le statistiche del flusso, la modifica del flusso e l'eliminazione del flusso (vedere 6.4). Non viene utilizzato dalla pipeline di elaborazione dei pacchetti e quindi non deve risiedere nell'hardware. Il valore -1 (0xffffffff) è riservato e non deve essere utilizzato. Quando una voce di flusso viene inserita in una tabella tramite un messaggio OFPFC_ADD, il suo campo cookie viene impostato sul valore fornito. Quando una voce di flusso viene modificata (messaggi OFPFC_MODIFY o OFPFC_MODIFY_STRICT), il suo campo cookie non cambia. Se il campo cookie_mask è diverso da zero, viene utilizzato con il campo cookie per limitare la corrispondenza del flusso durante la modifica o l'eliminazione delle voci di flusso. Questo campo viene ignorato dai messaggi OFPFC_ADD. Il comportamento del campo cookie_mask è spiegato nella Sezione 6.4.

Il campo table_id specifica la tabella in cui la voce di flusso deve essere inserita, modificata o eliminata.

La Tabella 0 indica la prima tabella nella pipeline. L'uso di OFPTT_ALL è valido solo per le richieste di eliminazione.

Il campo comando deve essere uno dei seguenti: (slide successiva)

I campi idle_timeout e hard_timeout controllano la rapidità con cui scadono le voci di flusso (vedere 6.5). Quando una voce di flusso viene inserita in una tabella, i suoi campi idle_timeout e hard_timeout vengono impostati con i valori del messaggio. Quando una voce di flusso viene modificata (messaggi OFPFC_MODIFY o OFPFC_MODIFY_STRICT), i campi idle_timeout e hard_timeout vengono ignorati.

I valori di timeout della voce di flusso vengono utilizzati dal meccanismo di scadenza del flusso dello switch. Se idle_timeout è impostato e hard_timeout è zero, la voce deve scadere dopo idle_timeout secondi senza traffico ricevuto. Se idle_timeout è zero e hard_timeout è impostato, la voce deve

scadere in `hard_timeout` secondi indipendentemente dal fatto che i pacchetti raggiungano o meno la voce. Se sono impostati sia `idle_timeout` che `hard_timeout`, la voce di flusso scadrà dopo `idle_timeout` secondi senza traffico o `hard_timeout` secondi, a seconda di quale si verifica per primo. Se sia `idle_timeout` che `hard_timeout` sono zero, la voce è considerata permanente e non scadrà mai. Può comunque essere rimossa con un messaggio `flow_mod` di tipo `OFPP_DELETE`. L'accuratezza del processo di scadenza del flusso non è definita dalla specifica e dipende dall'implementazione dello switch.

Il campo `priority` indica la priorità della voce di flusso all'interno della tabella di flusso specificata. Numeri più alti indicano priorità più elevate quando si abbinano i pacchetti (vedere 5.3). Questo campo è utilizzato solo per i messaggi `OFPP_ADD` quando si abbinano e si aggiungono voci di flusso e per i messaggi `OFPP_MODIFY_STRICT` o `OFPP_DELETE_STRICT` quando si abbinano voci di flusso. Questo campo non è utilizzato per `OFPP_MODIFY` o messaggi `OFPP_DELETE` (non rigorosi).

Il `buffer_id` fa riferimento a un pacchetto bufferizzato nello switch e inviato al controller tramite un messaggio di ingresso pacchetto. Se nessun pacchetto bufferizzato è associato al `flow mod`, deve essere impostato su `OF_NO_BUFFER`.

Un `flow mod` che include un `buffer_id` valido rimuove il pacchetto corrispondente dal buffer e lo elabora attraverso l'intera pipeline OpenFlow dopo l'inserimento del flusso, a partire dalla prima tabella di flusso. Ciò equivale in effetti all'invio di una sequenza di due messaggi di un `flow mod` e di un inoltro di pacchetto in uscita alla porta logica `OFPP_TABLE` (vedere 7.3.6), con il requisito che lo switch debba elaborare completamente il `flow mod` prima dell'uscita del pacchetto. Questa semantica si applica indipendentemente dalla tabella a cui si riferisce il `flow mod` o dalle istruzioni contenute nel `flow mod`. Questo campo viene ignorato dai messaggi di `flow mod` `OFPP_DELETE` e `OFPP_DELETE_STRICT`. I campi `out_port` e `out_group` filtrano facoltativamente l'ambito dei messaggi `OFPP_DELETE` e `OFPP_DELETE_STRICT` in base alla porta di output e al gruppo. Se `out_port` o `out_group` contengono un valore diverso da `OFPP_ANY` o `OFPG_ANY` rispettivamente, introduce un vincolo durante la corrispondenza.

Questo vincolo è che la voce di flusso deve contenere un'azione di output diretta a quella porta o gruppo. Altri vincoli come le strutture e le priorità `ofp_match` vengono ancora utilizzati; questo è puramente un vincolo aggiuntivo. Nota che per disabilitare il filtraggio dell'output, sia `out_port` che `out_group` devono essere impostati rispettivamente su `OFPP_ANY` e `OFPG_ANY`. Questi campi vengono ignorati dai messaggi `OFPP_ADD`, `OFPP_MODIFY` o `OFPP_MODIFY_STRICT`. Il campo `flag` è una bitmap che può includere una combinazione dei seguenti flag:

Il campo `match` contiene la struttura `match` e il set di campi `match` che definiscono come la voce di flusso corrisponde ai pacchetti (vedere 7.2.3). La combinazione di campo `match` e campo `priority` identifica in modo univoco una voce di flusso nella tabella (vedere 5.2).

Il campo `importance` è l'importanza della voce di flusso. Questo campo può essere utilizzato facoltativamente dal meccanismo di espulsione della voce di flusso (vedere 6.5). Quando una voce di flusso viene inserita in una tabella, il suo campo `importance` viene impostato con i valori del

messaggio. Quando una voce di flusso viene modificata (messaggi OFPFC_MODIFY o OFPFC_MODIFY_STRICT), il campo importance viene ignorato.

Il campo instructions contiene il set di istruzioni per la voce di flusso quando si aggiungono o modificano le voci (vedere 7.2.5). Se il set di istruzioni non è valido o supportato, lo switch deve generare un errore (vedere 7.5.4.4).

Il flow mode dirà allo switch di aggiungere una nuova regola alla flow table.

Può essere fatto in maniera sincrona o asincrona. Lo switch avrà poi il match.

Flow Mod Commands

```
enum ofp_flow_mod_command {
    OFPFC_ADD          = 0, /* New flow. */
    OFPFC_MODIFY        = 1, /* Modify all matching flows. */
    OFPFC_MODIFY_STRICT = 2, /* Modify entry strictly matching wildcards and
                             priority. */
    OFPFC_DELETE        = 3, /* Delete all matching flows. */
    OFPFC_DELETE_STRICT = 4, /* Delete entry strictly matching wildcards and
                             priority. */
};
```

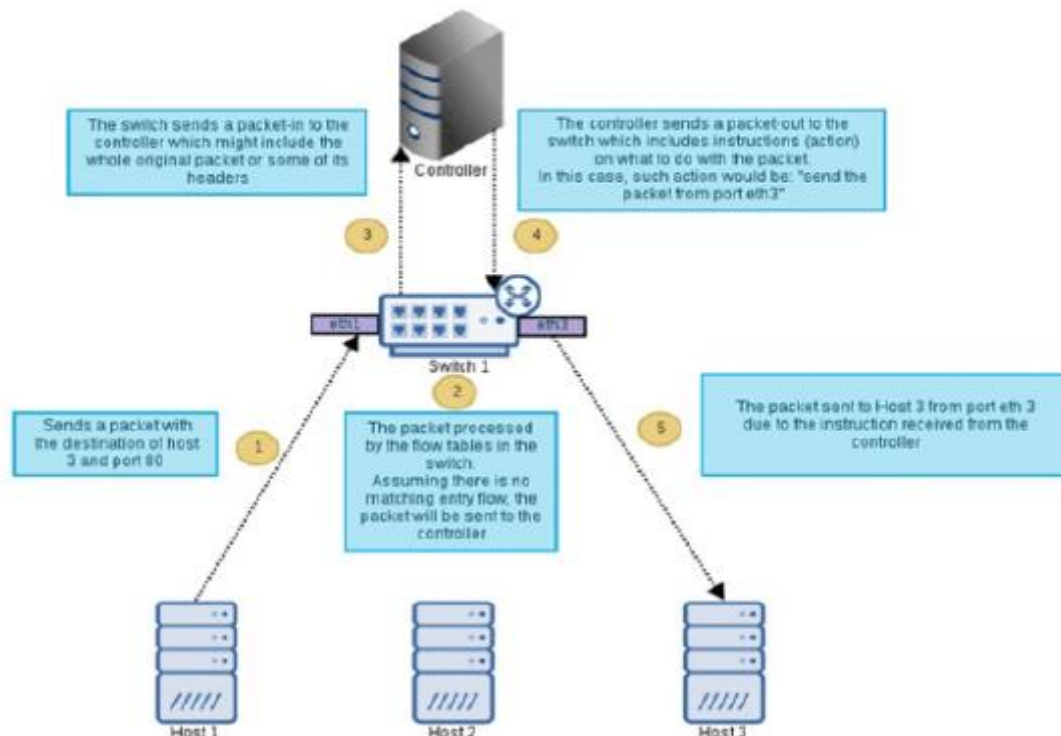
Entry Management

- Entries in the flow table expire
- As the entry is expired a Packet-In is sent to the Controller containing a **Flow-Expired** message
- Entries expire after a hard timeout (always) or after an idle timeout (if packets matching with the entry are not received)

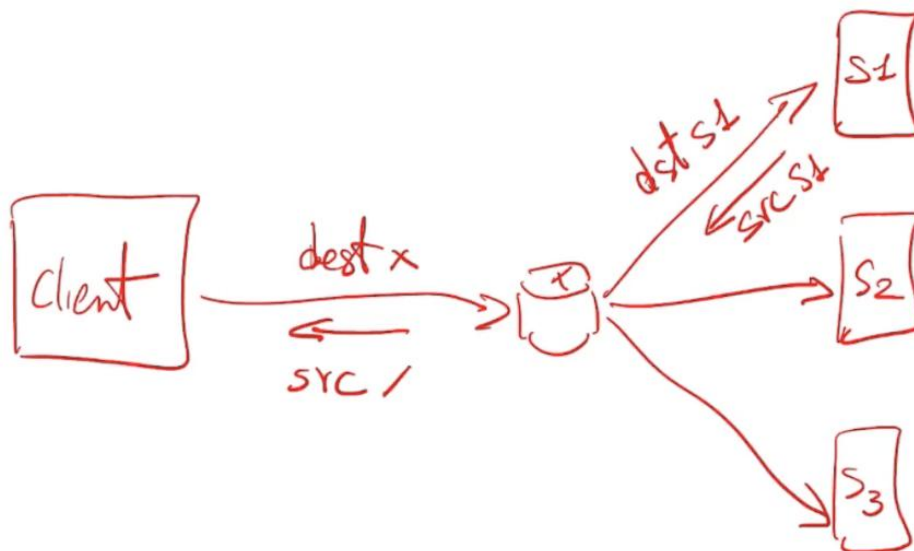


C'è un modo per gestire le entrate nella flow table, dopo un po' potrebbero scadere, viene quindi notificato il controller. Possono scadere per 2 timer: *hard time timeout* o un *soft time timeout*, il primo è quanto una entry deve stare nella tabella, il secondo invece dice quanto a lungo una entry starà nella flow table senza essere usata (quando è usato il soft timeout è resettato, viene utilizzato per mantenere solo le entrate attive).

Communication Example



Host 1 vuole parlare con host 3, manda il pacchetto allo switch con una richiesta http. Lo switch guarda la flow table, ipotizziamo sia la prima iterazione, ci potrebbe essere una *table miss*, quindi lo switch chiede al controller cosa fare con quel pacchetto. Il controller effettua dei calcoli per prendere la decisione. Se passa il pacchetto viene inoltrato alla destinazione finale.



Abbiamo un client che manda una richiesta per un servizio. Ipotizziamo che il servizio è fatto da molteplici server, questi 3 implementano lo stesso servizio per gestire la molteplicità di client. Il client sa solo il nome di x, poi si aggiunge un SDN device, devi inoltrare in base al bilanciamento del costo.

Se mando a server 1 lo scarta perché non lo trova. Il controller può scegliere uno dei server a cui associare x quando lo riceve. Al ritorno è S1 la sorgente, l'SDN device deve implementare una regola per cambiare S1 in x, in modo tale che non venga scartato quando arriva al client.

Evoluzione dell'SDN

Pertanto, aziende come Hewlett-Packard, IBM e NEC, così come numerose startup, hanno sviluppato soluzioni orientate a Open SDN e al protocollo OpenFlow.

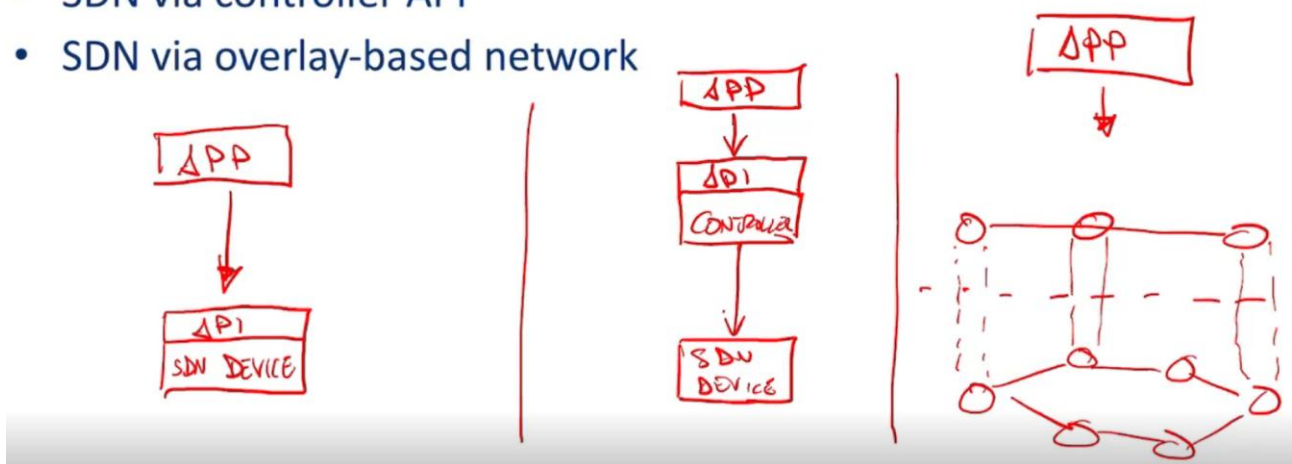
Negli ultimi anni, con l'ingresso nel mercato SDN di importanti operatori storici di reti, abbiamo iniziato a vedere dei cambiamenti.

Quando SDN ha iniziato a radicarsi e importanti fornitori come Cisco e Juniper se ne sono accorti, questo panorama e persino la definizione stessa di SDN hanno iniziato a cambiare.

Vedremo come si è evoluta l'idea di programmare il data plane dei devices utilizzando OpenFlow, come è sopravvissuta questa idea lungo gli anni.

Multiple “SDN views”

- SDN via device API
- SDN via controller API
- SDN via overlay-based network



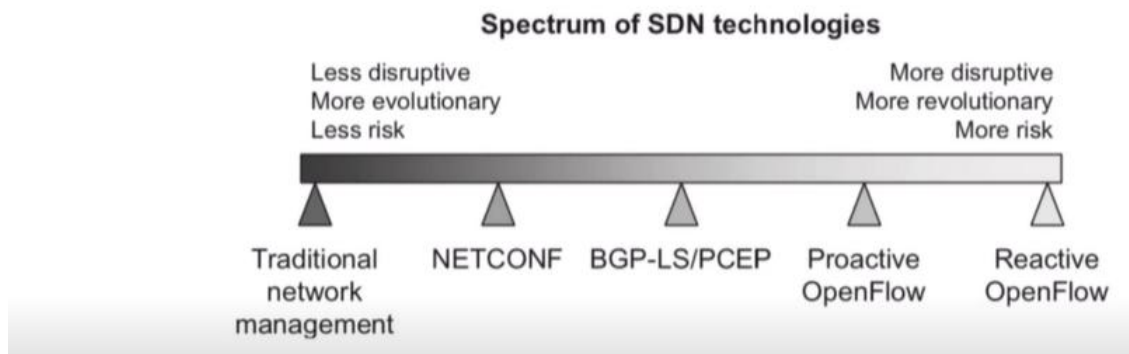
Possiamo programmare attraverso un API di un dispositivo, ovvero il manifatturiero espone un vero e proprio API esposto ai programmatori dell'applicazione.

Abbiamo un dispositivo SDN secondo uno standard, controllato da un controller, il quale espone un API per l'applicazione.

L'ultimo invece prevede di creare molteplici overlay.

Ricordiamo che le applicazioni sono usate per configurare un servizio di alto livello, non sono applicazioni classiche, per esempio configurare VMs.

Network Management vs SDN



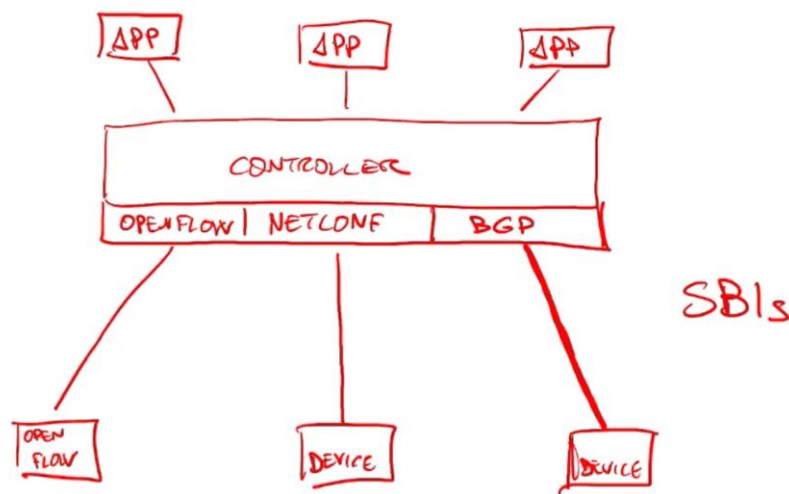
È difficile circoscrivere con precisione SDN. Ai fini di questa discussione, poiché la gestione della rete in generale condivide molti degli stessi attributi di SDN (ad esempio, controllo centralizzato, visualizzazioni a livello di rete, policy a livello di rete), consideriamo che tali soluzioni basate sulla gestione della rete rientrino anche nel più ampio spettro SDN.

Abbiamo uno spettro che si muove tra questi due punti.

Il Network management non è di per sé SDN ma cade sotto la sua stessa tipologia di tecnologia. Quello tradizionale prevedeva di configurare dispositivo per dispositivo.

Reactive OpenFlow è quella più rivoluzionaria tra tutte: cambia il modo in cui il dispositivo reagisce quando riceve un pacchetto (ricevi il pacchetto, controlla nella routing table, cambia la configurazione).

Using Existing Protocols

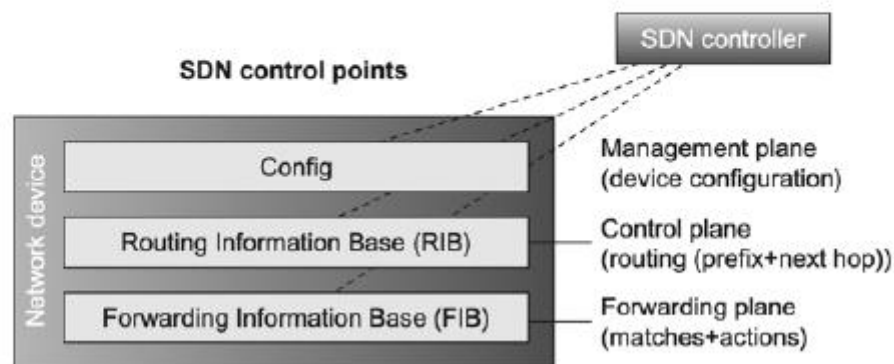


Mentre l'architettura di alto livello rimane invariata, i protocolli (SBI e NBI) e il comportamento del controller cambiano.

Abbiamo un controller logico centralizzato che permette alle applicazioni di mandare richieste. Il controller si interfaccia con i dispositivi sottostanti, per cui implementerà una o più southbound interface, può quindi comunicare con i device OpenFlow. Ci può essere anche una interfaccia *Netconf* oppure una interfaccia che usa BGP.

Nella slide seguente possiamo vedere i punti di configurazione di un dispositivo di rete:

Using Existing Protocols (2)



Mentre consideriamo questo utilizzo dei protocolli esistenti, una prospettiva utile potrebbe essere quella di guardare i diversi punti di controllo mostrati nella Figura che sono gestiti e configurati dall'applicazione SDN. Questi punti di controllo sono *Config*, dove viene eseguita la configurazione generale, Routing Information Base (RIB), dove vengono impostati i percorsi (ad esempio, prefissi e next-hop) e la Forwarding Information Base (FIB), che è di livello inferiore e riguarda la gestione dei flussi, dove le intestazioni dei pacchetti vengono abbinate e vengono intraprese azioni.

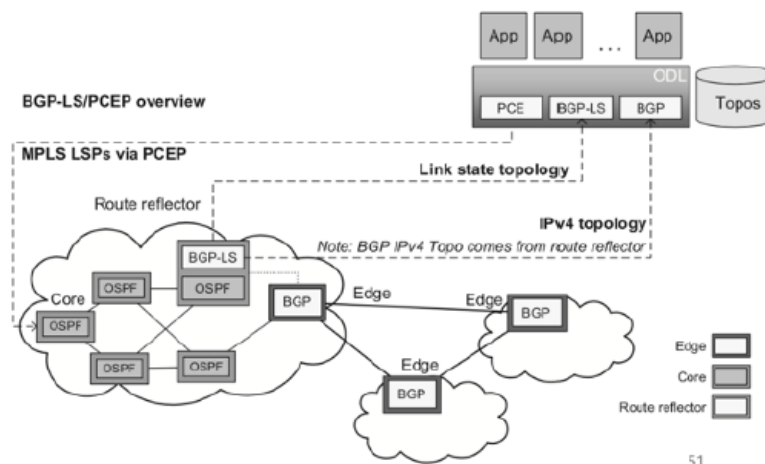
Andiamo nel dettaglio:

- Management plane → configurazione semistatica: si configurano interfacce cambiando parametri e indirizzi. Si può ad esempio usare Netconf ma ha come svantaggio il fatto che configura solo quegli elementi esposti dal dispositivo (tramite l'API fornita dal dispositivo stesso)
- RIB → ricevi un pacchetto, guardi la destinazione, effettui il match e sostituisci con il NH.
- FIB → attuale forwarding table, matcha un indirizzo all'interfaccia di uscita

Tutti questi elementi possono essere configurati dal controller o direttamente dall'applicazione.

Using existing protocols (3)

- NETCONF
- BGP
- BGP-LS
- BGP-FS
- PCE-P



51

Il controller ha due obiettivi: configurare la rete sottostante e creare una astrazione della rete, ma per fare ciò, deve ottenere informazioni dalla rete tramite protocolli esistenti:

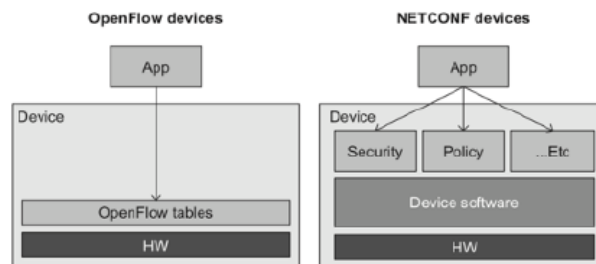
- Il ruolo di NETCONF è quello di impostare i parametri di configurazione.
- BGP è coinvolto nell'impostazione delle voci RIB
- PCE-P è utilizzato per impostare i percorsi MPLS attraverso la rete.
- BGP-LS è utilizzato per raccogliere informazioni sulla topologia dal RIB.
- BGP-FlowSpec (BGP-FS) è utilizzato per impostare corrispondenze e azioni, in modo simile a quanto fatto con OpenFlow, utilizzando invece la funzionalità BGP-FS del router. BGP-FS sfrutta l'infrastruttura BGP Route Reflection e può utilizzare BGP Route Target per definire quali router ottengono quali percorsi. A differenza di OpenFlow, BGP-FS non supporta corrispondenze di livello 2 ma solo di livello 3 e superiori.

Approfondiamo:

- BGP-LS: il protocollo BGP-LS è utilizzato da ODL per raccogliere informazioni sulla topologia dello stato del collegamento dai protocolli di routing in esecuzione nei cloud nella figura. Questa topologia riflette i router e i collegamenti di interconnessione all'interno dei domini Open Shortest Path First (OSPF) o Intermediate System to Intermediate System (IS-IS).
- BGP: il protocollo BGP è utilizzato da ODL per raccogliere la topologia IP Exterior Gateway Protocol (EGP) dai router BGP che collegano i cloud (domini) nell'immagine.
- PCE-P: il protocollo PCE-P è utilizzato da ODL per configurare i percorsi MPLS Label Switched Path (LSP) per l'inoltro del traffico attraverso tali reti.

NETCONF vs OPENFLOW

- NETCONF is a management protocol and as such it has the ability to configure capabilities which are exposed by the device
- NETCONF is a protocol developed in an *Internet Engineering Task Force* (IETF) working group and became a standard in 2006



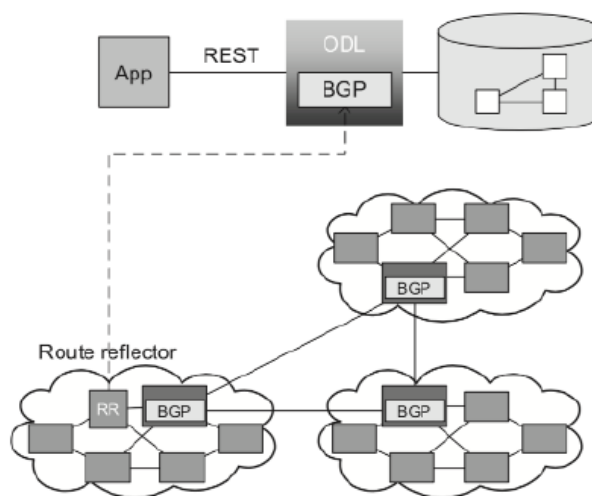
OpenFlow configura i livelli inferiori del dispositivo di rete (flow table), ovvero l'ASIC contenente il TCAM. Ciò comporta l'impostazione delle corrispondenze e delle azioni del FIB.

NETCONF, d'altro canto, esegue la configurazione tradizionale del dispositivo ma tramite il protocollo NETCONF anziché tramite CLI o SNMP. Lo sviluppatore dell'applicazione SDN è limitato da ciò che il dispositivo espone come configurabile. Se il dispositivo espone modelli di dati NETCONF che consentono la configurazione di Access Control List (ACL), l'applicazione può configurarli. Allo stesso modo, se il dispositivo ha modelli di dati per la configurazione di Quality of Service (QoS) o route statiche, anche questi saranno possibili.

I protocolli come NETCONF diventano utili tramite i modelli di dati che trasmettono informazioni e richieste da e verso il dispositivo. YANG fornisce un modo standardizzato per i dispositivi di supportare e pubblicizzare le proprie capacità. Una delle prime operazioni che si verificano tra un client NETCONF sul controller e un server NETCONF in esecuzione sul dispositivo è che il dispositivo informi il client quali modelli di dati sono supportati. Ciò consente all'applicazione SDN in esecuzione sul controller di sapere quali operazioni sono possibili su ogni dispositivo.

I controller SDN spesso supportano NETCONF basato su REST (RESTCONF) come meccanismo per la comunicazione tra il controller e i dispositivi. RESTCONF funziona come una API REST generale in quanto l'applicazione utilizzerà metodi HTTP come GET, POST, PUT e DELETE per effettuare richieste NETCONF al dispositivo.

BGP in SDN: IPv4topology

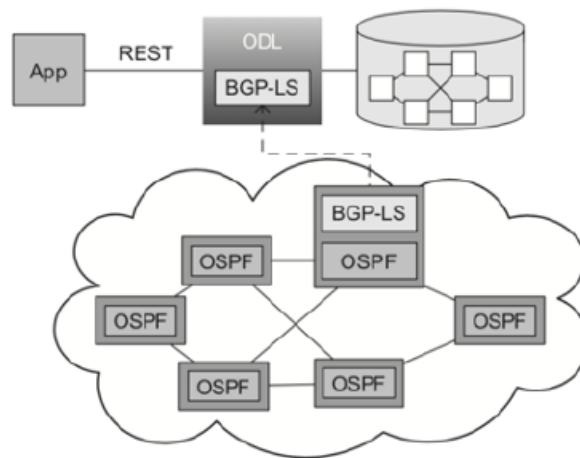


Puoi usare BGP come un southbound protocol per scoprire informazioni o raccoglierle tramite sessioni BGP.

Il plugin BGP in esecuzione all'interno di ODL implementa un nodo BGP effettivo e, in quanto tale, ha accesso alle informazioni topologiche tramite il Route Reflector (RR). Queste informazioni forniscono la topologia tra i dispositivi che implementano l'EGP, spesso definita topologia IPv4. La figura mostra una rete EGP con router che supportano BGP e un RR che comunica le informazioni sulla topologia al nodo BGP in esecuzione all'interno del controller ODL. Queste informazioni aiutano a fornire le viste a livello di rete caratteristiche delle soluzioni SDN e possono essere utilizzate per configurare dinamicamente percorsi di routing intelligenti in tutta la rete. All'interno di ODL sono presenti API per la creazione di un'applicazione RIB che può essere utilizzata per iniettare percorsi nella rete. Un aspetto fondamentale di questa tecnica è che il plugin BGP del controller ODL appare alla rete come un normale nodo BGP. Si noti che il plugin non si pubblicizza come un salto successivo (next hop) per nessuno dei percorsi. Tuttavia, apporterà modifiche ai nodi adiacenti che a loro volta propagheranno le informazioni di routing in tutta la rete. In questo modo, un'applicazione SDN in esecuzione su ODL può forzare le modifiche RIB in tutta la rete, ottenendo così un'agilità simile a SDN e una riconfigurazione automatica della rete.

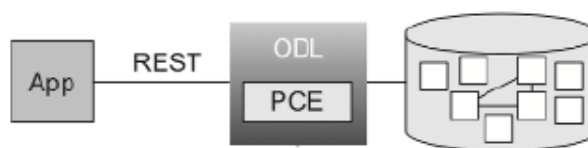
Route Reflector (RR): fai in modo che uno o più speaker iBGP agiscano come router di concentrazione, comunemente noti come route reflector. L'introduzione dei "riflettori di rotta" crea quindi una gerarchia tra gli speaker iBGP raggruppando un sottoinsieme di speaker iBGP con ogni riflettore di rotta.

BGP-LS + PCE + MPLS



La BGP entity viene eseguita nel controller, essa fa il *peering* con un altro nodo nell'AS per raccogliere informazioni sulla topologia. ODL quindi otterrà una mappa dell'AS.

BGP-LS + PCE + MPLS (2)



Puoi creare tunnel lungo la rete, da un end point ad un exit point, si può installare ad esempio attraverso i nodi MPLS. Si può fare con un PCE, ma questa scelta è fatta in base alla topologia costruita dove il tunnel verrà costruito.

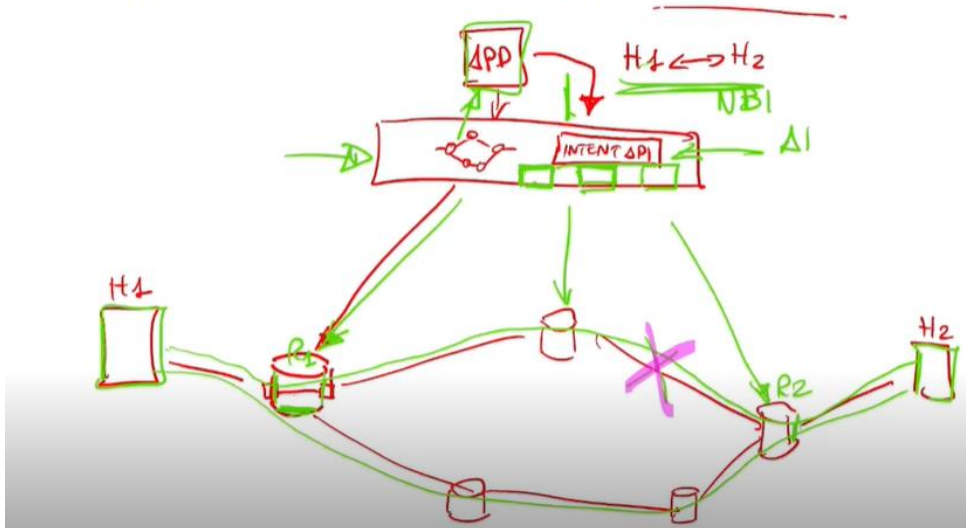
Types of SDN Applications

- Declarative vs Imperative
 - Proactive vs Reactive
 - External vs Internal
-
- Un sistema imperativo richiede che gli venga detto esattamente **come** fare qualcosa. Il risultato netto di tutti i “come” è che il sistema otterrà ciò che era stato previsto in primo luogo. In altre parole, specifica tutti i dettagli e tutti i comandi (operazioni low level) da scrivere nel device in modo imperativo, uno per uno (manda richieste al controller, configura

device A per permettere di inoltrare i pacchetti da una porta a un'altra porta ecc.). Per fare ciò, è necessario conoscere la topologia della rete.

- Un sistema dichiarativo ha solo bisogno di essere informato su **cosa** deve essere fatto; il sistema stesso capisce autonomamente come raggiungere l'obiettivo.

Example of declarative apps: Intents

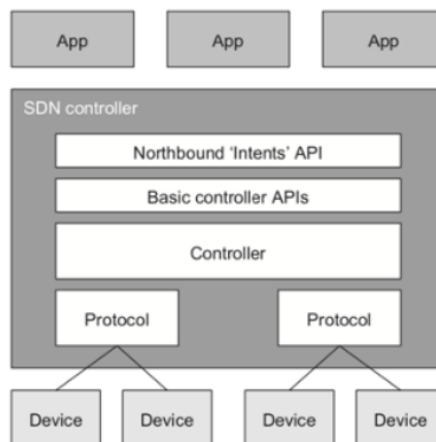


Ad esempio, un'applicazione che desidera fornire un certo livello di connettività tra due host A e B, utilizzando le API del controller di base esistenti, programmerebbe i flussi sugli switch in tutta la topologia, dallo switch che collega A alla rete, allo switch che collega B alla rete, inclusi tutti gli switch lungo il percorso tra gli host. Ciò aumenta chiaramente la complessità dell'applicazione SDN.

Il dichiarativo invece è un **intent based**, ovvero è l'opposto di quello appena descritto: l'applicazione esprime solo l'intenzione di cosa vuole ottenere, dunque non è necessario conoscere la topologia perché è tenuta dal controller. Un esempio di intento può essere il desiderio di consentire all'host A di comunicare con l'host B. L'invocazione di tale API (che ha l'obiettivo di tradurre le richieste dichiarative dall'applicazione in un set di operazioni imperative) comporterebbe che il controller SDN esegua la programmazione del flusso degli switch sul percorso dall'host A all'host B, supponendo che OpenFlow fosse il protocollo SDN in uso. Se fosse stato utilizzato un altro tipo di protocollo, ciò non avrebbe perturbato l'applicazione, poiché il controller sarebbe stato responsabile di far sì che tale connettività avvenisse con qualsiasi API southbound fosse appropriata.

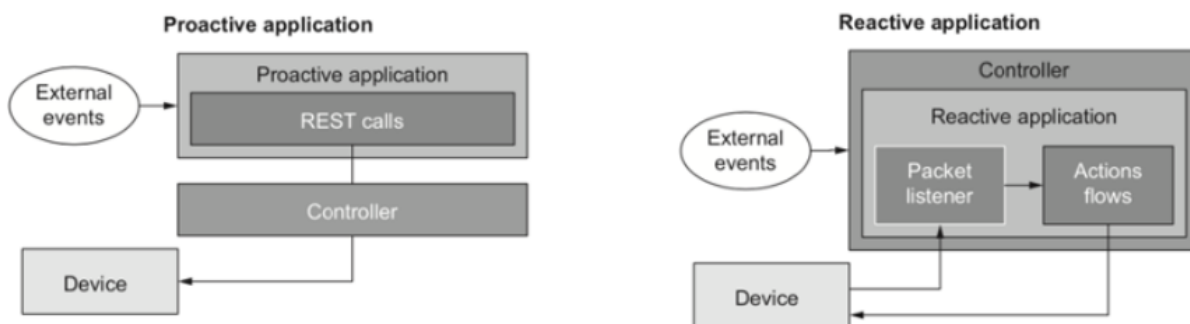
L'intento rimane nonostante quello che succede nella rete, si possono usare differenti approcci, ma in generale è il controller configurare il tutto. Se un link va down, l'intento è ancora valido, il controller deve cambiare il percorso trovandone uno alternativo.

Intent-based operations



Abbiamo il controller con una northbound che è intent oriented, poi abbiamo le API basic controller che aiutano la northbound a compiere il lavoro. Sotto si hanno uno o più southbound protocol per parlare con i dispositivi, come ad esempio BGP o OpenFlow (perché puoi anche avere protocolli diversi). Tutto viene separato, non si ha più un monolite.

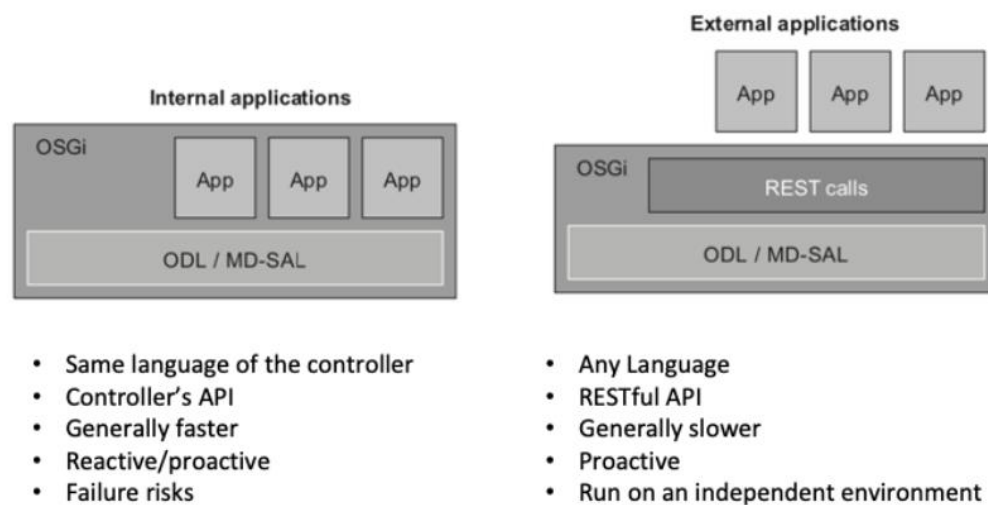
Proactive vs Reactive



Con approccio reattivo intendiamo che, quando il primo pacchetto di un flusso arriva a uno switch, il controller viene contattato per inserire una nuova voce di flusso nella tabella dei flussi. Vale a dire, le voci di flusso vengono inserite su richiesta ed eliminate quando il flusso è terminato. Sebbene ciò suggerisca che una tabella dei flussi verrebbe utilizzata in modo efficiente, vi è, tuttavia, un sovraccarico in termini di tempo di impostazione del flusso. In primo luogo, se uno switch perde la connessione al controller, allora uno switch non è in grado di ospitare un nuovo flusso.

Con un approccio proattivo, il controller calcola le voci della tabella dei flussi e vengono inserite negli switch in anticipo. Ciò elimina qualsiasi tempo di impostazione. In secondo luogo, un approccio proattivo, per sua natura, imposta le voci di flusso nella tabella dei flussi in base a regole jolly. In terzo luogo, se uno switch perde la connessione al controller, può comunque funzionare in modo indipendente per consentire il movimento del traffico attraverso la rete. Come puoi vedere, il routing aggregato si sposa bene con l'approccio proattivo.

External vs Internal



Le applicazioni interne devono essere scritte in Java (supponendo che il linguaggio nativo del controller sia Java, il che è vero per quasi tutti i controller); le applicazioni esterne possono essere scritte in qualsiasi linguaggio.

Le applicazioni interne utilizzeranno le API Java sul controller; le applicazioni esterne utilizzeranno le API RESTful.

Le applicazioni interne devono essere distribuite all'interno dell'ambiente (tipicamente OSGi) del controller, in esecuzione locale con il controller; le applicazioni esterne possono essere eseguite ovunque, anche nel cloud, se lo si desidera.

Le applicazioni interne sono relativamente più veloci; mentre le applicazioni esterne sono più lente a causa di:

1. utilizzo dell'API REST
2. possibile scrittura in un linguaggio più lento come Python
3. potenziale esecuzione in remoto dal controller.

Le applicazioni interne sono in grado di eseguire applicazioni reattive (ricezione di pacchetti inoltrati da uno switch OpenFlow); le applicazioni esterne non lo sono (perché non possono avere packet listeners essendo remote).

Sia le applicazioni interne che quelle esterne sono in grado di eseguire applicazioni proattive.

Le applicazioni interne sono in esecuzione all'interno del controller e gli errori possono avere effetti negativi sul funzionamento del controller; per le applicazioni esterne questo pericolo è meno grave.

Questi approcci non sono mutualmente esclusivi. Puoi avere applicazioni più piccole che fanno lavori diversi, tipo gestire i fallimenti. Poi avere applicazioni esterne che mandano comandi al controller.

Types of SDN Controllers

- Direct fabric programming VS Overlay
- Open vs Commercial
- Target devices
- A few examples:
 - ONOS
 - ODL
 - Tungsten Fabric (OpenContrail)
 - Calico
 - Cisco Application Centric Infrastructure
 - Big Cloud Fabric
 - Faucet

Ci sono controller più orientati al direct fabric, ovvero il controller può cambiare elementi nel livello più basso del fabric.

Exemplary SDN Controllers

- ODL (announced in April 2013 – first release Feb 2014)
- ONOS (announced 2013 – first release Dec 2014)
- Both written in java and based on karaf
- Both are part of the Linux foundation
- Functionally equivalent
- ODL -> strong focus on *backward compatibility*, more DC oriented
- ONOS -> strong focus on *Open SDN*, more service provider/telco oriented
- Different *contribution* models

Sia ONOS che ODL sono scritti in Java e progettati per un uso modulare con un'infrastruttura personalizzabile, ed è importante notare che ogni partner ONOS è anche un membro ODL. Entrambi sono progetti sotto l'ombrello della Linux Foundation. ODL è in produzione in oltre 150 organizzazioni in tutto il mondo, da grandi operatori di telecomunicazioni che supportano

collettivamente 1 miliardo di abbonati, a provider di contenuti su scala web, a imprese, organizzazioni di ricerca e università.

Network Automation

- Network automation, like most types of automation, is considered a means of doing things faster.
- There is more to automation than decreasing the time it takes to make a change.

Automatizzare il processo rende il tutto molto più veloce.

Avoid network device snowflakes

- It is still common today that network devices are configured as unique snowflakes
 - many one-off configurations
 - nonstandard configurations
- network engineers take pride in handling complexity
- The network becomes thus harder to maintain and to automate

Se si effettuano configurazioni molto precise per un device, in generale non sarà fatto in maniera standard. Per ogni problema ne esci con una configurazione nuova, la rete quindi diventa difficile da mantenere.

Automatizzare significa evitare “fiocchi di neve”, vogliamo definire procedure e algoritmi per rendere la configurazione automatica.

Expected benefits of automation

- Deterministic outcomes
- Reduce opex
- Business agility
- Enhanced security and risk reduction

Se abbiamo una richiesta per modificare la rete, vogliamo essere sicuri che il risultato sia quello atteso, se facciamo una cosa manuale non sappiamo cosa avremo, dato che il modo di modificare

dipende dal tecnico e c'è una buona possibilità di un errore umano. Se ti affidi a software è più raro e nel caso puoi fare un debug. Puoi anche fare il testing automatico rispetto a quello manuale.

Se invece dovessi fare una modifica manuale ci si mette di più ma guadagni di più.

Se trovi una nuova cosa nel mercato, puoi usare procedure ben stabilizzate per cambiare poi la rete. Ogni cosa è verso il ridurre il rischio ed aumentare la sicurezza. Applichi i cambiamenti della rete direttamente nella produzione, puoi testare il tutto prima di attivare le modifiche, poi vai nella produzione.

Understand the process before automating it

- First: learn and understand a configuration process and its implications
- Then: automate it depending on the requirements

Per automatizzare il processo bisogna capire quali sono gli step della mia procedura, automatizzi qualcosa che sai che funziona in pratica

Automazione e Programmazione della Rete

Ora discutiamo come automatizzare le procedure di configurazione e mantenimento della rete.

Network Automation

- Network automation, like most types of automation, is considered a means of doing things faster.
- There is more to automation than decreasing the time it takes to make a change.

L'obiettivo è rendere il tutto più veloce, in particolare automatizzare operazioni comuni e ripetitive.

Expected benefits of automation

- Deterministic outcomes
- Reduce opex
- Business agility
- Enhanced security and risk reduction

Scriviamo un programma che descrive un insieme di procedure, quindi significa che abbiamo il pieno controllo di quello che facciamo e ci aspettiamo un risultato deterministico essendo software.

Si può fare debugging e si può automatizzare il monitoraggio della rete, tutto questo al fine di ridurre il rischio.

Understand the process before automating it

- First: learn and understand a configuration process and its implications
- Then: automate it depending on the requirements

Per automatizzare qualcosa devi capire come fare il processo manuale, capisci le implicazioni e le difficoltà, poi in base ai requisiti puoi automatizzare.

Sappiamo come effettuare le configurazioni manuali, sappiamo alcuni errori che possono nascere, dobbiamo capire come automatizzare la procedura di configurazione dei container in primis.

Types of network automation

- Device provisioning
- Data collection and enrichment
- Migration
- Configuration Management
- Automating Data Collection

Dobbiamo partire dagli *use cases*, ovvero dove possiamo applicare l'automatizzazione della rete. Ce ne potrebbero essere di più rispetto a quelli della slide.

- Device provisioning: configurazione iniziale dei devices
- Data collection and enrichment: abilità di estrarre dati dai dispositivi e dalla rete in maniera automatica, ma anche arricchirli.
- Migration: non abbiamo solo in venditore ma molti di più, potremmo finire a muovere le configurazioni da un venditore ad un altro.
- Configuration Management: controllo dell'intero processo di configurazione a runtime
- Automating Data Collection: non solo collezionare dati da un device ma da tutti

Device Provisioning

- E.g.: creating and applying initial device configuration

```
sudo vim /etc/netplan/00-installer-config.yaml

# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s1:
      dhcp4: true
  version: 2
```

Il primo processo quando costruisci una rete, quando installi le componenti della rete (che sia fisica o virtuale).

Quando esegui una virtual machine all'inizio non si connette alla rete perché non ha un IP quindi bisogna chiamare il server DHCP: questo è un esempio di device provisioning, perché quel dispositivo non aveva un valore nella configurazione, quindi vogliamo che ciò avvenga in maniera automatica.

L'obiettivo è evitare di avere errori stupidi più che velocizzare.

Avremo un file di configurazione da qualche parte: all'inizio il sistema legge quel file e applica le configurazioni presenti. Nella slide è presente il *config* di Linux, il quale contiene tutte le interfacce configurate nel sistema. Di default solo una interfaccia è attiva.

Dobbiamo specificare il *dhcp = true* per la nostra interfaccia interessata, modificando manualmente quel file (primo step nel device provisioning).

Si può automatizzare il modo di creare il file di configurazione.

Device provisioning (1)

- Two-step process
 - Create initial configuration file
 - Pushing it to the device
- For the configuration file we need to decouple
 - the inputs (configuration parameters)
 - the underlying vendor-proprietary syntax (CLI) of the configuration
- Tools: Ansible or Nornir

Inizialmente bisogna creare l'iniziale file di configurazione possibilmente in un "template oriented manner", ovvero abbiamo una struttura del configuration file, possiamo specializzarla ma in generale possiamo crearne tante personalizzate. Bisogna poi inserirle nei device: da un remote host si creano tutti i configuration file, che poi verranno "pushati" nei dispositivi.

Dobbiamo disaccoppiare gli input e la sintassi per la configurazione: pensa di avere per ogni device 4 interfacce, alle quale bisogna assegnare IP statici da un insieme, quindi per ogni interfaccia dobbiamo mettere il *dhcp* nella struttura del configuration file. Vogliamo avere un modo automatizzato per creare un insieme di configurazioni di file tramite input e sintassi di configurazioni.

Con i tool in figura si può fare l'automatizzazione che prende input e la struttura dei configuration file, creare l'insieme dei file personalizzati e poi inviarli ai dispositivi.

Device Provisioning (2)

Configuration file

```
hostname leaf1
ip domain-name ntc.com
!
vlan 10
  name web
!
vlan 20
  name app
!
vlan 30
  name db
!
```

Per configurare i router si possono configurare i protocolli o inserendo comandi nella command line oppure scrivendo grandi file di configurazione che contengono una lista di sessioni. Vediamo in figura che il file ha una certa struttura, infatti bisogna inizialmente identificare quale.

Sotto le prime due righe c'è una lista di interfacce, per crearne molte possiamo usare la keyword "loop".

Device Provisioning (2)

Configuration file

```
hostname leaf1
ip domain-name ntc.com
!
vlan 10
  name web
!
vlan 20
  name app
!
vlan 30
  name db
!
```

Template

```
!
hostname {{ inventory_hostname }}
ip domain-name {{ domain_name }}
!
{% for vlan in vlans %}
vlan {{ vlan.id }}
  name {{ vlan.name }}
{% endfor %}
!
```

Data model

```
---
hostname: leaf1
domain_name: ntc.com
vlans:
  - id: 10
    name: web
  - id: 20
    name: app
  - id: 30
    name: db
```

Creiamo un data model, ovvero un file che modella le informazioni del nostro host, passiamo il template e possiamo creare quante configurazioni vogliamo.

Data collection and enrichment



```
student@student1:~$ ip addr list enp0s1
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d2:db:e3:a6:e1:5a brd ff:ff:ff:ff:ff:ff
    inet 192.168.129.2/24 metric 100 brd 192.168.129.255 scope global dynamic enp0s1
        valid_lft 82465sec preferred_lft 82465sec
    inet6 fe80::d0db:e3ff:fe06:e15a/64 scope link
        valid_lft forever preferred_lft forever
```

```
from netmiko import ConnectHandler
```

```
device = ConnectHandler(
    device_type='cisco_ios',
    host='csr1',
    username='ntc',
    password='ntc123'
)
```

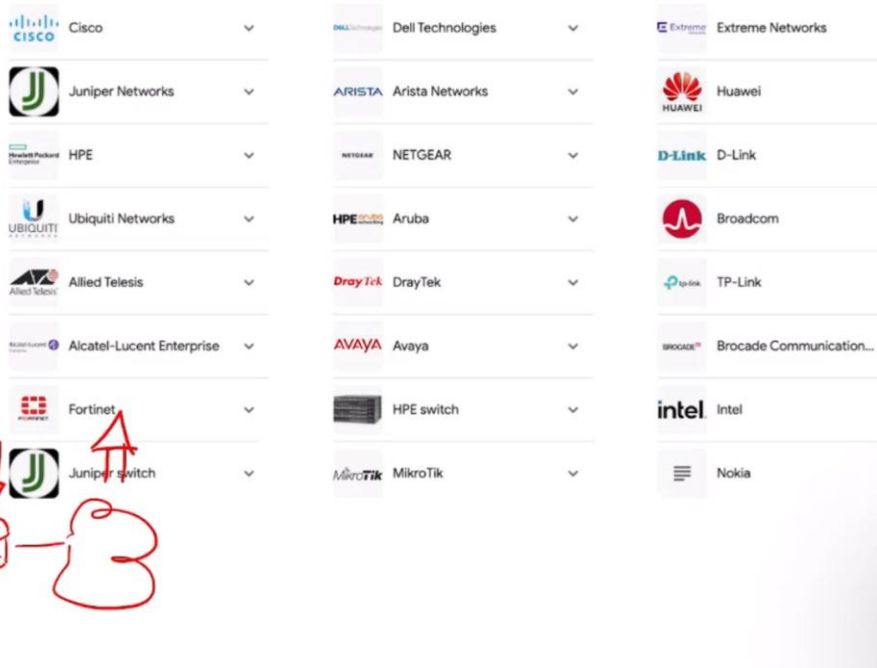
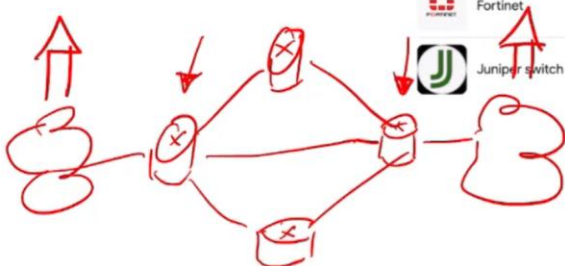
```
output = device.send_command('show version')
print(output)
```

Per data collection and enrichment cosa si intende? Vogliamo estrarre informazioni riguardo ai nostri dispositivi. Prima cosa che si fa quando si esegue una VM si controllano le interfacce, idem per questa parte, si lanciano dei comandi tipo quelli per ottenere le informazioni delle interfacce dei device.

Vogliamo automatizzare le connessioni per evitare di usare “ssh” ogni volta ed eseguire i comandi.

Migration

- Multivendor environment



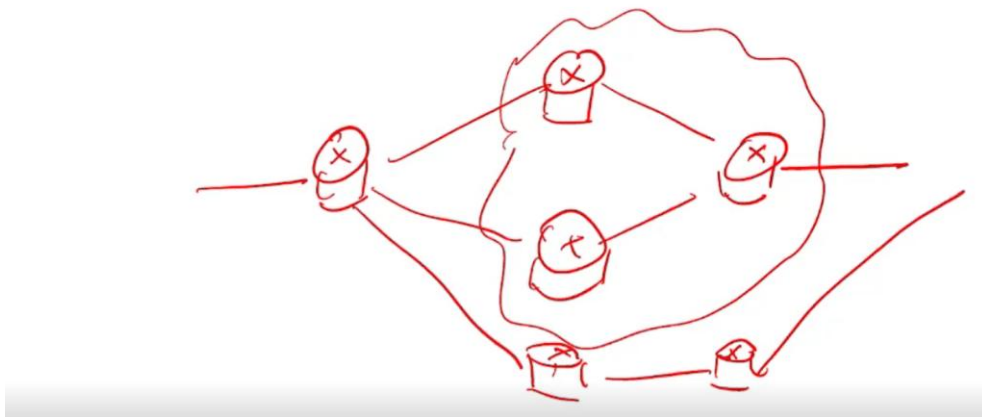
Ipotizziamo di avere la nostra per portare il traffico da una rete all'altra, magari si ha una transit network. Decidi di cambiare alcuni dispositivi perché sono vecchi o per altri motivi, quindi devi cambiare la configurazioni di quei dispositivi quando stanno lavorando: questo richiede tempo,

soprattutto su grandi reti, ma il problema più grande è avere dispositivi di altri venditori, che portano ad avere diversi comandi e configurazioni di protocolli → cambiare le configurazioni in maniera scalabile è complicato.

Non si può andare dispositivo per dispositivo e modificarli, è inimmaginabile.

Migration (1)

- Benefit in Disaster recovery



Alcuni piccoli datacenter possono fornire diverse reti per garantire la resilienza. Ipotezziamo di avere la rete in figura, la parte cerchiata può magari rompersi, in questo caso devi comunque garantire il servizio, per cui si avrà una rete di backup necessaria per gestire il traffico. Reagire velocemente è importante dunque l'automatizzazione è importante.

Parliamo di come configurare un device, pushare la configurazione nel device e gestirla successivamente.

Configuration Management

- We define configuration management as deploying, pushing, and managing the configuration state of a device.
- When you start automating configuration management, remember, with great power comes great responsibility.

Automatizzare la rete è un potere grande, ma può fare anche del grande casino e rompere tutto.

Molti ISP utilizzano queste possibilità: automatizzare le configurazioni di moltissimi dispositivi.

Prima di mandare un software in produzione si effettua un massiccio testing per vedere se le configurazioni funzionano. Con i prodotti hardware invece non puoi spendere altri soldi in testing.

Puoi invece usare dei dispositivi virtuali che assomigliano alla topologia della tua rete che si comportano allo stesso modo. La rete virtuale, quindi, può essere testata semplicemente, ma questo non evita tutti gli errori anche hardware nonostante li limiti molto.

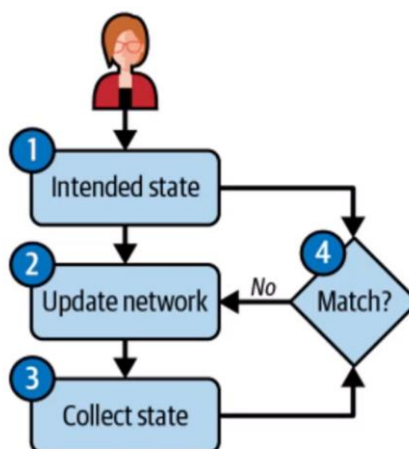
Automating data collection

- Configuration compliance
- State validation
- Reporting
- Troubleshooting

Perché dobbiamo automatizzare la collezione di dati?

1. Configuration compliance: basata su file di configurazione, i quali possono essere controllati per vedere se c'è una certa qualità e aderenza agli standard
2. State validation: la configurazione è quella che dici tu, lo stato è quello che fa il dispositivo; quindi l'importante è come il dispositivo applica la configurazione e come lo stato evolve nel tempo quando il dispositivo interagisce con la rete
3. Reporting: non devi solo guardare alla configurazione del device, ma come sta funzionando (velocità, errori ecc..)
4. Troubleshooting: scrivere *print*, vedere i valori dei dati, migliorare le performance ... questo è parte della risoluzione dei problemi, nel capire come funziona il nostro programma.

Putting everything together



Mettere tutto insieme è l'obiettivo dell'automatizzazione di rete.

Inizialmente si designa quello che vuole essere l'obiettivo finale (intended state), si prepara e in modo automatico si aggiorna la rete in modo automatico, poi si monitora lo stato per vedere se c'è

un match con lo stato voluto. Quindi non si guarda solo al file di configurazione ma anche allo stato (cosa succede ai flussi di pacchetti? Ecc.). Se c'è un match tutto ok ma continuo a monitorare, se non c'è effettuo *Troubleshooting*.

Tutto questo deve essere un loop automatizzato in software, questo perché i comandi sono fatti per gli umani, il loro risultato pure, ma con il software tutto questo cambia.

Queste operazioni possono essere fatte nelle fase di produzione ma è rischioso, infatti, in generale, sono fatte in ambienti di test. Questi loop poi sono usati anche nella fase di produzione.

First automation attempt: Linux networking

- Automate the following procedure
 - creation of a custom network (my-net)
 - creation of two docker container (alpineA and alpineB) attached to said network
 - perform a ping from alpineB towards alpineA
 - kill and destroy containers

Per fare questo useremo il *bash scripting*, il modo più semplice di automatizzare in Linux. Si mettono i comandi bash visti primi in uno script in sintassi bash.

Automating Linux Networking

- Create a bash script

```
1  #!/usr/bin/bash
2  docker network create --driver bridge my-net
3
4  docker run -dit --name alpineA --network my-net alpine
5
6  docker run -dit --name alpineB --network my-net alpine
```

- make it executable -> `chmod +x myscript.bash`

Il *Bash scripting* prevede un semplice file di testo, nel quale vengono inseriti i comandi. Con un comando poi si rende lo script eseguibile. Vedrai che alcuni comandi non sono uguali perché prima erano interattivi ora non lo sono più.

Il primo problema è che, quando si fa ping da A a B, per farlo, bisogna entrare nella macchina A. Mi serve un modo per evitare di entrare nella macchina: c'è un comando per dire a Docker di eseguire un comando, ad esempio, in Alpine B: questo comando è `exec`.

`Exec ping 172.18.0.2` → se lo lancio così non ritorna niente, continua a pingare: c'è un modo per fare solo un ping e poi stoppare.

- Simple solution

```
1  #!/usr/bin/bash
2  docker network create --driver bridge my-net
3
4  docker run -dit --name alpineA --network my-net alpine
5
6  docker run -dit --name alpineB --network my-net alpine
7
8  docker exec -it alpineA sh -c "ping -c 1 172.18.0.2"
9
10 docker kill alpineA alpineB
11
12 docker rm alpineA alpineB
```

I comandi vanno trasformati da interattivi a non interattivi.

Quello presente in figura è uno *snowflake*, è una specifica configurazione per Alpine A, stiamo facendo “*hardcoding*”. Bisogna trovare un modo di fare una discovery dell’IP di Alpine B.

- A better solution:

- avoid hardcoded constant values
- How do I get the IP address of alpineA?

- A better solution:

```
1  #!/usr/bin/bash
2  docker network create --driver bridge my-net
3
4  docker run -dit --name alpineA --network my-net alpine
5
6  docker run -dit --name alpineB --network my-net alpine
7
8  IP_ADDR=$(docker exec -it alpineA sh -c "ip addr list eth0" | grep inet | awk '{print substr($2,1,length($2)-3)}')
9
10 docker exec -it alpineB sh -c "ping -c 1 $IP_ADDR"
11
12 docker kill alpineA alpineB
13
14 docker rm alpineA alpineB
15
16
17
```

Il problema del bash scripting è che risulta molto imperativo, senza guardare lo stato della rete.

Abbiamo bisogno di un modo per rappresentare le caratteristiche e le informazioni riguardo ad un dispositivo in forma di file, come ne abbiamo avuto bisogno in questo caso: un dispositivo Alpine A ha un interfaccia con un indirizzo IP, per renderlo gestibile da programmi esterni abbiamo bisogno che questa informazione sia strutturata.

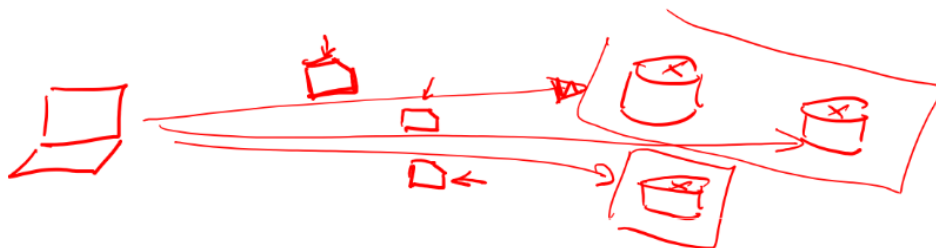
Bash Scripting: drawbacks

- The most notable drawback is that bash scripts are imperative: they'll execute a command whether it needs to be executed or not
- A bash script will attempt to add an interface regardless of whether the interface has already been added.
- This is in contrast to declarative systems, in which changes are made only if necessary.

Data Formats, Models, Templates

Discutiamo di dati riguardo ai dispositivi e alle reti, in modo tale da formattarli e modellizzarli, fino a muoverci verso il *templating*.

Why defining data formats



Abbiamo un dispositivo o un set di dispositivi da qualche parte, noi lavoriamo in un computer remoto e in qualche modo definiamo un set di comandi da lanciare in questi device per configurarli. Andiamo a lanciare comandi con dei parametri per essi (i parametri cambiano ma i comandi sono gli stessi): ci serve un modo per dire a questi dispositivi di eseguire uno script con certi valori presi in input. Può essere complicato per una serie di ragioni:

- bisogna selezionare un formato dei dati sia *user friendly* che *program friendly*, senza errori o fraintendimenti.
- vogliamo questo formato di dati non legato ad uno specifico linguaggio: se ho uno script di configurazione scritto in python, un altro in C++, voglio usare lo stesso formato di dati per tutti i linguaggi, ovvero renderlo portabile.

Data Formats: key traits

- Structured: designed to be easier for machines to understand
- Supported: standardized and widely adopted, so you'll almost never have to rewrite your own code to handle them
- Portable: formats are language agnostic

Le altre macchine possono creare strutture dati diverse a seconda dei dati, esse devono essere facilmente convertibili in altre strutture.

Supported: non avrai mai da scrivere il tuo codice per gestire quel formato di dati, devi renderlo utilizzabile in un altro linguaggio; non serve solo la descrizione del data format ma anche tool per gestirlo.

Structured: why bother?

```
[student@student11:~$ ip addr list enp0s1
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d2:db:e3:a6:e1:5a brd ff:ff:ff:ff:ff:ff
    inet 192.168.129.2/24 metric 100 brd 192.168.129.255 scope global dynamic enp0s1
        valid_lft 82465sec preferred_lft 82465sec
    inet6 fe80::d0db:e3ff:fea6:e15a/64 scope link
        valid_lft forever preferred_lft forever
```

```
[student@student11:~$ ip addr list enp0s1
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d2:db:e3:a6:e1:5a brd ff:ff:ff:ff:ff:ff
    inet 192.168.129.2/24 metric 100 brd 192.168.129.255 scope global dynamic enp0s1
        valid_lft 82465sec preferred_lft 82465sec
    inet6 fe80::d0db:e3ff:fea6:e15a/64 scope link
        valid_lft forever preferred_lft forever
```



~~screen scraping?~~

In figura si vede un comando per prendere informazioni da un interfaccia: `ip addr`.

Ma se il bash è leggermente diverso tra un sistema e l'altro? Magari si invertono delle righe, magari il formato dell'IP indica anche la net mask? In questi casi fai scraping:

Lo **scraping** (o web scraping) è una tecnica per estrarre dati da pagine web in modo automatico, utilizzando programmi o script. Questi strumenti accedono al contenuto di una pagina, ne analizzano la struttura e ne estraggono informazioni specifiche come testi, immagini, o altri dati di interesse, che vengono poi salvati in formati come CSV, JSON o database per un uso successivo.

Structured: why bother? (1)

```
student@student1:~$ ip -j addr list enp0s1
[{"ifindex":2,"ifname":"enp0s1","flags":["BROADCAST","MULTICAST","UP","LOWER_UP"],"mtu":1500,"qdisc":"fq_codel","operstate":"UP",
"group":"default","txqlen":1000,"link_type":"ether","address":"d2:db:e3:a6:e1:5a","broadcast":"ff:ff:ff:ff:ff:ff","addr_info":[{"family":"inet","local":"192.168.129.2","prefixlen":24,"metric":100,"broadcast":"192.168.129.255","scope":"global","dynamic":true,"label":"enp0s1","valid_life_time":82273,"preferred_life_time":82273}, {"family":"inet6","local":"fe80::d0db:e3ff:fea6:e15a","prefixlen":64,"scope":"link","valid_life_time":4294967295,"preferred_life_time":4294967295}]]
student@student1:~$
```



91

Puoi usare *ip addr* con opzione *-j* option, questa produrrà lo stesso output di prima ma formattato in JSON, dato che il target di questo formato è un programma di un computer.

```
student@student1:~$ ip -j addr list enp0s1 | jq
{
  {
    "ifindex": 2,
    "ifname": "enp0s1",
    "flags": [
      "BROADCAST",
      "MULTICAST",
      "UP",
      "LOWER_UP"
    ],
    "mtu": 1500,
    "qdisc": "fq_codel",
    "operstate": "UP",
    "group": "default",
    "txqlen": 1000,
    "link_type": "ether",
    "address": "d2:db:e3:a6:e1:5a",
    "broadcast": "ff:ff:ff:ff:ff:ff",
    "addr_info": [
      {
        "family": "inet",
        "local": "192.168.129.2",
        "prefixlen": 24,
        "metric": 100,
        "broadcast": "192.168.129.255",
        "scope": "global",
        "dynamic": true,
        "label": "enp0s1",
        "valid_life_time": 82210,
        "preferred_life_time": 82210
      },
      {
        "family": "inet6",
        "local": "fe80::d0db:e3ff:fea6:e15a",
        "prefixlen": 64,
        "scope": "link",
        "valid_life_time": 4294967295,
        "preferred_life_time": 4294967295
      }
    ]
  }
}
```

Structured: why bother? (2)

Puoi renderlo comunque leggibile con il programma *jq* da concatenare al comando. L'output è lo stesso ma strutturato meglio.

Categories of Data Formats

- Markup vs data serialization
- Text-based vs binary

HTML è il tipico *markup* per descrivere un documento e renderlo visualmente leggibile.

Siamo interessati alla gestione dei dati, magari vogliamo ricercare solo uno specifico dato in mezzo agli altri.

Nel JSON ci sono molti separatori, quindi risulta meno efficiente intrinsecamente poiché contiene informazioni aggiuntive per aiutare il programma.

Differenza tra *binary* e *Text-based*:

- Text-based: sono gli esempi visti fino ad ora, le informazioni sono decodificate, sono convertite in testo e poi sono serializzate in una struttura bene definita.
- Binary: sono nella forma pura, ovvero bit.

YAML

- YAML Ain't Markup Language
- "YAML is a human-friendly data serialization language for all programming languages"

Non è un markup language, è un *data serialization language*. YAML ha il requisito di essere strutturato con un JSON ma in modo *human friendly*, in modo tale che sia l'umano che il computer possono capire le informazioni.

Understanding YAML through examples

```
- Cisco
- Juniper
- Brocade
- VMware
```

```
- Core Switch
- 7700
- false
- ['switchport', 'mode', 'access']
```

```
- Cisco    # ocsiC
- Juniper  # repinuJ
- Brocade  # edacorB
- VMware   # erawMV
```

Ogni elemento inizia con un trattino, non servono le virgolette per identificare una stringa. Puoi anche avere commenti, ovvero dei tag nei dati che non saranno letti dal programma ma ne migliorano la leggibilità.

Si possono avere differenti tipi di dati come si può vedere in figura

Understanding YAML through examples (1)

```
Juniper: Also a plant
Cisco: 6500
Brocade: True
VMware:
- esxi
- vcenter
- nsx
```

```
{Juniper: Also a plant, Cisco: 6500, Brocade: true,
VMware: ['esxi', 'vcenter', 'nsx']}
```

```
import yaml
with open("example.yml") as f:
    result = yaml.load(f)
    print(result)
    type(result)

{'Brocade': True, 'Cisco': 6500, 'Juniper': 'Also a plant',
'VMware': ['esxi', 'vcenter', 'nsx']}
<type 'dict'>
```

Si possono leggere le coppie chiave valore, con valori di diversi tipi. YAML può dare diversi modi di scrivere i dati, sempre in modo equivalente, come vedi in figura.

Il tipo in alto a destra è simile a JSON.

In Python puoi fare quello che vuoi con un YAML file.

Binary Data Formats

- Text-based data formats are inherently non efficient
- This is fine in most network-automation use cases
- There are few cases wherein having a more efficient data-representation may be required.

I Text based non sono efficienti, ciò risulta comunque buono nell'automazione delle reti perché non mandi tanti dati, dato che raramente effettui la configurazione, non c'è un grande problema in termini di efficienza ma in alcuni casi può essere importante. Se hai, ad esempio, una grande rete e cerchi di capire le performance hai molte statistiche, perciò se non sei efficiente avrai informazioni di telemetria che occupano la banda: in questi casi bisogna avere un *binary based format*.

Vogliamo dati di tipo binary: li codifichi, poi li serializzi e ottieni la struttura dati.

An Example of Binary Data Formats: Protobuf

- Protocol Buffer
- Originally designed by Google, now open source
- Two components:
 - a schema definition language (programming-language agnostic)
 - tools for automatic code-generation in a specific language



Google ha progettato un formato dati: a differenza di JSON ecc. che sono *self describing*, ovvero anche senza sapere la struttura capisci se ci sono coppie chiavi valori, se hai un binary hai solo bits, quindi hai bisogno di uno schema per definirlo. Per ogni messaggio mandi un byte per deserializzare la struttura dati per poi serializzarla. Hai anche bisogno di un tool per automatizzare il code generation, leggere dati ecc.

Protoc è un compilatore che prende il messaggio e crea un set di *definition file* (puoi anche includere e importare il definition file nel codice, tipo in python, per ottenere una classe).

Adesso passiamo a interazioni tra software e software.

Second automation attempt: creating templates

```
# This is the network config written by 'subiquity'  
network:  
  ethernets:  
    enp0s1:  
      dhcp4: true  
  version: 2
```

Abbiamo un pc che manda e riceve dati ad un altro dispositivo, abbiamo bisogno che i dati siano ben strutturati quindi di data format.

Quando scriviamo un file di configurazione, i file hanno una struttura ripetitiva, con campi ed elementi da configurare, spesso le interfacce hanno gli stessi parametri ma magari hanno alcuni parametri. Bisogna generare un data format che separi i token uno dall'altro, questo è il processo di creazione di template.

Idea of templates

- Many configuration procedures include repetitive actions:
 - inefficient
 - error prone
- One of the key benefit of automation is consistency
 - create templates for each interaction with the network

Creiamo un template per un file di configurazione, specifichi la configurazione di interfacce di rete, puoi specificare se lavorano con dhcp e altri parametri.

Abbiamo in questo esempio solo una interfaccia enp0s1 che ha dhcp4, abbiamo in generale però più interfacce, configuriamo alcune per avere vari indirizzi ecc. I file crescono in grandezza, il testo non è libero ma è strutturato, in generale sono file YAML, possiamo capire facilmente la struttura.

La nuova interfaccia è composta dal nome dell'interfaccia e set di parametri, stessa interfaccia ripetuta, possiamo non solo riconoscere il formato, possiamo riconoscere il template: il file inizia con la network section, poi ethernet ecc.. quando la riconosciamo possiamo creare un template.

Idea of template (1)

- Web use case
- Three steps:
 - create a template
 - retrieve and collect data (to be fed to the template)
 - put data in the template and render it

In tanti campi, come anche interfacce web, dove hai porzioni ripetute, tu cambi solo alcuni valori interni. Se ripeti azioni puoi sbagliare e è anche inefficiente, per esempio se fai copia incolla potresti perdere qualcosa, dimentichi di modificare qualcosa ecc.. quindi manualmente è pericoloso.

La consistenza è molto importante, se creiamo una procedura standard abbiamo un risultato prevedibile. Vogliamo creare un template per la parte comune ma permettere personalizzazione

Idea of template (2)

- Application(s) to network automation

Il tipico caso d'uso è il web: avrai una pagina per ogni membro del team, ognuno ha la stessa struttura, puoi anche separare la parte che deve essere personalizzata.

Non è solo questione di identificare dati, magari pensi di creare manualmente un template, poi inserisci i valori dei campi, ma devi sempre ricordarti dove vanno messi i valori quindi puoi dimenticarlo facilmente.

Facciamo 3 step: creiamo un template, quali sono le operazioni comuni e formare un template, a run time prendiamo i dati e li collezioniamo (possiamo avere un template per team member, per riempirlo dobbiamo accedere ad un database per ottenere i dati, possiamo magari connetterci ad un device per avere un set di indirizzi ip, quindi ci serve l'ip per un dispositivo ecc). Infine dobbiamo mettere i dati nel template in modo automatico.

Introduction to Jinja2

- *Jinja is a fast, **expressive, extensible templating engine**.*
- *Special placeholders in the template allow writing code similar to Python syntax.*
- *Then the template is passed data to render the final document.*

Provisioning: processo di installare un nuovo device, riconfigurarli, la prima cosa da fare è creare una base line configuration che sarà comune a tutti i dispositivi. Hai dispositivi, cerchi la configurazione comune, si crea con un common template. Per esempio, un campo per il nome e poi lo riempi per ogni dispositivo.

Bisogna dare la possibilità che si possa configurare poi anche dopo.

Example 0: Hello World

Template definition

```
Hello, {{name}}!
```

Input data

```
name = "World"
```

Template Engine (python)

```
environment1 = Environment()
templatel = environment1.from_string("Hello, {{name }}!")
print(templatel.render(name="World"))
```



```
[antonio@Antonio-Lavoro lecture9 % python3 exercise0.py
Hello, World!]
```

Fatto per qualsiasi tipo di file. Permette di creare un template, lasciando spazio per speciali placeholder, permettendoci di scrivere simboli simili a python.

Creeremo template file con il linguaggio Jinja, metteremo insieme il template con i dati, lo passeremo al template engine, che creerà uno o più documenti attuali. Bisogna solo installare Jinja2 con pip.

Introduction to Jinja2 (1)

- {% ... %} Statements
 - if – elif – else – endif
 - loops
- {{ ... }} Expressions
 - variables
 - variables manipulation
- {# ... #} Comments

Un template è strutturato come in figura, c'è del testo libero all'inizio, poi {{ sono costrutti del linguaggio, name è una variabile, c'è del testo statico e poi un placeholder. Passeremo input data con un certo valore, si passa ad un template engine come python dove possiamo creare linee semplici di codice, leggiamo il template passandolo all'ambiente, poi lo si renderizza passando i dati. Il template crea un risultato che viene stampato a schermo ma potrebbe essere anche messo in un file o mandato ad un dispositivo.

Introduction to Jinja2 (3)

```
1 import jinja2
2 from jinja2 import Environment, FileSystemLoader
3
4 environment = Environment(loader=FileSystemLoader("./"))
5
6 template = environment.get_template("template.j2")
7
8 local_var="my_value"
9 local_list= [ {"n1": "v1"} , {"n2": "v2"} ]
10
11
12 content = template.render(
13     var1="value1"
14     var2=local_var
15     list=local_list
16 )
17
18 print(content)
```



```
{{ var1 }}
{{ var2 }}
{{ list[0].n1 }}
```

Abbiamo vari statements, il costrutto principale sono {%}, rappresentano uno statement con dentro keywords. Possiamo creare espressioni con {{}} che rappresentano variabili o manipolazioni di variabili come operazioni matematiche o modifiche alle stringhe. Infine abbiamo i commenti con {#}. Alternerai questi costrutti del linguaggio con testo libero.

Exercises: create an interface config file

```
network:
  ethernets:
    eth0:
      addresses:
        - "10.0.0.15/24"
    eth1:
      addresses:
        - "10.0.0.43/24"
    eth2:
      dhcp4: true
```

Bisogna creare del codice per l'engine

Possiamo creare variabili locali e poi renderizzare il template passando i valori a quelli attuali

Prendo la variabile lista, prendo il primo elemento e prendo il valore associato a n1 che è un dizionario.

exercise 1: generate eth0 and eth1 with configurable DHCP

```
network:
  ethernets:
    eth0:
      dhcp4: true
    eth1:
      dhcp4: true
```

La struttura parte con la sezione network, specifichi le interfacce (wireless, ethernet..), si specificano varie interfacce ethernet con nomi diversi. Creeremo un file pienamente personalizzabile.

Exercise1: solution

```
network:
  ethernets:
    eth0:
      dhcp4: {{enabled}}
    eth1:
      dhcp4: {{enabled}}
```

```
1 environment = Environment(loader=FileSystemLoader("./"))
2 template = environment.get_template("template1.j2")
3
4 content = template.render(
5     enabled="true"
6 )
7
8 print(content)
```

exercise2: generate N interfaces with DHCP

```
network:
  ethernet:
    eth0:
      dhcp4: true
    eth1:
      dhcp4: true
    eth2:
      dhcp4: true
    ...
    eth50:
      dhcp4: true
```

```
{% for i in set %}
  {{i}}
{% endfor %}
```

Exercise2: solution

```
network:
  ethernet:
    {% for i in range(counter) -%}
      {{ base_name }}{{i}}:
        dhcp4: {{enabled}}
    {% endfor -%}
```

```
1 [...]
2
3 content = template.render(
4     counter=5
5     base_name=eth
6     enabled="true"
7 )
8
9 print(content)
```

exercise3: generate interfaces from a list, with parameters (IPs)

```
network:
  ethernet:
    eth0:
      addresses:
        - "10.0.0.15/24"
    eth4:
      addresses:
        - "10.0.0.20/24"
    eth7:
      addresses:
        - "10.0.0.40/24"
```

```
{% for v in values %}
  {{ v.field1 }}
  {{ v.field2 }}
{% endfor %}
```

Exercise3: solution

```
network:
  ethernet:
    {%- for i in interfaces %}
      {{ i.name }}:
        addresses:
          - {{ i.addr }}
    {%- endfor -%}
```

```
1  [...]
2  intf = [
3      {"name": "eth0", "addr": "192.168.10.2/24"},
4      {"name": "eth5", "addr": "192.168.10.3/24"}
5  ]
6
7  content = template.render(
8      interfaces=intf
9  )
10 print(content)
```

exercise4: conditional & automatic addr creation

```
network:
  ethernet:
    eth0:
      addresses:
        - "10.0.0.1/24"
    eth1:
      addresses:
        - "10.0.0.15/24"
    eth2:
      dhcp4: true
```

```
{% if val == "foo" %}
  something
{% else %}
  something else
{% endif %}
```

to obtain a loop index, you can use

```
loop.index
```

Exercise4: solution

```
network:
  ethernet:
  {%- for i in interfaces %}
    {{ i.name }}:
    {%- if i.dhcp == True %}
      dhcp4: true
    {%- else %}
      addresses:
        192.168.56.{{ loop.index +1 }}/24
    {%- endif -%}
  {% endfor -%}
```

```
1  [...]
2  intf = [
3      {"name": "eth0", "dhcp": True},
4      {"name": "eth5", "dhcp": False}
5      {"name": "eth9"}
6  ]
7
8  content = template.render(
9      interfaces=intf
10 )
11 print(content)
```

115

```
[antonio@Antonio-Lavoro lecture9 % python3 exercise4.py
network:
  ethernet:
    eth0:
      dhcp4: true
    eth5:
      addresses:
        192.168.56.3/24
    eth6:
      addresses:
        192.168.56.4/24
    eth7:
      dhcp4: true
    eth9:
      addresses:
        192.168.56.6/24
    eth11:
      addresses:
        192.168.56.7/24
```

Exercise4: Alternate solution

```
network:
  ethernet:
  {%- set iterator = namespace(val=2) %}
  {%- for i in interfaces %}
    {{ i.name }}:
    {%- if i.dhcp == True %}
      dhcp4: true
    {%- else %}
      addresses:
      192.168.56.{{ iterator.val }}/24
      {%- set iterator.val = iterator.val+1 %}
    {%- endif %}
  {% endfor %}
```

exercise5: passing configuration files

```
network:
  ethernet:
  eth0:
    addresses:
    - "10.0.0.2/24"
  eth1:
    addresses:
    - "10.0.0.3/24"
  eth2:
    dhcp4: true
  eth3:
    addresses:
    - "10.0.0.4/24"
```

YAML file

```
- name: eth0
- name: eth1
- name: eth2
  dhcp: True
- name: eth3
  dhcp: False
```

to load a YAML file into a variable, you can use

```
with open("my_data.yaml") as file:
    intf = yaml.safe_load(file)
```

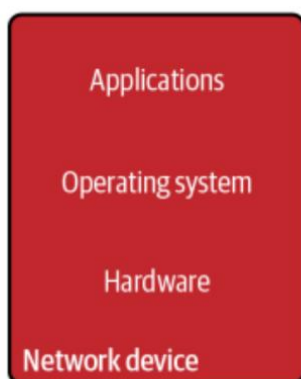
```
1 hostname HQ-SWITCH-01
2 !
3 ip domain name packet.net
4 !
5 ntp server 192.168.10.1 prefer
6 ntp server 192.168.20.1
7 !
8 interface GigabitEthernet0/0
9 ip address 10.10.25.10 255.255.255.0
10 negotiation auto
11
12 ip route 0.0.0.0 0.0.0.0 10.10.25.1
13 !
14 snmp-server community s3creT RO 6
15 snmp-server trap-source GigabitEthernet0/0
16 snmp-server location London london-hq
17 snmp-server enable traps license
18 snmp-server enable traps cpu threshold
19 snmp-server enable traps envmon
20 snmp-server enable traps errdisable
21 snmp-server host 10.10.12.15 s3creT
22 snmp-server host 10.20.12.25 s3creT
23 !
24 vlan 10
25 name isp-01
26 vlan 11
27 name isp-02
28 !
29 interface Gi 1/0/1
30 description FIREWALL-01-UPLINK
31 switchport mode trunk
32 switchport mode trunk allowed vlan 10-11
33 no shutdown
34 !
35 interface Gi 1/0/2
```

Bare Metal Switching and Open-Source Networking

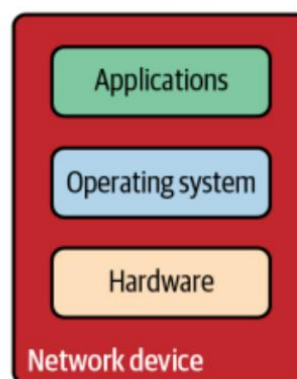
Questo che segue è solo un modo per configurare i dispositivi di rete (siamo sempre nel contesto delle configurazioni di dispositivi di rete).

What is a Bare-Metal Switch?

Traditional network device



Bare-metal (disaggregated) device



Un dispositivo di rete tradizionale, come uno switch, implementa 3 funzioni principali:

1. hardware per inoltrare i pacchetti, per il routing dei pacchetti ma più in generale per la gestione dei pacchetti
2. sistema operativo → non si eseguono funzioni a basso livello ma invece ad alto livello
3. applicazioni che offrono interfacce per aiutare un utente a configurare un dispositivo.

Tutti questi elementi sono collegati nell'hardware e tutti e 3 sono venduti da un'unica azienda. (Approccio monolitico).

C'è stato un cambio verso il **bare-metal switching**: si parla di un dispositivo **disaggregato**, cioè si separano gli elementi, si hanno quindi le stesse funzioni ma i 3 elementi sono divisi. In questo caso, sono forniti da differenti aziende.

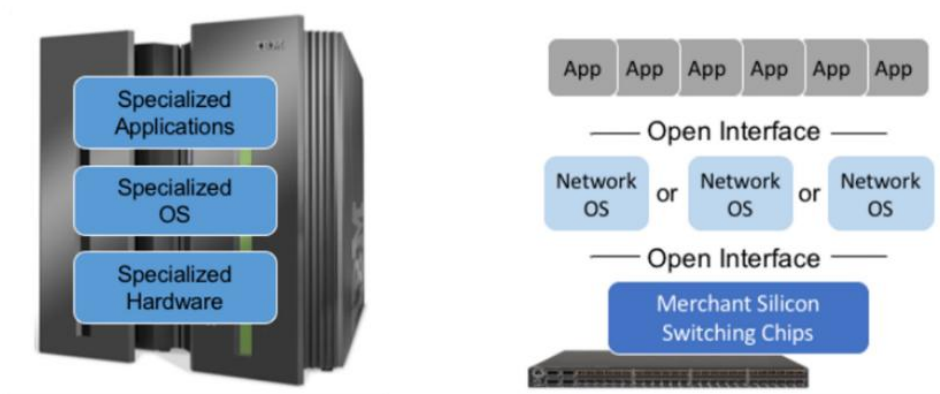
In generale, le compagnie di rete sono diventate di sviluppo software per la rete.

A parallelism: Specialized Servers vs General Purpose hardware



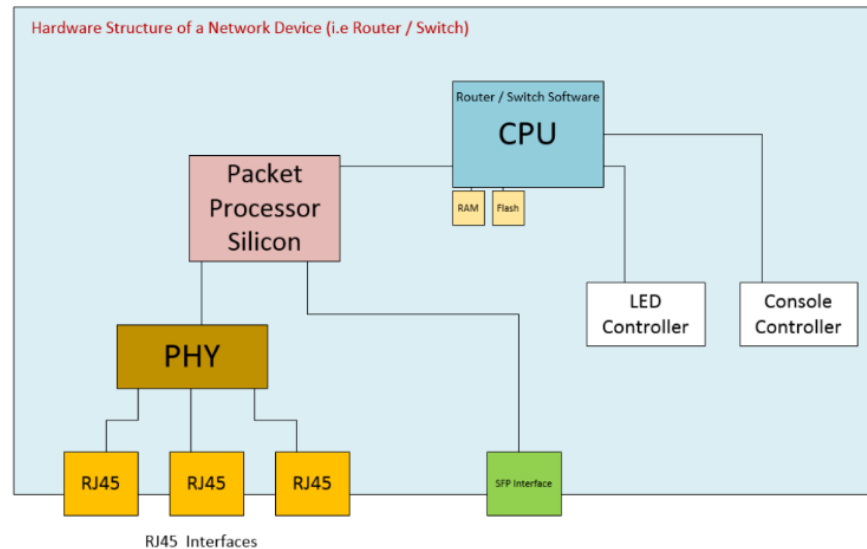
Nel mercato dei computer hardware software e sistema operativo era tutto fatto dallo stesso manifatturiero, ora, invece, si hanno vari microprocessori fatti da vari manifatturieri, si hanno tanti diversi sistemi operativi. Abbiamo applicazioni totalmente indipendenti, indipendenti dall'hardware e dal sistema operativo.

Specialized Network Device vs Disaggregated switch



È l'approccio monolitico ha portato il mercato verso la *network ossification*: l'innovazione era lenta. Con questo nuovo approccio si poteva sviluppare i vari elementi in maniera separata, quindi molti sono entrati in questo mondo, si è aperto alla community open source.

Legacy network devices

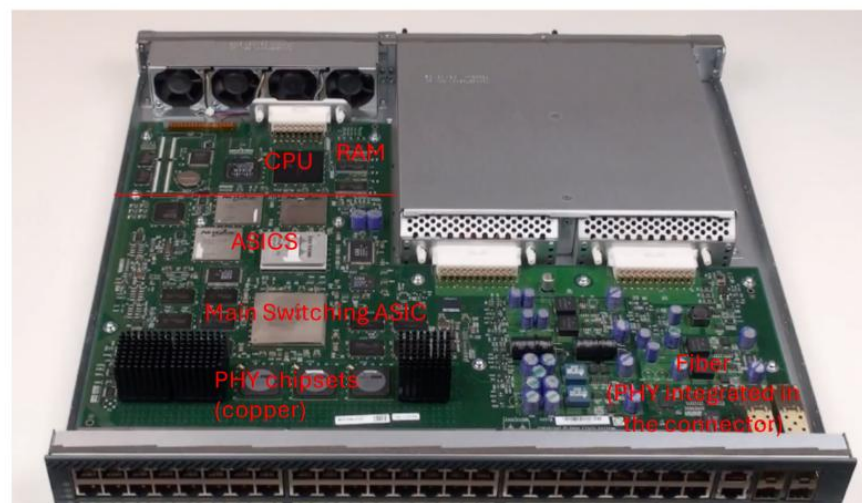


Il Packet processor è responsabile di ricevere pacchetti e prendere decisioni su di essi (l'output interface, NH...). Abbiamo anche una CPU con una RAM, oltre ad altri controller per configurare il dispositivo tramite la console, controllare i led ecc.

Infine, si ha un set di interfacce e una parte che comanda queste interfacce.

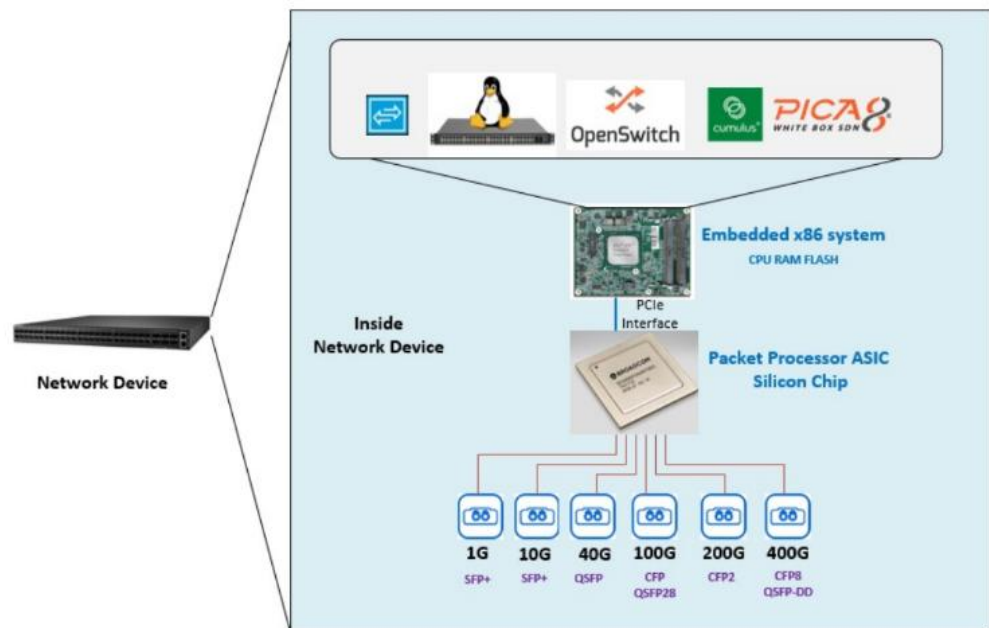
Questa è la struttura ad alto livello e tipica nei dispositivi non disaggregati.

Legacy Network Device



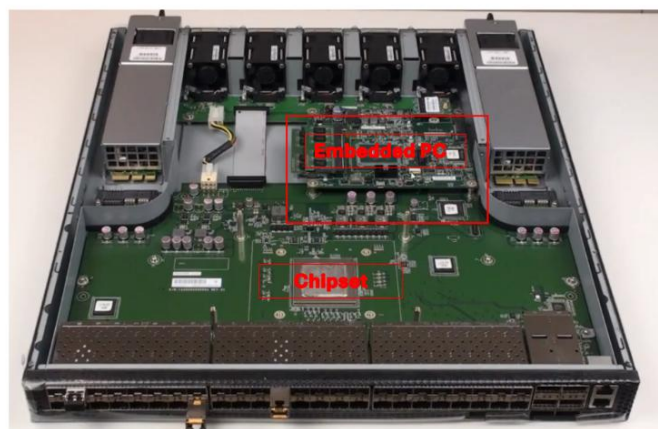
C'è una single board: un pezzo di hardware con differenti chips, progettato da CISCO in questo caso. Ci sono tutti gli elementi visti prima, come differenti ASICs che gestiscono i pacchetti. Ci sono anche i chip che controllano le interfacce (PHY).

Enter disaggregation



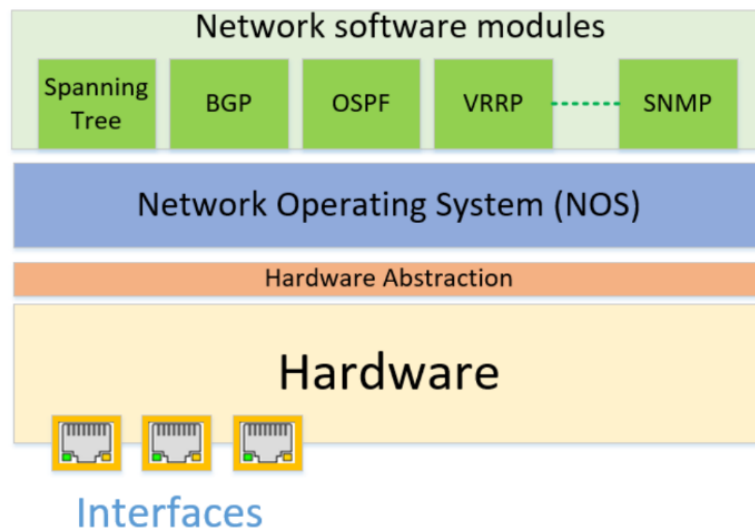
Quando è presente la disaggregation si avrà sempre un dispositivo di rete con differenti interfacce, ognuna può supportare ethernet o fibra ottica. Hanno un ASIC per gestire il packet forwarding. Sono un piccolo pc general purpose che può eseguire un sistema operativo, o differenti, cosa non vera per quello precedenti, che avevano il loro sistema operativo (CISCO OS).

Bare-metal switch



C'è un PC embedded che puoi rimuovere, con CPU, RAM... è a tutti gli effetti un PC standalone (x86). Ha ovviamente le interfacce, per le fibre ottiche, c'è anche l'ASIC chipset per gestire i pacchetti in modo efficiente. Bisogna installarci un sistema operativo di rete anche se supporta qualsiasi sistema operativo.

Network OS



Il network software sono tutti quei protocolli di rete, per i quali sono presenti vari daemons (per ogni protocollo come OSPF, BGP...). L'hardware in questo caso sono gli ASIC, che inoltrano i pacchetti.

Abbiamo un sistema operativo generico che viene eseguito su una CPU ma come interagisce con gli ASIC (circuiti personalizzati che forniscono delle funzioni per il sistema operativo)? Abbiamo un altro livello di astrazione, chiamato "Hardware Abstraction" che permette al sistema operativo, qualunque sia l'hardware, di effettuare le chiamate.

Nel legacy è lo stesso ma è tutto insieme e non ha l'hardware Abstraction.

Switch Abstraction Interface

- Defines an **abstraction interface** for switching "ASIC"
- It is designed to be **vendor independent**
- originally designed by Microsoft, now under Open Compute Project (OCP)
- Can control both *hardware switching* entities and *software switches*
- *SAI allows the same network software stack to program and manage many different switch chips without undergoing any changes*

SAI è uno standard, definisce una interfaccia di astrazione per switchare gli ASIC, astraiamo il chipset per renderli vendor independent dal sistema operativo e dall'hardware. SAI permette di usare il dispositivo visto prima con hardware virtuale o reale.

Example of Network Operative Systems



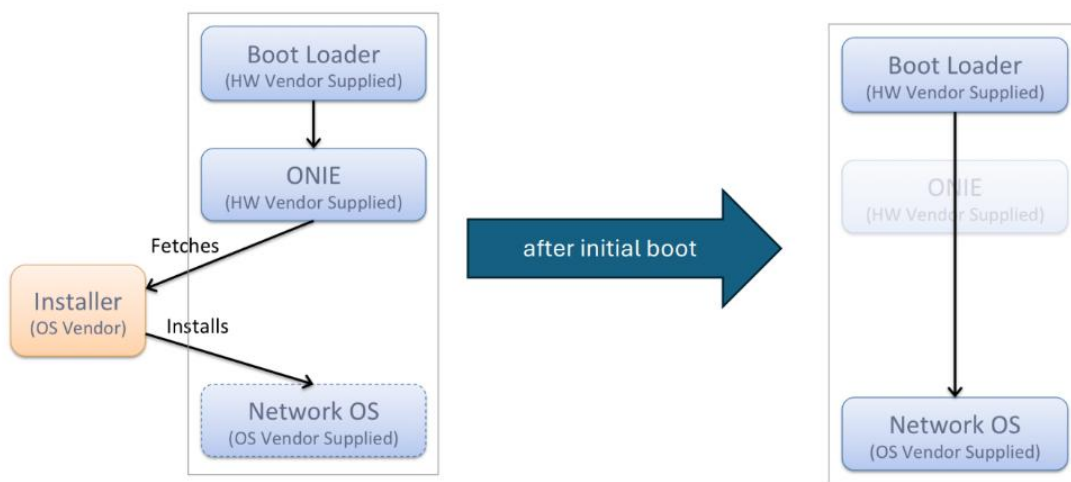
- In case of open NOS, installation can be performed through Open Network Install Environment (ONIE)

Invece di avere solo Juniper e Cisco, abbiamo altri giocatori differenti con questa nuova struttura.

ONIE permette di installare il sistema operativo sopra i dispositivi di rete.

Open Network Install Environment (ONIE)

- Defines an open “install environment” for modern networking hard
- ONIE utilize facilities in a Linux kernel and BusyBox environment



ONIE usa un piccolo sistema operativo pre-installato nell'hardware, ha un kernel Linux e la BusyBox che permette di connettere il dispositivo ad internet, in modo tale che possa prendere il sistema operativo e installarlo con ONIE.

Il Boot Loader è dato dal vendor, si ha ONIE che è un piccolo sistema operativo che permette di loggarsi, connesso ad internet, permette di fetchare l'installer dal vendor.

Dopo il boot iniziale c'è il boot loader che direttamente carica il sistema operativo di nostra scelta.

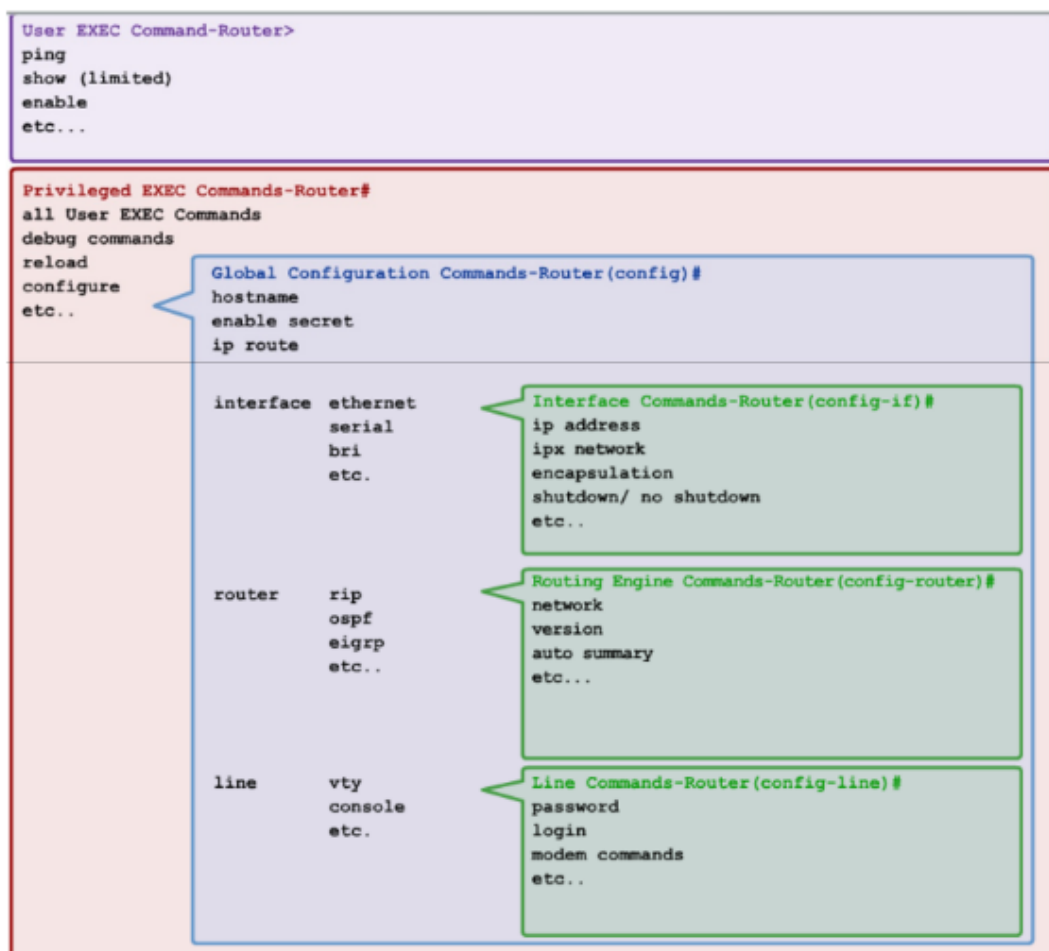
NOS Example: Cisco IOS

- Operating system used in Cisco devices
- Used in both routers and switches
- Several versions available having different functions
- Provides a command line interface (CLI) that can be accessed directly or remotely (e.g., through SSH or Telnet)
- The CLI accepts a set of multiple-word commands

Cercheremo di vedere un esempio di un sistema operativo di rete e poi useremo differenti tipi di sistemi operativi che possono essere eseguiti in ambienti virtuali.

Ogni comando è composto da una serie di keyword.

Cisco IOS modes



Ci sono *modal command line interface*: quando lanci una command line ci sono differenti modalità che si possono usare.

1. Quando lanci sarai in *exec mode*: nella quale puoi fare simple diagnostic (pings, vedere il set di informazioni dei dispositivi ...).
2. Quando lanci il comando *enable* ti muovi alla prossima modalità che è *privileged execution mode*: potrai usare molti comandi di debug, sarai capace di gestire le configurazioni generali, sarai capace di usare tutti i tipi di comandi *show*.
3. Poi puoi lanciare il *configure command* per muoverti alla *global configuration mode*: accederai a tutte le funzioni di configurazione.

In generale, il set di comandi dipenderà dalla modalità in cui sei. C'è un comando utile che è il "?": quando sei in una di queste modalità puoi scriverlo e il sistema ti mostra tutti i comandi possibili che puoi eseguire in quella modalità.

IOS prompt structure

```
Router>ping 192.168.10.5  
  
Router#show running-config  
  
Router(config)#Interface FastEthernet 0/0  
  
Router(config-if)#ip address 192.168.10.1 255.255.255.0
```

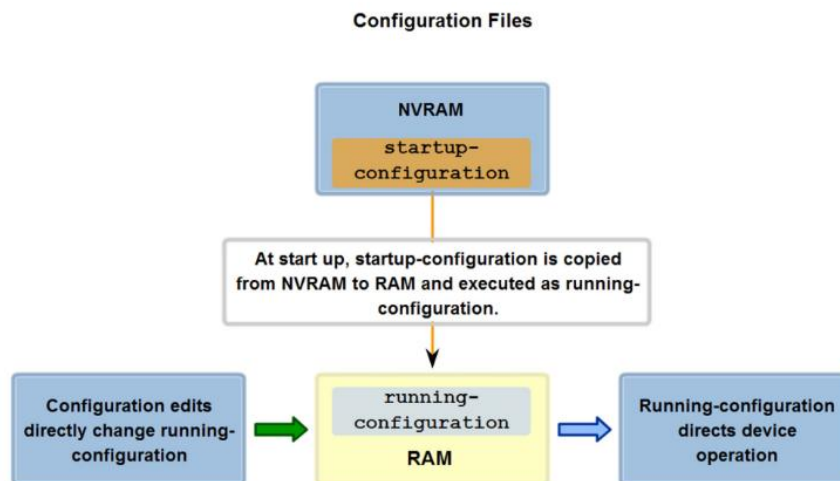
The prompt changes to denote the current CLI mode.

```
Switch>ping 192.168.10.9  
  
Switch#show running-config  
  
Switch(config)#Interface FastEthernet 0/1  
  
Switch(config-if)#Description connection to WEST LAN4
```


Moving between primary modes

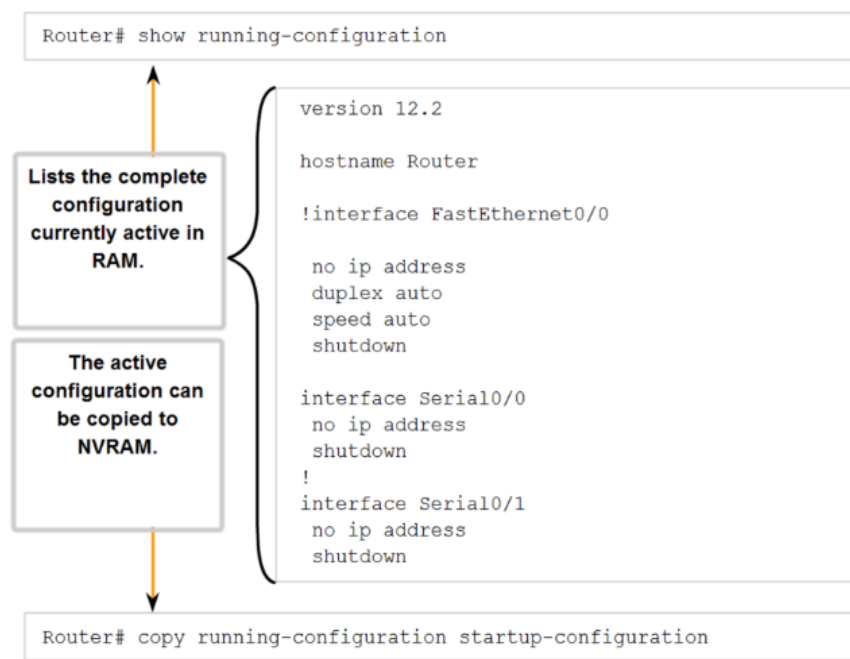
```
Router con0 in now available.  
  
Press RETURN to get started.  
  
User Access Verification  
Password:  
Router>enable  
Password:  
Router#configure terminal  
Enter configuration commands, one per line. End with CNTL/Z.  
Router(config)#^Z  
Router#disable  
Router>exit
```

Configuration files



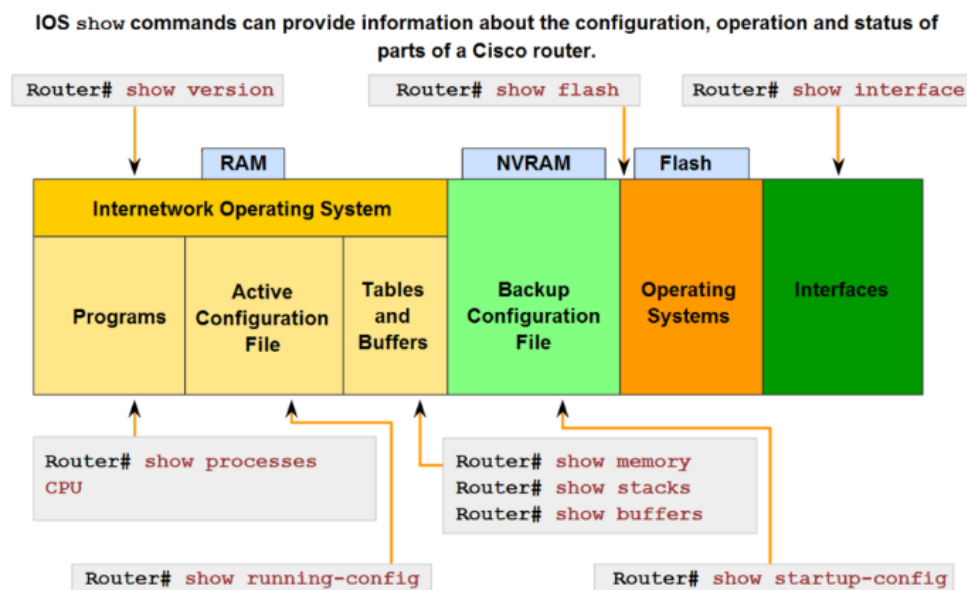
Tutti i comandi che lanci vanno in RAM. Ci sarà un file tipicamente chiamato “running configuration”, ovvero i comandi che sono attivi, che sono stati lanciati (N.B: non è uno storico, non sono in ordine cronologico). startup è nella memoria non volatile, viene preso e caricato in RAM.

Managing configuration files



Contiene una lista di comandi. Il termine “no” cancella l’effetto del comando seguente.

The IOS **show** command



Lo “show” command permette di vedere molte parti del sistema.

Network configurations

Automating networking lab creation

- Need to run virtualized NOSs
- Need to create versatile topologies
- Container-orchestration tools (e.g., docker-compose) are not a good fit

Containerlab installation

- ```
curl -sL https://containerlab.dev/setup |
sudo -E bash -s "all"
```



Viene usato come un tool per l'automazione delle reti, è usato anche per testarle. Creeremo un configuration file in YAML, manderemo un comando in *containerlab*, che crea una rete di dispositivi di rete "containerizzati".

Di seguito troviamo la topologia da creare dopo aver creato una cartella e dentro di essa un file yaml.

### Containerlab: topology

- The topology definition file is a configuration file expressed in YAML and has a name pattern of \*.clab.yml

```
1 name: ceos01
2
3 topology:
4 nodes:
5 ceos1:
6 kind: arista_ceos
7 image: ceos:4.33.0F
8 ceos2:
9 kind: arista_ceos
10 image: ceos:4.33.0F
11
12 links:
13 - endpoints: ["ceos1:eth1", "ceos2:eth1"]
```

Name → nome della configurazione

Topology → stai per descrivere una topologia

Nodes → la topologia è composta da nodi

I nodi sono un set di elementi che hanno quella forma:

- Nome
- Kind (deve essere supportato da containerlab)
- Immagine del container che deve essere disponibile nel sistema

Dobbiamo poi specificare i link che sono composti da endpoints (coppie di interfacce appartenenti a nodi).

Bisogna scaricare l'immagine del container da internet:

## Containerlab: adding a new node kind

- Download arista switch
  - virtual switches can be download

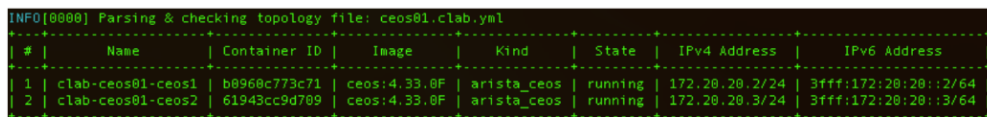
- Add the switch image

```
import container image and save it under ceos:4.32.0F name
docker import cEOS64-lab-4.32.0F.tar.xz ceos:4.32.0F
```

Nella cartella dove hai il file yaml lanci il seguente comando:

## Containerlab: deploy a new network

- `sudo containerlab deploy`



```
INFO[0000] Parsing & checking topology file: ceos01.clab.yml
```

| # | Name              | Container ID | Image        | Kind        | State   | IPv4 Address   | IPv6 Address         |
|---|-------------------|--------------|--------------|-------------|---------|----------------|----------------------|
| 1 | clab-ceos01-ceos1 | b0960c773c71 | ceos:4.33.0F | arista_ceos | running | 172.20.20.2/24 | 3fff:172:20:20::2/64 |
| 2 | clab-ceos01-ceos2 | 61943cc9d709 | ceos:4.33.0F | arista_ceos | running | 172.20.20.3/24 | 3fff:172:20:20::3/64 |

La rete verrà distribuita ovvero i container verranno creati. La figura in verde sopra indica che sono state create due macchine partendo da quelle due immagini di un certo tipo, entrambe sono in esecuzione (running) e hanno i due indirizzi IP in figura che sono quelli delle interfacce "Management" (di *eth0*).

## Containerlab: destroying a network

- `sudo containerlab destroy`

Puoi distruggerla con questo semplice comando.

Puoi ispezionare la rete con i comandi seguenti:

## Containerlab: inspecting the network

- `docker exec -it clab-ceos01-ceos1 bash`
- `ip addr`
- `ip link show type bridge`
- `bridge link show`
- `ssh admin@clab-ceos01-ceos1`

Il comando che segue, che si può vedere anche in figura, viene utilizzato per accedere alla console del container `ceos1` appartenente alla topologia (clab) `ceos01`, questo perché quando i container vengono creati sono avviati in modalità *detached*, quindi dobbiamo accederci direttamente:

```
docker exec -it clab-ceos01-ceos1 bash
```

### Descrizione del comando:

1. **docker exec:** Serve per eseguire un comando all'interno di un container Docker già in esecuzione.
2. **-it:** Specifica due opzioni:
  - **-i:** Mantiene l'input attivo.
  - **-t:** Alloca un terminale pseudo-TTY per interazioni testuali.
3. **clab-ceos01-ceos1:** È il nome o l'ID del container Docker su cui si vuole eseguire il comando.
4. **bash:** È il comando che viene eseguito all'interno del container, aprendo una shell Bash interattiva.

I due container sono basati su linux.

## Esempio pratico

Se lancio il comando *ip addr* da student1:

```
antonio — ubuntu@containerlab: ~/clab-quickstart — ssh ubuntu@172.16.4.202 — 122x27
/Users/antonio — ubuntu@containerlab: ~/clab-quickstart — ssh ubuntu@172.16.4.202
ubuntu@containerlab:~/clab-quickstart$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
 inet 127.0.0.1/8 scope host lo
 valid_lft forever preferred_lft forever
 inet6 ::1/128 scope host noprefixroute
 valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1458 qdisc pfifo_fast state UP group default qlen 1000
 link/ether fa:16:3e:8c:30:1d brd ff:ff:ff:ff:ff:ff
 altname enp0s3
 inet 172.16.4.202/16 metric 100 brd 172.16.255.255 scope global dynamic ens3
 valid_lft 49293sec preferred_lft 49293sec
 inet6 fe80::f816:3eff:fe8c:301d/64 scope link
 valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
 link/ether 02:42:bc:39:0e:ef brd ff:ff:ff:ff:ff:ff
 inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
 valid_lft forever preferred_lft forever
 inet6 fe80::42:bcff:fe39:eef/64 scope link
 valid_lft forever preferred_lft forever
65: br-7b4472b303d1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
 link/ether 02:42:cf:3d:ef:28 brd ff:ff:ff:ff:ff:ff
 inet 172.20.20.1/24 brd 172.20.20.255 scope global br-7b4472b303d1
 valid_lft forever preferred_lft forever
 inet6 3fff:172:20:20::1/64 scope global
 valid_lft forever preferred_lft forever
 inet6 fe80::42:cfff:fe3d:ef28/64 scope link
 valid_lft forever preferred_lft forever
```

Ho una serie di interfacce, in particolare come si vede in fondo alla figura ho un linux bridge che ha l'ip dell'interfaccia dell'host (student1) verso il bridge stesso cioè 172.20.20.1 (che è la stessa rete dove si trovano le interfacce *Management* il che significa che si può accedere alle interfacce Management attraverso il bridge) e due interfacce *veth* (connesse in qualche modo alle interfacce 66 e 68), una marcata con 67 e una marcata con 69 (che sono le interfacce del bridge verso i due container ceos1 e ceos2):

```
67: vethf0e9bba@if66: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
 link/ether 6a:18:dc:d1:b7:d8 brd ff:ff:ff:ff:ff:ff link-netnsid 0
 inet6 fe80::6818:dcff:fed1:b7d8/64 scope link
 valid_lft forever preferred_lft forever
69: veth2d2cd66@if68: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
 link/ether de:3e:ff:0c:fb:84 brd ff:ff:ff:ff:ff:ff link-netnsid 1
 inet6 fe80::dc3e:ffff:fe0c:fb84/64 scope link
 valid_lft forever preferred_lft forever
```

Lanciando questo comando che segue entriamo nella shell di ceos1:

```
ubuntu@containerlab:~$ sudo docker exec -it clab-ceos01-ceos1 bash
bash-5.1# uname
```

Se lanciamo il comando *ip address* in questa shell si ottiene:



```

9: fwd0: <BROADCAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group default q
n 1000
 link/ether 00:1c:73:30:5c:f8 brd ff:ff:ff:ff:ff:ff
 inet6 fe80::21c:73ff:fe30:5cf8/64 scope link
 valid_lft forever preferred_lft forever
10: mirror0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group d
ault qlen 1000
 link/ether 26:91:9e:c8:b2:fa brd ff:ff:ff:ff:ff:ff
11: mirror1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group d
ault qlen 1000
 link/ether 46:f6:01:9e:08:4f brd ff:ff:ff:ff:ff:ff
12: mirror2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group d
ault qlen 1000
 link/ether e2:d9:f8:30:cf:5a brd ff:ff:ff:ff:ff:ff
13: mirror3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group d
ault qlen 1000
 link/ether 3e:94:d3:7e:81:b4 brd ff:ff:ff:ff:ff:ff
66: eth0@if67: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
 link/ether 02:42:ac:14:14:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
 inet 172.20.20.2/24 brd 255.255.255.255 scope global eth0
 valid_lft forever preferred_lft forever
 inet6 3fff:172:20:20::2/64 scope global
 valid_lft forever preferred_lft forever
 inet6 fe80::42:acff:fe14:1402/64 scope link
 valid_lft forever preferred_lft forever
70: eth1@if71: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9194 qdisc noqueue state UP group default
 link/ether aa:c1:ab:6a:63:26 brd ff:ff:ff:ff:ff:ff link-netnsid 1
bash-5.1#

```

Vediamo che ci sono molte interfacce, possiamo vederne un sommario con il comando *brief*:

```

|bash-5.1# ip -brief addr
lo UNKNOWN 127.0.0.1/24 ::1/128
cpu UNKNOWN
fabric UNKNOWN
fabric1 UNKNOWN
fabric2 UNKNOWN
fabric3 UNKNOWN
arpsnoop UNKNOWN
arpinject UNKNOWN
fwd0 UNKNOWN fe80::21c:73ff:fe30:5cf8/64
mirror0 UNKNOWN
mirror1 UNKNOWN
mirror2 UNKNOWN
mirror3 UNKNOWN
eth0@if67 UP 172.20.20.2/24 3fff:172:20:20::2/64 fe80::42:acff:fe14:1402/64
eth1@if71 UP

```

Vediamo che *eth0* è una interfaccia *veth* connessa all'interfaccia 67, che è proprio l'interfaccia vista prima nell'host: quindi *eth0* è la interfaccia di management dalla prospettiva di ceos1 (ed ha l'ip che ci aspettavamo, ovvero quello visto nella tabella dove sono presenti le descrizioni delle due ceos), quindi la management network di ceos1 è connessa ad un bridge nell'host1. Stessa cosa avviene con ceos2, solo con una connessione tra 68 e 69.



```

antonio ~ ubuntu@containerlab: ~ -- ssh ubuntu@172.16.4.202 -- 96x28
-- ubuntu@containerlab: ~ -- ssh ubuntu@172.16.4.202
66: link/ether 3e:94:d3:7e:81:b4 brd ff:ff:ff:ff:ff:ff
eth0@if67: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
link/ether 02:42:ac:14:14:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
inet 172.20.20.2/24 brd 255.255.255.255 scope global eth0
valid_lft forever preferred_lft forever
inet6 3fff:172:20:20::2/64 scope global
valid_lft forever preferred_lft forever
inet6 fe80::42:acff:fe14:1402/64 scope link
valid_lft forever preferred_lft forever
70: eth1@if71: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9194 qdisc noqueue state UP group default
link/ether aa:c1:ab:6a:63:26 brd ff:ff:ff:ff:ff:ff link-netnsid 1
bash-5.1# ip -brief addr
lo UNKNOWN 127.0.0.1/24 ::1/128
cpu UNKNOWN
fabric UNKNOWN
fabric1 UNKNOWN
fabric2 UNKNOWN
fabric3 UNKNOWN
arpnsnoop UNKNOWN
arpinject UNKNOWN fe80::21c:73ff:fe30:5cf8/64
fwd0 UNKNOWN
mirror0 UNKNOWN
mirror1 UNKNOWN
mirror2 UNKNOWN
mirror3 UNKNOWN
eth0@if67 UP 172.20.20.2/24 3fff:172:20:20::2/64 fe80::42:acff:fe14:1402/64
eth1@if71 UP
bash-5.1# ip -bri
antonio ~ ubuntu@containerlab: ~ -- ssh ubuntu@172.16.4.202 -- 82x25
-- ubuntu@containerlab: ~ -- ssh ubuntu@172.16.4.202
ubuntu@containerlab:~$ sudo docker exec -it clab-ceos01-ceos2 bash
bash-5.1# ip -brief addr
lo UNKNOWN 127.0.0.1/24 ::1/128
cpu UNKNOWN
fabric UNKNOWN
fabric1 UNKNOWN
fabric2 UNKNOWN
fabric3 UNKNOWN
fwd0 UNKNOWN fe80::21c:73ff:fe80:be3c/64
arpnsnoop UNKNOWN
arpinject UNKNOWN
mirror0 UNKNOWN
mirror1 UNKNOWN
mirror2 UNKNOWN
mirror3 UNKNOWN
eth0@if69 UP 172.20.20.3/24 3fff:172:20:20::3/64
eth1@if70 UP
bash-5.1#

```

Da questa figura si può notare come ci siano due *eth1* all'interno di entrambe le ceos che sono connesse fra loro (70 e 71), sono interfacce *veth*. Dalle due figura di seguito si può notare meglio:

```

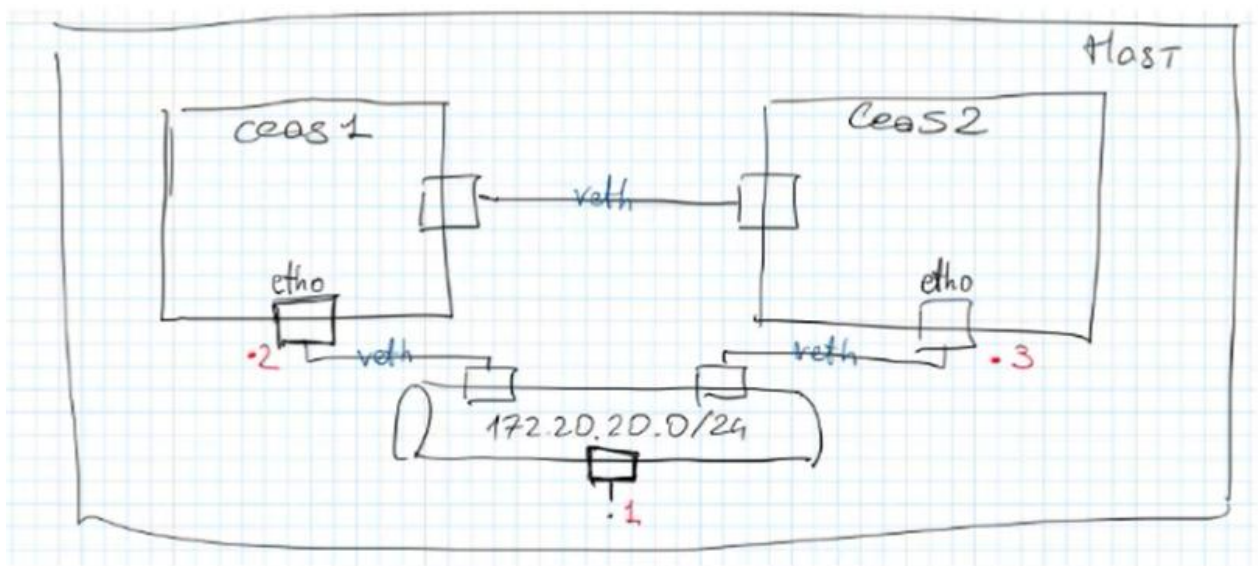
eth1@if71 UP
[bash-5.1# ip addr list eth1
70: eth1@if71: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9194 qdisc noqueue state UP group default
link/ether aa:c1:ab:6a:63:26 brd ff:ff:ff:ff:ff:ff link-netnsid 1
bash-5.1#]

eth1@if70 UP
[bash-5.1# ip addr list eth1
71: eth1@if70: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9194 qdisc noqueue state UP group default
link/ether aa:c1:ab:d5:c0:98 brd ff:ff:ff:ff:ff:ff link-netnsid 1
bash-5.1#]

```

Quello che si può notare è che nessuna delle due *eth1* in entrambi i ceos hanno un indirizzo IP.

La nostra topologia, quindi, sarà qualcosa del genere:



Posso usare il comando `ssh admin@clab-ceos01-ceos1` per accedere all'utente `admin` in `ceos1`, la password è `admin`, qui si possono:

- vedere i comandi con *show commands*
- entrare in modalità di configurazione con *enable*

- vedere la configurazione in esecuzione con *show running-config*, come la seguente:

```
ceos1# show running-config

! Command: show running-config
! device: ceos1 (cEOSLab, EOS-4.33.0F-39050855.4330F (engineering build))
!
no aaa root
!
username admin privilege 15 role network-admin secret sha512
$6$4KHBBJsr4JNR.LUz$Sx94Hp5rxg8Okhj1u/jBqOPrEf-
feGAzqjT8sYayM9tud8hDh1Uuo/zFD29/C47WNMy98GdRGwypGmJB/huJC81
!
management api http-commands
 no shutdown
!
no service interface inactive port-id allocation disabled
!
transceiver qsfp default-mode 4x10G
!
service routing protocols model multi-agent
!
hostname ceos1
!
spanning-tree mode mstp
!
system 11
 unsupported speed action error
 unsupported error-correction action error
!
management api gnmi
 transport grpc default
!
management api netconf
 transport ssh default
!
interface Ethernet1
!
interface Management0
 ip address 172.20.20.2/24
 ipv6 address 3fff:172:20:20::2/64
!
no ip routing
!
ip route 0.0.0.0/0 172.20.20.1
!
ipv6 route ::/0 3fff:172:20:20::1
!
router multicast
 ipv4
 software-forwarding kernel
 !
 ipv6
 software-forwarding kernel
!
```

Abbiamo quindi visto un sistema simile a quello di CISCO (questa è la versione Arista).

Abbiamo una management network che connette i dispositivi con il nodo host (attraverso il bridge e le interfacce *eth0*).

In una rete reale, abbiamo dispositivi veri (sullo stesso rack o distribuiti geograficamente), saranno connessi in qualche modo: ognuno dei quali deve essere **logicamente** connesso ad un management node attraverso interfacce di management. Questo *management node* deve essere capace di fare *ssh* e configurare i nodi.

I due host (quelli che si vedono nel disegno sotto) si comportano come se fossero nella stessa LAN grazie al lavoro della management network.

*Out of band management*: quando ci sono interfacce dedicate per il management, ovvero usa la rete di management per controllare e configurare i dispositivi in modo scollegato al data path. Nella *in band* invece si usa la stessa rete per entrambi.

I dispositivi potrebbero voler comunicare attraverso quella rete, a meno che non specifichiamo diversamente, quindi bisogna escludere quella di management (se passo attraverso il bridge, quindi attraverso le interfacce *eth0* di management degli host, il ping ovviamente funziona).

Containerlab ha già assegnato gli indirizzi ip alle interfacce di management degli switch, ora vogliamo farli comunicare.

Se lancio in *ceos1 show ip interfaces* vedo che solo l'interfaccia di management ha un indirizzo ip assegnato mentre *eth1* non ha nessuna informazione allegata (vedi immagini precedenti). Dobbiamo quindi assegnare un indirizzo ip alle due interfacce *eth1* per permettere gli switch di comunicare.

## Configuring interfaces

```
enable

configure

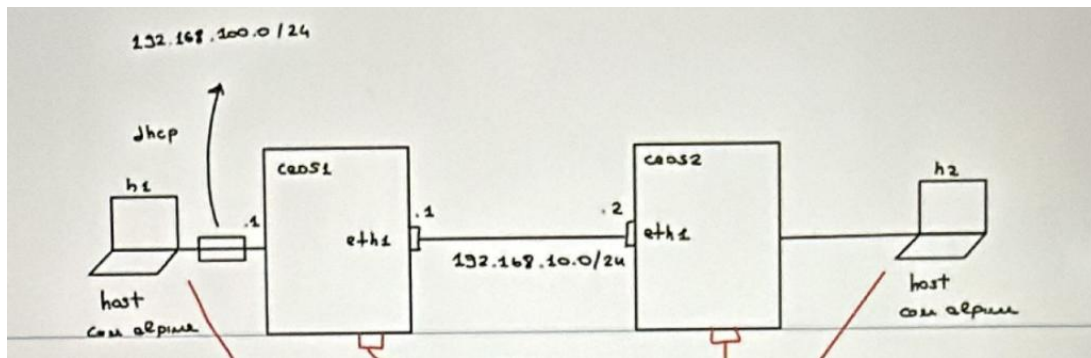
interface ethernet1
 ip address 192.168.10.1 255.255.255.0
 no switchport
 exit

show ip interface brief
```

*Switchport*: è una porta fisica che permette di scambiare i pacchetti a livello due, non ha ip come visto prima, per cui va cancellata questa opzione e resa di livello 3 con ip ("no switchport" va fatto prima di "ip address...").

Dalla figura sopra si possono vedere i passaggi che ora verranno spiegati. Siamo in *ceos1*, si lancia il comando *configure*, si entra quindi nella modalità di configurazione. Ora vogliamo configurare l'ip

dell'interfaccia *eth1*, il comando è *ip address indirizzo\_ip*. La rete è 192.168.10.0 come vediamo in figura, assegniamo quindi a ceos1 il .1 e a ceos2 il .2. Se ora lanciamo in ceos1 il comando *ip address 192.168.10.1* abbiamo un errore: "ogni configurazione viene ignorata finché *eth1* non è una router port". Ricordiamo che questi dispositivi sono configurati come switch, ognuno ha una switch port quindi non si può assegnare un indirizzo ip ad essa. Per cui, dobbiamo dire a quell'interfaccia "no switchport", fatto questo viene aggiunto alla configurazione ed essa non sarà più una interfaccia switch ma sarà una **interfaccia router**.



Ricordiamoci che il comando *show active* è utile per vedere la porzione della running config che è rilevante per quel punto della configurazione.

Adesso se provo a fare il ping da ceos1 a ceos2 funziona (sia se lo faccio dal bash che se lo faccio dalla modalità admin di configurazione).

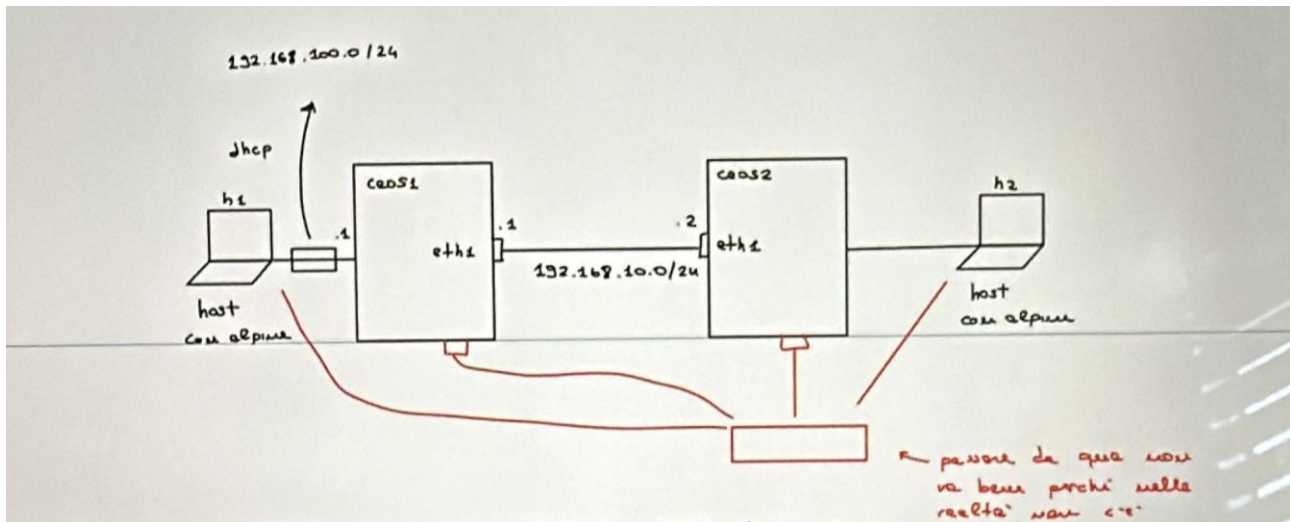
In base allo startup config, creare un template Jinja che può creare molteplici running config per device differenti, bisogna indicare i dispositivi e il set di interfacce, ognuna delle quali avrà un indirizzo ipv4 e ipv6. Sostanzialmente esegue i comandi presenti sopra.

## Exercise1: template it

- based on the startup-config, create a jinja template for multiple devices, specifying:
  - the device/host name
  - a set of interfaces each one having an IPv4 and IPv6

Se utilizzo il comando *write* rendo la configurazione permanente.

Lo scopo di questo esercizio è evitare di ripetere gli stessi comandi ripetitivi nei file di configurazione dei router, questi file verranno creati in gran numero attraverso un template e ogni file viene inviato al corrispettivo router attraverso la "macchina di management" (che è quella in basso nella figura sotto):



Il file startup config seguente:

*! Startup-config last modified at Fri Nov 22 08:25:05 2024 by admin*

*! device: ceos1 (cEOSLab, EOS-4.33.0F-39050855.4330F (engineering build))*

*!*

*no aaa root*

*!*

*username admin privilege 15 role network-admin secret sha512  
\$6\$opljA1CJ8Vuhhy2G\$Er03mhZgPwQchrbEN08rUaWBpBmLPXcZXeWJbMo.tWPa8CsQ4x6yG8OR7  
tAuhkILLbKWA6VNuT92u36ecJ6zu1*

*!*

*management api http-commands*

*no shutdown*

*!*

*no service interface inactive port-id allocation disabled*

*!*

*transceiver qsfp default-mode 4x10G*

*!*

*service routing protocols model multi-agent*

*!*

*hostname ceos1*

*!*

```
spanning-tree mode mstp
!
system l1
 unsupported speed action error
 unsupported error-correction action error
!
management api gnmi
 transport grpc default
!
management api netconf
 transport ssh default
!
interface Ethernet1
 no switchport
 ip address 192.168.10.1/24
!
interface Management0
 ip address 172.20.20.2/24
 ipv6 address 3fff:172:20:20::2/64
!
no ip routing
!
ip route 0.0.0.0/0 172.20.20.1
!
ipv6 route ::/0 3fff:172:20:20::1
!
router multicast
 ipv4
```

```
software-forwarding kernel
!
ipv6
software-forwarding kernel
!
End
```

Viene parametrizzato come segue:

```
! Command: show running-config
! device: ceos1 (cEOSLab, EOS-4.33.0F-39050855.4330F (engineering build))
!
no aaa root
!
username admin privilege 15 role network-admin secret sha512
$6$4KHBBJsR4JNR.LUz$Sx94Hp5rxg8Okhj1u/jBqOPrEffeGAzqjT8sYayM9tud8hDh1Uuo/zFD29/C47
WNMy98GdRGwypGmJB/huJC81
!
management api http-commands
no shutdown
!
no service interface inactive port-id allocation disabled
!
transceiver qsfp default-mode 4x10G
!
service routing protocols model multi-agent
!
hostname ceos1
!
```



```

spanning-tree mode mstp
!
system l1
 unsupported speed action error
 unsupported error-correction action error
!
management api gnmi
 transport grpc default
!
management api netconf
 transport ssh default
!
{%%- for interface in interfaces %}
interface {{ interface['name'] }}
 {%%- if interface['ipv4_address'] and interface['ipv4_mask'] %}
 ip address {{ interface['ipv4_address'] }}{{ interface['ipv4_mask'] }}
 {%%- endif -%}
 {%%- if interface['ipv6_address'] and interface['ipv6_mask'] %}
 ipv6 address {{ interface['ipv6_address'] }}{{ interface['ipv6_mask'] }}
 {%% endif -%}
{%%- endfor %}
!
no ip routing
!
ip route 0.0.0.0/0 172.20.20.1
!
ipv6 route ::/0 3fff:172:20:20::1
!

```

*router multicast*

*ipv4*

*software-forwarding kernel*

!

*ipv6*

*software-forwarding kernel*

!

Dobbiamo fare un template che vada a creare dei file di configurazione per più dispositivi, per ognuno vanno specificato le interfacce e gli indirizzi ip. Dal livello più esterno si ha una lista di hostname (dove la chiave è l'hostname e il valore è l'host).

Ciascun hostname ha una lista di interfacce, ogni interfaccia è una mappa che accoppia chiave-valore e chiave-valore. Il primo valore (della prima chiave) è una stringa (nome), il secondo valore (della seconda chiave) è una lista di interfacce che è di nuovo una mappa che associa nome-indirizzo (ipv4) e nome-indirizzo (ipv6).

Esternamente in jinja si fa un *for* che lavora sui device, li considera uno alla volta.

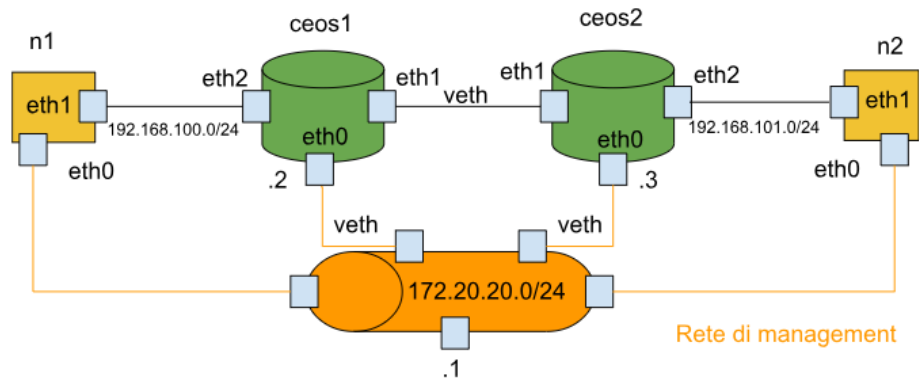
Nella realtà si ha un insieme di dispositivi con caratteristiche comuni, scriviamo un file template e definiamo un modello di dati, poi si utilizzerà un sistema di automazione al quale si dice: hai un elenco di dispositivi, per ognuno di questi, creami un template jinja, poi fai ssh e copia il file config risultante dentro a quel dispositivo e poi accendilo (questa procedura è detta *device provisioning*, ovvero i dispositivi vengono configurati).

Possiamo poi inserire degli host in maniera virtualizzata (un container) che si attaccano ad una nuova interfaccia del router e dobbiamo configurare il tutto. Si utilizzeranno i container "alpine".

## Containerlab: network with hosts

```
1 # adding end nodes to ceos01 lab
2 name: ceos02
3
4 topology:
5 nodes:
6 n1:
7 kind: linux
8 image: alpine:latest
9 n2:
10 kind: linux
11 image: alpine:latest
12 ceos1:
13 kind: arista_ceos
14 image: ceos:4.33.0F
15 ceos2:
16 kind: arista_ceos
17 image: ceos:4.33.0F
18
19 links:
20 - endpoints: ["ceos1:eth1", "ceos2:eth1"]
21 - endpoints: ["n1:eth1", "ceos1:eth2"]
22 - endpoints: ["n2:eth1", "ceos2:eth2"]
```

| # | Name              | Container ID | Image         | Kind        | State   | IPv4 Address   | IPv6 Address         |
|---|-------------------|--------------|---------------|-------------|---------|----------------|----------------------|
| 1 | clab-ceos02-ceos1 | 5a950535dd00 | ceos:4.33.0F  | arista_ceos | running | 172.20.20.5/24 | 3fff:172:20:20::5/64 |
| 2 | clab-ceos02-ceos2 | e694a1eaa081 | ceos:4.33.0F  | arista_ceos | running | 172.20.20.4/24 | 3fff:172:20:20::4/64 |
| 3 | clab-ceos02-n1    | d6c65e772a40 | alpine:latest | linux       | running | 172.20.20.3/24 | 3fff:172:20:20::3/64 |
| 4 | clab-ceos02-n2    | 3df532260720 | alpine:latest | linux       | running | 172.20.20.2/24 | 3fff:172:20:20::2/64 |



| Device       | eth0           | eth1              | eth2             |
|--------------|----------------|-------------------|------------------|
| ceos1        | 172.20.20.5/24 | 192.168.10.1/24   | 192.168.100.1/24 |
| <u>ceos2</u> | 172.20.20.4/24 | 192.168.10.2/24   | 192.168.101.1/24 |
| n1           | 172.20.20.2/24 | 192.168.100.10/24 | /                |
| n2           | 172.20.20.3/24 | 192.168.101.10/24 | /                |

I container sono di tipo “linux” (il tipo è un modo per dire come containerlab si deve interfacciare con esso). Containerlab quindi configura questi dispositivi, anche per questi crea una interfaccia di management. Se ci si connette a questi dispositivi si nota che essi non possono comunicare poiché bisogna assegnargli un ip e configurare la parte di routing, ma invece che andarci a mano si può usare il dhcp. Si può quindi configurare un server dhcp nei due router che lavori sulla parte della rete (nella sottorete) dove sono presenti queglii host.

## Configure DHCP

```
interface Ethernet2
 no switchport
 dhcp server ipv4
 ip address 192.168.100.1/24
 exit
dhcp server
 subnet 192.168.100.0/24
 range 192.168.100.10 192.168.100.99
```

- on client
  - `udhcp -i eth1`

Ricordiamo che l’interfaccia del server dhcp necessita di un ip da inserire manualmente. Nel comando in figura abbiamo indicato di usare il server dhcp sull’interfaccia *eth2*.

Ci si può quindi connettere a ceos1 e facendo “enable” si possono configurare le interfacce e il dhcp. Questo va fatto anche per ceos2, in modo tale da assegnare l’ip anche all’host 2.

Con *show ip interface* vediamo le interfacce configurate.

## arista: act as a router (static routes)

```
ip routing
ip route 192.168.101.0/24 192.168.10.1
exit
```

```
show ip route
show ip route summary
```

Bisogna configurare l'ip routing nel router e poi settare una route statica.

## arista: act as a router (dynamic routes)

```
no ip route 192.168.101.0/24 192.168.10.2
```

```
router ospf 1
router-id 10.1.1.1
network 192.168.100.0/24 area 0
```

```
show ip route
show ip ospf neighbor
```

Con “no” revochiamo un comando mandato in precedenza, togliamo la riga dal configuration file.

Con la keyword “router” entriamo in modalità di configurazione dei protocolli, noi vogliamo configurare OSPF (1 è il numero dell'istanza). Specifichiamo il router id e la rete sulla quale deve lavorare questo protocollo.

Con *show ip route* possiamo vedere la routing table con anche i protocolli che hanno generato quelle entrate, quindi non solo i prefissi ma come essi sono stati “imparati”.

## Exercise2: template it

- based on the previously-created template, configure also:
  - dhcp server from both system and interface perspective
  - optional static rules

## Exercise3: template it

- based on the previously-created template, configure also:
  - general OSPF configuration
  - a list of networks

## Arista: device API -> eAPI

- Allows accessing device commands remotely
- Based on JSON-RPC
- Can be configured via CLI

```
enable
configure
management api http-commands
 [no] shutdown
 [no] protocol https [port]
 [no] protocol http [port]

show management api http-commands
```

Un modo per configurare i dispositivi da remoto è attraverso l'API esposta dai dispositivi stessi.

L'interfaccia di Arista è eAPI, permette di fare chiamate alle procedure da remoto mandate attraverso una connessione http. Queste chiamate possono essere fatte attraverso un browser o attraverso python.

L'output non sarà verboso ma sarà in formato json.

## Arista: eAPI hello world (python)

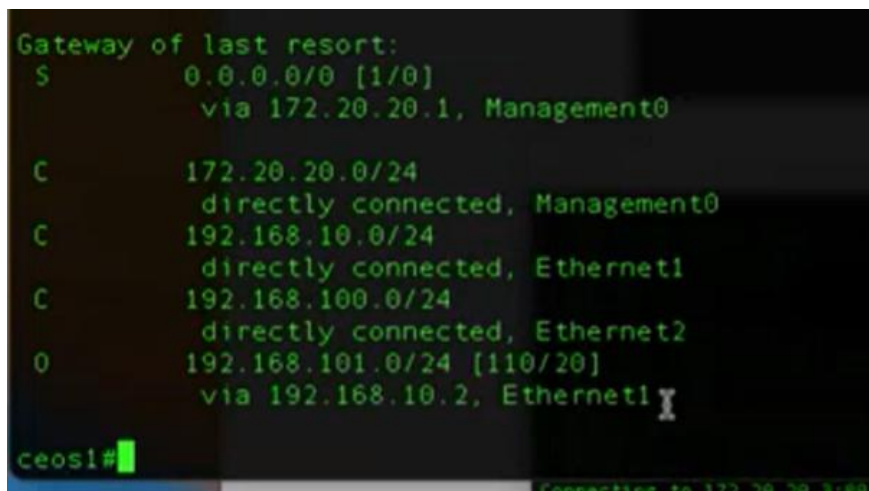
```
1 from jsonrpclib import Server
2
3 switch = Server("http://admin:admin@172.20.20.3/command-api")
4
5 response = switch.runCmds(1, ["show hostname"])
6 print ("Hello, my name is: ", response[0]["hostname"])
7
8 response = switch.runCmds(1, ["show version"])
9 print ("My MAC address is: ", response[0]["systemMacAddress"])
10 print ("My version is: ", response[0]["version"])
11
```

Si accede alla risorsa *command-api*. La risposta è json e viene salvata in una variabile che può essere “navigata”, sia accede al valore dell’hostname.

## Arista eAPI: print a full JSON output

```
1 from jsonrpclib import Server
2 import pprint
3
4 switch = Server("http://admin:admin@172.20.20.3/command-api")
5
6 response = switch.runCmds(1, ["show hostname"])
7 pprint.pprint(response)
```

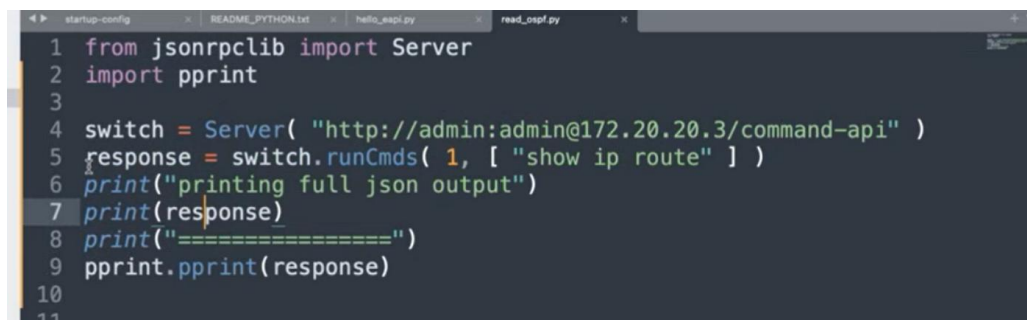
Sostanzialmente vogliamo ottenere queste informazioni attraverso la eAPI:



```
Gateway of last resort:
S 0.0.0.0/0 [1/0]
 via 172.20.20.1, Management0

C 172.20.20.0/24
 directly connected, Management0
C 192.168.10.0/24
 directly connected, Ethernet1
C 192.168.100.0/24
 directly connected, Ethernet2
O 192.168.101.0/24 [110/20]
 via 192.168.10.2, Ethernet1
ceos1#
```

Con questo codice è possibile farlo:



```
1 from jsonrpclib import Server
2 import pprint
3
4 switch = Server("http://admin:admin@172.20.20.3/command-api")
5 response = switch.runCmds(1, ["show ip route"])
6 print("printing full json output")
7 print(response)
8 print("=====")
9 pprint.pprint(response)
10
11
```

E si ottiene:

```

rue, 'routeType': 'connected', 'routeLeaked': False, 'kernelProgrammed': True, 'routeAction': 'forward', 'vias':
92.168.100.0/24': {'hardwareProgrammed': True, 'routeType': 'connected', 'routeLeaked': False, 'kernelProgrammed':
hernet2'}], 'directlyConnected': True}, '192.168.101.0/24': {'hardwareProgrammed': True, 'routeType': 'OSPF', 'rou
10, 'metric': 20, 'routeAction': 'forward', 'directlyConnected': False, 'vias': [{'nextHopAddr': '192.168.10.2',
=====
[{'vrfs': {'default': {'allRoutesProgrammedHardware': True,
 'allRoutesProgrammedKernel': True,
 'defaultRouteState': 'reachable',
 'routes': {'0.0.0.0/0': {'directlyConnected': False,
 'hardwareProgrammed': True,
 'kernelProgrammed': True,
 'metric': 0,
 'preference': 1,
 'routeAction': 'forward',
 'routeLeaked': False,
 'routeType': 'static',
 'vias': [{'interface': 'Management0',
 'nextHopAddr': '172.20.20.1'}]}},
 '172.20.20.0/24': {'directlyConnected': True,
 'hardwareProgrammed': True,
 'kernelProgrammed': True,
 'routeAction': 'forward',
 'routeLeaked': False,
 'routeType': 'connected',
 'vias': [{'interface': 'Management0'}]}},
 '192.168.10.0/24': {'directlyConnected': True,
 'hardwareProgrammed': True,
 'kernelProgrammed': True,
 'routeAction': 'forward',
 'routeLeaked': False,
 'routeType': 'connected',

```

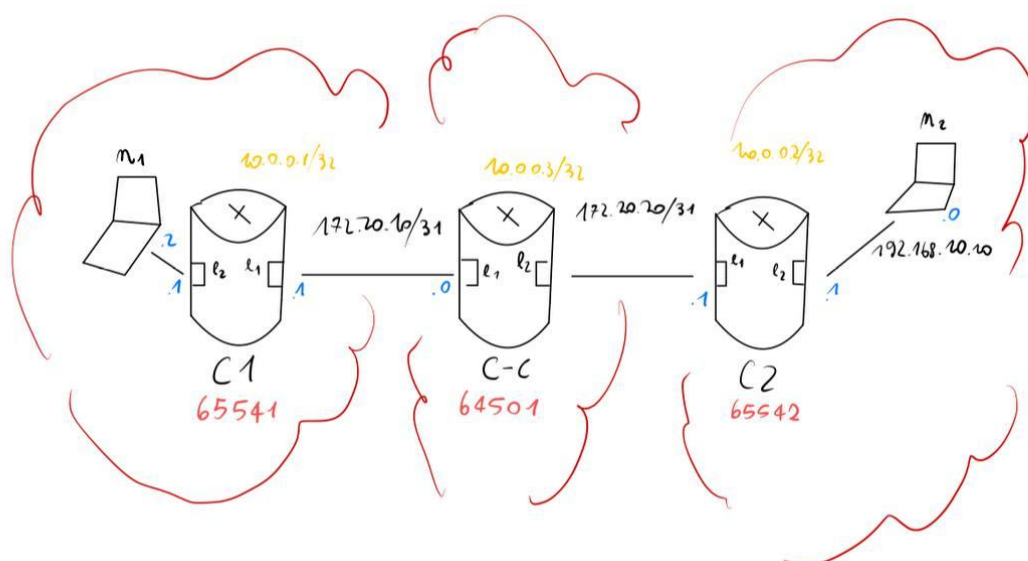
Stessa informazione di prima ma in formato json.

## Exercise4: gather and parse routing info

- retrieve the info regarding routing on ceos1
- print all discovered non directly-connected networks, together with next hop

## Peering BGP con Arista

Innanzitutto, questa sarà la nuova topologia che verrà utilizzata:





Questo è il rispettivo file yaml:

```
name: ceos02

topology:
 nodes:
 n1:
 kind: linux
 image: alpine:latest
 n2:
 kind: linux
 image: alpine:latest
 c1:
 kind: arista_ceos
 image: ceos:4.33.0F
 c-c:
 kind: arista_ceos
 image: ceos:4.33.0F
 c2:
 kind: arista_ceos
 image: ceos:4.33.0F

 links:
 - endpoints: ["c1:eth1", "c-c:eth1"]
 - endpoints: ["c-c:eth2", "c2:eth1"]
 - endpoints: ["n1:eth1", "c1:eth2"]
 - endpoints: ["n2:eth1", "c2:eth2"]
```

Vogliamo stabilire un peering BGP, per farlo bisogna:

- settare l'interfaccia di loopback
- attivare l'ip routing
- configurare il BGP

in c1, c-c e c2. Per fare queste operazioni eseguire i seguenti comandi in ogni cammina "c":

```
interface Loopback0
 ip address 1.1.1.1/24

ip routing

router bgp 1
 neighbor 192.168.10.2 remote-as 2
 network 1.1.1.0 mask 255.255.255.0
! the last command announces the network 1.1.1.0

show ip bgp summary
show ip bgp
```

BGP non solo decide come le informazioni sono condivise attraverso dispositivi o come è calcolato il percorso migliore, ma è anche un set di tool utile per implementare *business policies*, ovvero negoziati tra AS.

## Arista: BGP

- define the AS number of the router
- define the BGP identifier
- define IP and AS number of neighbors
- choose networks to announce
- perform announce filtering and manipulation

Vediamo la configurazione di base in Arista. I comandi disponibili in arista sono uguali quasi a quelli di Cisco (IOS XE). Le informazioni in figura servono per configurare Arista.

L'AS number è usato per vedere se il vicino è parte del proprio AS, per capire se è una connessione iBGP o se è eBGP. Alla fine, si ottiene la mappa BGP mesh, dove sono presenti tutte le connessioni (saranno in stato *established*).

Poi bisogna scegliere la rete da annunciare, le reti si prendono, ad esempio, dall'OSPF o si specificano direttamente. Il secondo step è il filtraggio degli annunci e manipolazione degli stessi. Siamo capaci di implementare tutto quello che abbiamo visto in teoria: come, ad esempio, il fatto che alcuni prefissi imparati da qualcuno li teniamo e altri no.

*eth0* è sempre per la management interface.

## Arista: simple BGP peering

```
interface Loopback0
 ip address 10.0.0.1/32

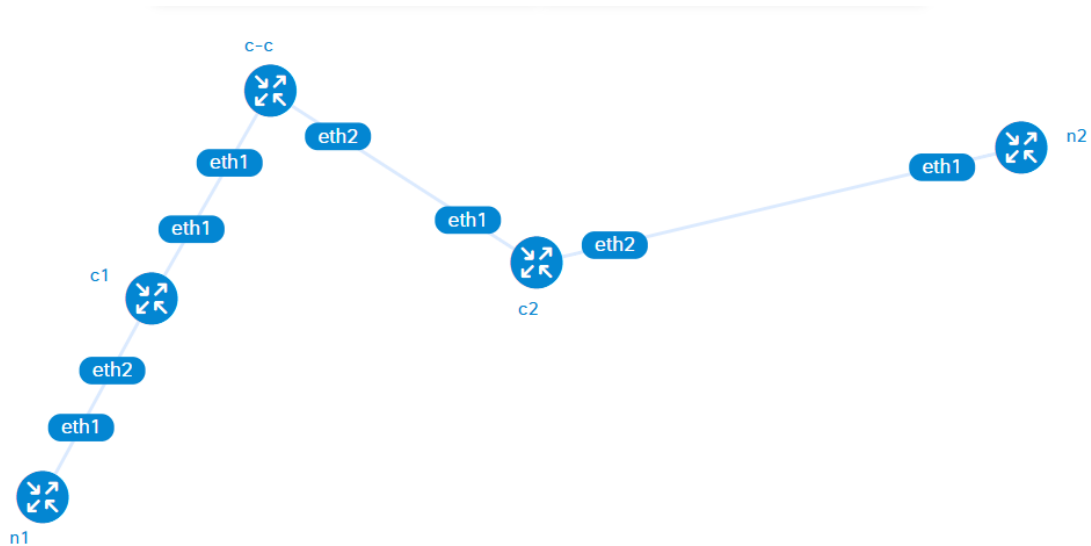
ip routing

router bgp 65541
 router-id 10.0.0.1
 neighbor 172.20.1.0 remote-as 64501
```

Dopo la configurazione si ha un BGP agent attivo. Prima di fare il peering si specifica il nostro primo vicino (si fa sempre in configuration mode del router BGP) utilizzando i comandi in figura, in cui si indica l'ip e l'AS.

Con `show ip interface brief` vediamo se le interfacce che abbiamo creato hanno l'indirizzo ip corretto.

Tipicamente quando implementiamo la configurazione BGP, sarà molto semplice sapere se la topologia è simile a quella che volevamo. Containerlab espone un tool per stampare la topologia, basta usare il comando “`containerlab graph`”, il quale crea un png e un http server esponente in una porta configurabile (5080), che non è altro che una pagina web che mostra la topologia.



<http://192.168.56.101:50080/>

```
student@student2:~/lab10$ sudo containerlab graph
[sudo] password for student:
INFO[0000] Parsing & checking topology file: lab11.clab.yaml
INFO[0000] Serving topology graph on http://0.0.0.0:50080
```

Come si annuncia una route in un AS?

## Arista BGP: announcing networks

```
router bgp 65541
 network 192.168.100.0 mask 255.255.255.0
```

Dopo averlo annunciato, è possibile vedere nei vari router se è stato annunciato correttamente.

Il router **di default** accetta tutti gli update in input e output.

## Arista BGP: show commands

```
show ip bgp summary
show ip bgp
show ip bgp neighbors
show bgp ipv4 unicast
show bgp ipv4 unicast ip_prefix
```

Ecco cosa accade generalmente:

1. **Origine pacchetto (C1):** Quando C1 invia un pacchetto utilizzando la sua interfaccia di loopback, l'indirizzo sorgente del pacchetto sarà l'IP della loopback (ad esempio, 10.0.0.1/32).
2. **Router C-C riceve il pacchetto:** Se C1 invia questo pacchetto verso una destinazione inoltrata al router C-C (come potrebbe essere un indirizzo dell'interfaccia di loopback remota), C-C consulta la sua tabella di routing.
3. **Inoltro verso C1 (loopback):** Se il router C-C deve ritornare il pacchetto a C1, lo inoltra all'indirizzo corrispondente nella sua tabella di routing. Se la route dice che l'IP di destinazione è raggiungibile attraverso C1 sull'interfaccia **e1**, allora il pacchetto verrà inviato lì.

## Arista BGP: safe policy

```
bgp missing-policy direction in action deny-in-out
bgp missing-policy direction out action deny-in-out
```

C'è un RFC che suggerisce che ogni router di default non debba accettare né propagare update a meno che non specificamente indicato. Come se una persona non ricevesse consigli a meno che non viene specificamente detto.

Questi comandi configurano il comportamento del protocollo BGP (Border Gateway Protocol) su dispositivi **Arista** quando non è presente una policy esplicita applicata per l'instradamento (routing) di pacchetti. Analizziamoli:

**Sintassi:**

1. **bgp missing-policy direction in action deny-in-out**
2. **bgp missing-policy direction out action deny-in-out**

### Dettaglio dei parametri:

- **missing-policy:** Questo parametro specifica cosa deve fare il BGP quando una politica di routing (policy) è mancante.
  - **direction:** Indica la direzione a cui la policy si applica:
    - **in:** Per i pacchetti in ingresso verso il router BGP.
    - **out:** Per i pacchetti in uscita dal router BGP.
  - **action:** Definisce l'azione da intraprendere in assenza di una policy.
  - **deny-in-out:** Blocca (deny) i pacchetti sia in ingresso sia in uscita.
- 

### Effetto dei comandi:

#### 1. Comando 1 (direction in)

Se un peer BGP invia aggiornamenti o route verso il dispositivo e non c'è una policy configurata per gestire tali aggiornamenti, il dispositivo li **bloccherà**.

#### 2. Comando 2 (direction out)

Se il dispositivo BGP deve inviare aggiornamenti o route a un peer e non c'è una policy configurata, tali aggiornamenti saranno **bloccati**.

---

### Scopo:

- **Sicurezza:** Impedire che aggiornamenti BGP imprevisti o indesiderati vengano accettati (in ingresso) o inviati (in uscita), riducendo il rischio di errori o vulnerabilità nel routing.
- **Prevenzione di comportamenti non intenzionali:** Se non è configurata una policy, il comportamento predefinito è di negare tutto, invece di permettere tutto, seguendo il principio del "**deny by default**".

## Arista BGP: route-map

```
route-map YES_C1-in permit 10
route-map YES_C1-out permit 10
```

```
router bgp 64501
 neighbor 172.20.1.1 route-map YES_C1-in in
 neighbor 172.20.1.1 route-map YES_C1-out out
```

## Dettagli della configurazione

### 1. Route-map definiti:

- route-map YES\_C1-in permit 10: Questo route-map è applicato alle route in entrata, accettandole (permit) con la sequenza 10.
- route-map YES\_C1-out permit 10: Questo route-map è applicato alle route in uscita, accettandole (permit) con la sequenza 10.

Questi route-map possono essere usati per filtrare, modificare attributi (come il prefisso o il next-hop), o applicare policy specifiche. Tuttavia, dal contenuto mostrato, non ci sono condizioni o azioni specifiche associate.

### 2. Configurazione BGP:

- Il router è configurato con l'AS (Autonomous System) locale 64501.
- Viene definito un vicino (neighbor) con l'indirizzo IP 172.20.1.1.
  - **route-map YES\_C1-in in:** Questo route-map si applica alle route ricevute da 172.20.1.1.k
  - **route-map YES\_C1-out out:** Questo route-map si applica alle route inviate al vicino.

### Funzionamento:

- La direttiva permit nei route-map significa che tutte le route che li attraversano saranno accettate (a meno che non siano specificate condizioni aggiuntive non mostrate nell'immagine).
- **In entrata (in):** Le route provenienti dal vicino 172.20.1.1 vengono processate dal route-map YES\_C1-in.
- **In uscita (out):** Le route che il router invia a 172.20.1.1 vengono processate dal route-map YES\_C1-out.

### Uso tipico:

- Questa configurazione serve per applicare policy di routing personalizzate. Per esempio:
  - Filtrare specifici prefissi. (per esempio, per chi non ha pagato l'inoltro)
  - Modificare gli attributi BGP (come il local preference, AS path, o MED).
  - Applicare modifiche alle route accettate o propagate.

Se non ci sono ulteriori condizioni definite nei route-map, questa configurazione accetta tutte le route senza modifiche.

## Arista BGP: route-map

```
route-map NAME permit|deny
 match [condizione]
 set valore
```

Questa è la struttura di route-map. Specifica la regole applicate ad un update per vedere se sono matchate. Puoi settare una local preference. Questo decide il comportamento di bgp.

Sono un AS, ho una relazione commerciale con un altro AS, esso mi paga per essere un transito per lui. Il primo AS, dice all'altro i suoi prefissi e quelli che conosce.

Specifichiamo matching rules, espresse con if else if ...

Per un vicino applicare la route map chiamata YES.. per "in", ovvero gli annunci che arrivano, idem per quelli verso l'esterno. Si possono filtrare quelli interni, poi in base alla routing table puoi decidere se inoltrarle o meno.

La route map dice che permettiamo tutto il traffico che matcha la route map, che non ha regole, quindi lo accetta tutto. Perché abbiamo un numero alla fine? Route map possono essere implementate in molteplici statement, il primo è già un if, quindi route-map, se aggiungiamo un else dobbiamo avere un differente contatore di linea.

Si usano modifier per modificare il contenuto dell'update come il med value, puoi anche modificare l'AS path.

Se l'indirizzo che vedi nell'update matcha la lista di prefissi allora applica quella regola.

## Arista BGP: prefix list

- strumento per il filtraggio degli annunci sulla base dei prefissi annunciati

```
ip prefix-list NOME SEQ permit|deny PREFISSO [ge maschera] [le maschera]
```



Sono tools per filtrare gli annunci basati sui prefissi che sono annunciati.

Una prefix list può avere molteplici linee e lo dici con SEQ.

Stiamo costruendo un set, per ogni indirizzo se diciamo permit mettiamo l'indirizzo nel set, altrimenti non lo mettiamo. Si può aggiungere "ge" (greater or equal) o "le" (lower or equal), si possono scegliere network mask maggiori o minori di 24 ad esempio.

## Arista BGP: avoid transit

```
ip prefix-list NO_C2_102 seq 10 permit 192.168.102.0/24 eq 24

route-map NO_C2_102 deny 10
 match ip address prefix-list NO_C2_102

router bgp 54501
 neighbor 172.20.1.1 route-map NO_C2_102 out
```

Dal punto di vista del router centrale, stiamo applicando per il vicino di destra la route map NO\_C2..., manipoliamo gli annunci da verso destra. Matchiamo gli IP address degli update con una prefix list chiamata NO\_C2\_102... ognuno di quelli sarà negato.

Stiamo permettendo indirizzi nella prefix list, ma stiamo negando tutti gli indirizzi nella prefix list.

Invece che filtrare in base ai prefissi, possiamo filtrare in base all'AS number. Si può creare una route map per negare il traffico, possiamo includere in questa regola tutti gli update che hanno un AS path contenente quell'AS path. Lo applico a tutti gli outgoing update dal vicino 65541

## Arista BGP: set local pref

```
ip prefix-list YES_C1_100 seq 15 permit 192.168.100.0/24 eq 24

route-map C1_LP permit 20
 match ip address prefix-list YES_C1_100
 set local-preference 245
```

Si può creare una route map come C1\_LP, permettiamo e applichiamo la route map a tutti i prefissi che matchano la prefix list e applichiamo la local preference 245. Sopra si definisce la lista di prefissi.

Match → if

Set → then

Di default in ogni route map c'è un else statement che nega tutto il resto. Implicitamente nega tutti gli update che non matchano quelli nella lista.









